Reference: how a system should behave in the real world

Purpose: a source-backed description of how each concept in this simulator works in real systems. It is written **without reading the engine's behaviour**, so it can serve as the independent oracle for the test catalogue in `../user-flows-catalogue.md`. When the simulator disagrees with a chapter here, that is a finding to triage: engine bug, wrong chapter, or a deliberate simplification.

Method

1. One **topic card** per concept (template below). Cards cite sources.
2. Source quality is marked on every claim that carries a number or rule:

Mark	Meaning
[P]	Primary: original paper, official documentation, the author's own page, or a standards text. Fetched and read.
[S]	Secondary: textbook-level summary, university notes, reputable encyclopedia. Fetched or searched.
[K]	Established knowledge written from memory, **not yet checked against a fetched source**. Verify before using as a test oracle.
[?]	Disputed, version-dependent, or implementation-specific. Needs a decision from the product owner.

3. Each card ends with **Testable expectations**: statements a flow can assert (a formula, a bound, a direction). These feed the `A`, `I` and `G` marks in the flows catalogue.
4. Each card has **Simulator scope**: what the simulator is expected to model and what it deliberately simplifies. This column is filled from the written specs and the product owner, not from the code.

Topic card template

```
### <Name>
What it is: one or two sentences.
How it works in real systems: mechanism, parameters, defaults seen in practice.
Math: formulas with every symbol defined, units, validity conditions.
Failure modes: what goes wrong and how it presents (latency, errors, saturation).
Trade-offs / when to use / when not to.
Testable expectations: bullets, each assertable.
Simulator scope: modelled / simplified / not modelled (owner-confirmed).
Sources: [P]/[S] links.
```

Chapter map

Status: `done` = written and sourced; `partial` = written, some claims `[K]`; `todo` = not started.

#	Chapter	What it covers	Status
01	Queueing and performance math	Little's Law, utilization, M/M/1, M/M/c, Erlang B/C, M/M/c/K, M/D/1, Pollaczek-Khinchine, Kingman, percentiles, golden signals, saturation	partial

#	Chapter	What it covers	Status
02	Reliability and resilience patterns	Circuit breaker, retry and backoff, jitter, timeouts, rate limiting (token/leaky bucket), bulkhead, load shedding, backpressure, health checks	partial
03	Probability and distributions	All 14 distributions in the simulator (constant, deterministic, normal, log-normal, exponential, uniform, Weibull, Poisson, binomial, gamma, beta, Pareto, empirical, mixture): pdf, mean, variance, CV, percentiles, when real latency looks like each, sampling and seeding, parameter map	partial
04	Discrete-event simulation	Event list, clock, warmup and transient bias, replications and confidence intervals, batch means, common random numbers, determinism by seed, verification, fluid/analytic versus DES, stop conditions	partial
05	Traffic and workload patterns	Constant, Poisson, bursty, diurnal, spike, sawtooth, replay; open vs closed vs partly open; coordinated omission; request mix and Zipf popularity; interactive response time law	partial
06	Compute nodes	Every `ComputeType`: vCPU and instance families, concurrency and Little's Law, workers per vCPU, memory admission, Amdahl and USL scaling, serverless concurrency and cold starts, sidecar tax, instance parameter map	partial
07	Network nodes and routing	Every `NetworkType`: L4 vs L7, load-balancing algorithms and their math (power of two choices, consistent hashing), health checks, API gateway, CDN hit-ratio arithmetic, service mesh, NAT/VPN/transit, routing parameter map	partial
08	Edges, protocols and network physics	Per-hop cost: propagation (fibre), transmission and bandwidth-delay product, TCP/TLS handshake round trips, packet loss and the Mathis bound, protocols, connection pools, batching, fan-out, path-type calibration, edge parameter map	partial
09	Storage and databases	Every `StorageType`: IOPS and throughput limits (EBS gp3/gp2), B-tree depth and WAL, LSM-tree and compaction, Bloom filters, object/block/file storage and S3 consistency, columnar and analytical stores, vector indexes (HNSW), backups and restore time, capacity arithmetic, parameter map	partial
10	Caching	Effective latency, Zipf ceilings and the Che approximation for LRU, eviction policies, cache patterns and write policies, TTL math, stampede and request collapsing (XFetch, leases), down-cache load jump, parameter map	partial
11	Replication, consistency and consensus	Async/semi-sync/sync replication and lag, consistency models, CAP and PACELC, quorums (R+W>N) and quorum availability, Raft, leader election, leases and fencing tokens, conflict resolution, parameter map	partial
12	Sharding and partitioning	Hash, range, consistent and directory schemes, shard-key choice, hot keys under Zipf and per-key limits, scatter-gather tail (E[max] = H_n x mean), rebalancing, cross-shard operations, parameter map	partial
13	Messaging and streaming	Queue, pub/sub, stream, event bus, event sourcing, broker, task queue; delivery guarantees, visibility timeout and DLQ, partitions, consumer groups and ordering, lag and backlog maths, replay and snapshots, windowing, parameter map	partial
14	Orchestration and infrastructure	Kubernetes (scheduling, bin packing), registries, config, secrets, cluster autoscaler, horizontal autoscaling math, CI/CD, IaC	todo
15	Security and identity	IAM/RBAC, WAF, firewall, bastion, CA, KMS, DLP, IdP, SIEM, rate-limit/abuse, crypto cost, auth cost	todo
16	Observability	Logging, tracing, metrics, alerting, dashboards, RUM, health checks, profiling, sampling, SLIs/SLOs/error budgets, burn rate	todo
17	DevOps and delivery	Artifact repo, build, feature flags, deployment strategies (rolling, blue-green, canary), chaos engineering, policy as code	todo
18	Data infrastructure and AI	ETL, streaming analytics, windowing, feature store, memory fabric, model serving, LLM gateway, token cost, agent orchestration, vector search	todo
19	Real-time and media	WebSocket gateway, push notifications, transcoder, signalling, SFU/MCU, WebRTC mesh, persistent connection fan-out	todo

#	Chapter	What it covers	Status
20	Integration	Webhook gateway, third-party API, payment gateway (idempotency), third-party auth	todo
21	DNS and certificates	Authoritative, internal DNS, weighted/geo/latency routing, TTL, certificate distribution	todo
22	Failure semantics and chaos	Fail-stop, crash-restart, gray failure, blackhole, hang, degraded, cascade, retry storm, thundering herd, metastable failure, blast radius, AZ outage, DB failover, traffic spike, cache stampede	todo
23	Capacity planning and cost	Headroom, N+1, utilization targets, instance sizing, reserved/spot pricing, cost per request, QuickCart arithmetic	todo
24	Architecture and system-design patterns	Layered, microservices, CQRS, event-driven, saga, outbox, fan-out on write/read, strangler, cell-based, multi-region, active-active/passive; the standard interview-style systems (URL shortener, news feed, chat, etc.) as expected-shape references	todo
25	Architecture decisions	ADR format; the trade-off tables behind each choice (SQL vs NoSQL, sync vs async, push vs pull, cache placement, consistency level)	todo
26	Configuration, JSON and types	Every field of `TopologyJSON`, node `config`, `EdgeDefinition`, workload, global, faults, question package: type, unit, range, default, real-world meaning	todo
27	Metrics and output glossary	Every output metric (percentiles, throughput, error rate, utilization, queue depth, saturation, L, W, lambda) with its exact definition and unit	todo
28	Glossary	Every technical term used in the app UI, docs and code, one line each, linked to its chapter	todo
29	Question and grading concepts	Structural rules, metric rules, invariants, rubric, discrimination, misconceptions, classroom concepts, evaluation envelope	todo

The simulator's catalogue (what chapters 06 to 21 must cover)

Taken from `src/engine/core/types.ts`. 131 component types in 14 groups.

- **ComputeType** (12): `api-endpoint`, `microservice`, `sidecar`, `batch-worker`, `serverless-function`, `faas-background`, `container`, `vm-instance`, `edge-compute`, `gpu-node`, `auth-service`, `search-service`
- **NetworkType** (17): `load-balancer`, `load-balancer-l4`, `load-balancer-l7`, `global-traffic-manager`, `edge-router`, `nat-gateway`, `transit-gateway`, `vpn-gateway`, `cdn`, `api-gateway`, `service-mesh`, `ingress-controller`, `reverse-proxy`, `high-perf-nic`, `network-policy`, `routing-rule`, `routing-policy`
- **StorageType** (19): `relational-db`, `nosql-db`, `object-storage`, `block-storage`, `distributed-file-system`, `in-memory-cache`, `search-index`, `time-series-db`, `columnar-db`, `graph-db`, `vector-db`, `data-warehouse`, `data-lake`, `kv-store`, `archive-storage`, `schema-registry`, `cdc`, `backup-service`, `kms-storage`
- **MessagingType** (7): `queue`, `pub-sub`, `stream`, `event-bus`, `event-sourcing-store`, `message-broker`, `task-queue`
- **OrchestrationType** (13): `kubernetes-cluster`, `container-registry`, `service-registry`, `tool-registry`, `config-store`, `secrets-manager`, `cluster-autoscaler`, `agent-orchestrator`, `orchestrator-scheduler`, `ci-cd-runner`, `iac-engine`, `container-runtime`, `provisioner`
- **SecurityType** (11): `iam-rbac`, `waf`, `firewall`, `bastion-host`, `certificate-authority`, `secrets-rotation`, `kms-security`, `dlp-inspection`, `identity-provider`, `siem`, `privilege-escalation-control`
- **ObservabilityType** (9): `centralized-logging`, `distributed-tracing`, `metrics-store`, `alerting-hook`, `dashboard`, `rum-monitoring`, `health-check-manager`, `safety-observability-mesh`, `profiling-service`

- **DevopsType** (7): `artifact-repository`, `build-system`, `feature-flag-service`, `deployment-controller`, `chaos-engineering-framework`, `policy-as-code`, `pipeline-secrets`
- **DataInfraType** (5): `etl-pipeline`, `streaming-analytics`, `feature-store`, `memory-fabric`, `model-serving`
- **RealTimeType** (6): `websockets-gateway`, `push-notification-service`, `transcoder`, `signaling-server`, `sfu-mcu`, `webrtc-mesh`
- **IntegrationType** (5): `webhook-gateway`, `llm-gateway`, `third-party-api-connector`, `payment-gateway`, `third-party-auth`
- **ConsensusType** (4): `etcd-consul-kv`, `leader-election`, `distributed-lock`, `coordination-service`
- **AuxiliaryType** (13): `service-mesh-telemetry`, `policy-engine`, `sharding`, `hashing`, `shard-node`, `partition-node`, `rate-limiter`, `circuit-breaker-controller`, `idempotency-manager`, `reservation-store`, `request-tracking`, `backpressure-controller`, `throttler`
- **DnsType** (3): `dns-authoritative-server`, `internal-dns`, `certificate-distro`

Behaviour traits (51 files in `src/engine/traits`)

Each is a real-world behaviour a node can carry. Chapters 06 to 22 own them as listed.

Trait	Real-world concept	Chapter
`cache`	Cache hit/miss, derived-LRU, declared hit rate	10
`circuitBreaker`	Circuit breaker	02
`retryBackoff`	Retries with backoff and jitter	02
`rateLimiter`	Token bucket / limiter	02
`bulkhead`	Bulkhead isolation, partitioned capacity	02
`loadShedding`	Load shedding	02
`autoscaler`	Horizontal autoscaling	14
`coldStart`	Serverless cold start	06
`capacityLimit`, `memoryPressure`, `storageCapacity`	Capacity and memory ceilings	06, 09
`healthAwareRouting`, `healthProber`	Health checks, outlier removal	02, 07
`contentRouting`, `keyBasedRouting`, `dnsRoutingPolicy`	Content, key and DNS routing	07, 12, 21
`geoLatency`	Region-pair latency	08
`readWriteSplit`, `readOnly`, `replication`, `consistencyModel`	Replication and consistency	11
`lockLease`, `reservationStore`, `idempotencyDedup`	Locks, reservations, dedup	11, 20
`streamBroker`, `consumerLag`, `consumerGroupMembership`, `broadcastFanout`, `ackAndRelease`, `asyncOnly`, `batching`	Messaging semantics	13
`changeStream`, `logReplay`, `windowing`	CDC, replay, windowing	13, 18
`persistentConnFanout`, `protocolSession`	Connections, sessions	08, 19
`fanoutQuery`, `tieredRetrieval`, `deliveryPartnerPool`	Scatter-gather, tiered retrieval, pools	07, 18
`telemetrySink`	Observability drop under load	16
`authentication`, `cryptoCost`, `inspectionCost`	Auth, TLS, WAF cost	15
`tokenCost`	LLM token cost	18

Trait	Real-world concept	Chapter
`externalLatency`	Third-party latency	20
`scheduler`	Kubernetes bin packing	14
`storageProfile`, `serviceTimeOverride`	Read vs scan, per-request service time	09
`capabilityModules`, `resolveTraits`, `types`	Plumbing, no real-world concept	n/a

Sources and licensing

Fetched pages are summarised, not copied. Quotes are kept under 15 words and attributed.

01. Queueing and performance math

Marks: **[P]** primary, **[S]** secondary, **[K]** from established knowledge, not yet source-checked, **[?]** needs an owner decision. See README.

Every server-like node in the simulator is a queue with `c` servers (workers), finite room `K`, a service time distribution and a timeout. This chapter is the theory behind the numbers that node reports.

1. Vocabulary and symbols

Symbol	Meaning	Unit
λ (lambda)	arrival rate: requests offered per second	req/s
μ (mu)	service rate of one server = 1 / mean service time	req/s per server
S or τ	mean service time = 1/μ	s
c	number of parallel servers (workers)	count
K	total capacity of the node: servers plus waiting room	count
ρ (rho)	utilization of the node = λ / (c·μ)	0 to 1 (can exceed 1 offered)
A	offered load = λ/μ, in "Erlangs" (average servers' worth of work)	Erlang
L	average number in the system (waiting plus in service)	count
Lq	average number waiting (not in service)	count
W	average time in the system (response time) = Wq + S	s
Wq	average time waiting before service	s
c_a^2, c_s^2	squared coefficient of variation (variance / mean²) of inter-arrival and of service time	dimensionless
P_K	blocking probability: chance an arrival finds the node full	0 to 1
X	throughput: completions per second	req/s

Kendall notation `A/S/c/K`: arrival process / service distribution / servers / capacity. `M` = Markovian (Poisson arrivals or exponential service), `D` = deterministic, `G` = general. So `M/D/1` is Poisson arrivals, constant service time, one server; `G/G/c/K` is the general model the simulator implements. [K]

2. Operational laws (hold for any run, any distribution)

These are bookkeeping identities over a measured interval. They are the best oracles because they need no assumptions about the distributions. [K] (Denning and Buzen, 1978, "operational analysis"). Sources to fetch: Harchol-Balter, *Performance Modeling and Design of Computer Systems*, chapters on operational laws.

Law	Statement	Use as a test
Utilization law	U = X · S (busy fraction = throughput x mean service time), per server group: U = X·S / c	Reported utilization must equal completions/s x mean service time / workers

Law	Statement	Use as a test
Throughput bound	X ≤ c / S	Throughput can never exceed c·μ, however high λ is
Flow conservation	arrived = completed + rejected + failed + still in system	Must balance exactly at the end of a run (the simulator's conservation check)
Forced flow law	X_i = V_i · X, where V_i = visits per request to node i	A node visited twice per request sees twice the system throughput

3. Little's Law

What it is. L = λ·W: the long-run average number in a system equals the long-run average arrival rate times the long-run average time each customer spends in it. [S] Wikipedia article on Little's law; original proof is J. D. C. Little, *Operations Research* 9(3), 1961, doi 10.1287/opre.9.3.383 [P, cited, not fetched].

Conditions.

- The system is stationary and ergodic (long-run averages exist and are finite). [S]
- `λ` must be the **effective** arrival rate: the rate at which customers actually **enter**. Where customers are turned away because the system is full, the effective rate is smaller than the offered rate. [S]
- Over a finite window the law holds only approximately; customers present at the start or still present at the end must be accounted for. [S]
- No assumption about the arrival process, service distribution, service order or number of servers. [S] (Rigorous conditions are more technical: finite means and stationarity in Little's original statement; see Wolff and Yao 2014, *Queueing Systems*, for cases with infinite waiting-time mean. [S])

Applied to the parts of a node.

- Whole node: L = λ_eff · W.
- Waiting room only: Lq = λ_eff · Wq.
- Servers only: (average busy servers) = λ_eff · S, which is the utilization law with ρ = that / c.

Pitfalls to test (real bugs found in this repository's history are listed in `docs/engine-probe-findings.md`):

- Using offered λ instead of admitted λ in a node that rejects makes a correct system look inconsistent. Admitted rate = λ·(1 − P_K).
- L must be a **time average** of the number in system, not an average of a few sampled snapshots. Snapshots taken between regular arrivals can all see an empty system. Poisson arrivals are safe because of PASTA (Poisson Arrivals See Time Averages). [K]
- Warmup transient: a short run that starts empty biases L and W low.

Worked example [S]: 9 jobs waiting plus 1 in service = 10 in the system, throughput 50 /s, so W = L/λ = 10/50 = 0.2 s. Store: 10 customers/hour, 0.5 hour each, so L = 5.

Testable expectations

- For any node over a long window: |L − λ_eff·W| / L within a small tolerance (state the tolerance per run length).
- A node that rejects: the check uses admitted arrivals; consistent overload must not raise a warning.

- Constant-rate arrivals into an under-loaded server (50 req/s, 10 ms service, ρ = 0.5): true time-average L is 0.5 (λ·S = 50 x 0.01), not 0.

4. Single-server queues

4.1 M/M/1 [K, standard result; verify in Harchol-Balter ch. 14 or Kleinrock vol. 1]

Poisson arrivals rate λ, exponential service rate μ, $\rho = \lambda/\mu < 1$.

- $P(n \text{ in system}) = (1 - \rho)\rho^n$
- $L = \rho / (1 - \rho)$, $Lq = \rho^2 / (1 - \rho)$
- $W = 1 / (\mu - \lambda) = S / (1 - \rho)$, $Wq = \rho / (\mu - \lambda) = \rho \cdot S / (1 - \rho)$
- Response time is exponential with mean W, so the p-th percentile is $-W \cdot \ln(1 - p)$. Example p99 = 4.605·W.

The $1/(1 - \rho)$ factor is the "hockey stick": at ρ = 0.5 W = 2·S; at 0.8 W = 5·S; at 0.9 W = 10·S; at 0.95 W = 20·S; as $\rho \to 1$ it diverges. This is why saturation matters long before 100% utilization (also stated in the SRE book [P]).

4.2 M/D/1 and M/G/1: Pollaczek-Khinchine [S for the form; K for the arithmetic]

For Poisson arrivals and general service with mean S and squared CV c_s^2:

$Wq = \rho / (1 - \rho) \cdot (1 + c_s^2) / 2 \cdot S$

- Exponential service: $c_s^2 = 1$, so $Wq = \rho \cdot S / (1 - \rho)$ (equals M/M/1).
- **Deterministic service (M/D/1): $c_s^2 = 0$, so $Wq = \rho \cdot S / (2(1 - \rho))$**, exactly half the M/M/1 wait. [S, derived from the form]

Worked example used in this project's probes. One server, S = 10 ms, ρ = 0.8: M/D/1 Wq = 0.8 x 10 ms / (2 x 0.2) = **20 ms**. M/M/1 Wq would be 40 ms. Mean time in system W = Wq + S = 30 ms. Average number waiting Lq = λ·Wq = 80 x 0.020 = 1.6; in service 0.8; L = 2.4. (Little's Law check.)

Testable expectations

- Single server, constant service 10 ms, Poisson 80 req/s, long run: utilization → 0.80, Wq → 20 ms, W → 30 ms, L → 2.4. A finite run starting empty will read slightly low (transient); the oracle must carry a tolerance that shrinks with run length and say which seeds were used. [?] Owner to set the tolerance. The repository's own probe saw mean wait of 16 to 19 ms across seeds, which suggests the tolerance must be about 10 to 20% on a short run.
- Doubling c_s^2 from 0 to 1 (constant to exponential service at the same mean) doubles the mean wait.

4.3 Kingman's approximation, G/G/1 [S: Wikipedia "Kingman's formula"]

$E(Wq) \approx [\rho / (1 - \rho)] \cdot [(c_a^2 + c_s^2) / 2] \cdot \tau$ (the "VUT" equation: Variability x Utilization x Time)

- Reduces to Pollaczek-Khinchine when $c_a^2 = 1$ (Poisson arrivals).
- Described as accurate near saturation. Treat it as an approximation: do not use it as an exact oracle. [S]
- Lesson used by tests: **variability, not just utilization, drives waiting.** Same ρ and mean service, higher c_s^2 (heavy-tailed service) gives proportionally longer waits.

5. Multi-server queues

5.1 M/M/c and Erlang C [S: TU Berlin queueing text; Wolfram reference]

Offered load $A = \lambda/\mu$, utilization $\rho = A/c < 1$.

- Probability an arrival has to wait (all c servers busy), Erlang C: C(c, A) = [(A^c / c!) · 1/(1 − ρ)] / [Σ_{k=0}^{c−1} A^k/k! + (A^c / c!) · 1/(1 − ρ)]
- Mean wait in queue: $Wq = C(c, A) / (c\cdot\mu - \lambda)$ [S]
- Mean time in system: $W = 1/\mu + C / (c\cdot\mu - \lambda)$ [S]
- Mean number waiting: $Lq = C\cdot\rho / (1 - \rho)$ [S]
- Given that a customer waits, the wait is exponential with mean $1/(c\cdot\mu - \lambda)$. [S]
- Relation to Erlang B: $C = c\cdot B / (c - A\cdot(1 - B))$. [S]

Why this matters: **pooling**. One pool of c servers with a shared queue waits far less than c separate single-server queues with the same total load. This is the case for a load balancer plus shared queue versus random assignment. [K]

5.2 Erlang B (loss system, M/M/c/c) [S/K]

No waiting room. `B(c, A) = (A^c / c!) / (sum of A^k / k! for k = 0 to c)`. Stable recursion: B(0) = 1, $B(n) = A\cdot B(n-1) / (n + A\cdot B(n-1))$. [K]

5.3 M/M/c/K, finite capacity [S for the properties; K for the formula]

Capacity $K \geq c$ (servers plus waiting slots). Unnormalised state weights:

- $p_n = A^n / n!$ for $0 \leq n \leq c$
- $p_n = (A^c / c!) \cdot (A/c)^{(n-c)}$ for $c < n \leq K$
- `P_K = p_K / (sum of p_n for n = 0 to K)` (blocking probability)

Properties [S]:

- Always stable: the finite buffer caps the population, so a steady state exists even if $\rho > 1$.
- Effective (admitted) arrival rate $\lambda_{eff} = \lambda\cdot(1 - P_K)$; throughput $X = \lambda_{eff}$; offered load beyond capacity is rejected.
- M/M/1/K closed form: $P_K = (1 - \rho)\cdot\rho^K / (1 - \rho^{(K+1)})$ for $\rho \neq 1$, and $1/(K + 1)$ for $\rho = 1$, with $\rho = \lambda/\mu$ (the standard M/M/1/K result; the hand-computed example below checks it). [S]
- c = K reduces to Erlang B. [S]

Overloaded finite node, worked example from the project's probe (this is arithmetic you can check by hand): c = 1, S = 10 ms (μ = 100/s), K = 5, offered λ = 300/s, so ρ = 3. A = 3. $p_n = 3^n$ for n = 0..5 gives 1, 3, 9, 27, 81, 243. Sum = 364. $P_K = 243/364 =$ **0.668**. Admitted rate $\lambda_{eff} = 300 \times (1 - 0.668) =$ **99.7 /s**, almost exactly the server's maximum μ = 100/s. Utilization ≈ 0.997. Average number in system $L = \Sigma\, n\cdot p_n / 364 = (0 + 3 + 18 + 81 + 324 + 1215) / 364 = 1641 / 364 =$ **4.51**. Mean time in system $W = L / \lambda_{eff} = 4.51 / 99.7 =$ **45 ms**. (This is the M/M/1/K version with exponential service. The project's probe used deterministic service, so its exact numbers differ; the shape, a nearly full node and rejecting about two thirds of arrivals, holds.) [K, hand-computed from the formula above]

Testable expectations

- Throughput never exceeds $c\cdot\mu$. Rejections $\approx \lambda - c\cdot\mu$ when $\lambda \gg c\cdot\mu$ and K is small.

- $\lambda_{eff} \times W = L$ must hold with λ_{eff} (admitted), not λ (offered).
- Conservation: offered = admitted + rejected, at every node.
- Timeouts: a request whose wait plus service exceeds the timeout is counted as a timeout, not a success; throughput then falls below $c \cdot \mu$. [K]

6. Latency distributions, percentiles and what to report

- Report percentiles (p50, p95, p99, p99.9), not just the mean. A 100 ms mean at 1,000 req/s can hide 1% of requests taking about 5 s. [P: Google SRE book, "Monitoring Distributed Systems"]
- Record latency as a histogram with exponentially spaced buckets. [P: same]
- Track latency of successful and failed requests **separately**; a fast error lowers the mean while hurting users. [P]
- **Do not average percentiles** across nodes or windows; merge the histograms and compute the percentile from the merge. [K]
- The tail comes from queueing and variability. Parallel fan-out makes it worse: if one request waits on n backends in parallel and each is slow with probability q, the request is slow with probability $1 - (1 - q)^n$. With q = 1% and n = 100 that is 63%. [K, Dean and Barroso, "The Tail at Scale", CACM 2013, to fetch]
- End-to-end latency along a chain = sum over hops of (queue wait + service + network). Means add; percentiles do not. [K]
- Coordinated omission: a closed-loop load generator that waits for each response before sending the next hides the worst latencies. An open-loop generator (arrivals independent of responses) is what the simulator's sources should be. [K, Gil Tene]

7. The four golden signals [P: Google SRE book]

Signal	Definition	Simulator output it maps to
Latency	Time to service a request, tracked separately for success and failure	latency percentiles, failure latency
Traffic	Demand on the system, e.g. requests per second	arrival rate, throughput
Errors	Explicit (HTTP 500), implicit (200 with wrong content), or by policy (slower than a committed limit)	error rate, timeouts, rejections
Saturation	How full the service is, measured on its most constrained resource; many systems degrade well before 100%	utilization, queue depth, drops

Further SRE guidance [P]: page on symptoms (what users feel), use causes to debug; monitor the whole distribution.

8. Utilization, headroom and capacity arithmetic

- Capacity of a node = $c \cdot \mu = c / S$. Maximum safe load = target utilization x capacity. [K]
- Why headroom (typically 60 to 80%): wait grows as $1/(1 - \rho)$ (section 4.1), variability amplifies it (section 4.3), and you need spare for failures (N+1) and spikes. [K]
- N+1: with n servers sharing load evenly, after one failure each survivor carries $n/(n - 1)$ of its previous load. 4 servers at 75% become 100% when one dies. [K]

- **QuickCart worked example** (from this repository's `builder-walkthrough.md`; a stated modelling contract, not an external source): per-server capacity = workers / service time = 128 / 0.00128 s = 100,000 req/s. Peak 1,000,000 req/s with a hard utilization limit of 0.8 needs at least 1,000,000 / (0.8 x 100,000) = 12.5, round **up** to **13** servers. 13 servers: load per server 76,923 req/s, utilization 76.9%. 12 servers: 83,333 req/s, 83.3%, over the limit. 8 servers: 125,000 req/s, 125%, capacity 800,000, so about 200,000 req/s (20%) is dropped. Rule: **always round up; never round down.**

9. Discrete-event simulation and statistical honesty [K, to expand in chapter 04]

- A single seeded run is one sample, not the truth. Compare to theory only with a tolerance, or average several seeds.
- Discard a warmup period so the empty-start transient does not bias averages. State the warmup used.
- Same seed, same inputs must give identical output (determinism). A different seed must give a different sample with the same expected value.
- Confidence interval for a mean from n independent replications: mean ± t(n−1, 0.975)·s/√n. [K]

10. Open items for this chapter

- Fetch and cite: Little (1961); Harchol-Balter textbook chapters (operational laws, M/M/1, M/G/1); Kleinrock vol. 1; Dean and Barroso, *The Tail at Scale*; Gil Tene on coordinated omission. Upgrade the `[K]` marks above to `[P]`.
- Owner decision [?]: numeric tolerance policy for comparing a finite seeded run with a closed-form value.
- Owner decision [?]: which node service-time distributions the simulator treats as "Poisson arrival" versus constant-rate arrivals when applying the P-K formula, so the oracle matches the intended arrival process of each workload pattern.

Sources

- Little's law, Wikipedia [S]
- Kingman's formula, Wikipedia [S]
- M/M/c, M/M/1/K and Erlang models, TU Berlin course text [S]
- Google SRE book: Monitoring Distributed Systems [P]
- Wolff and Yao 2014, Little's Law with infinite waiting times [S]
- Little 1961, A proof for the queuing formula L = λW [P, cited only]
- Technion lecture notes on G/G queues [S]

02. Reliability and resilience patterns

Marks: **[P]** primary, **[S]** secondary, **[K]** from established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

Patterns that decide what a node does when it is slow, failing or overloaded. In the simulator they are the behaviours carried by `timeout`, `retryBackoff`, `circuitBreaker`, `rateLimiter`, `bulkhead`, `loadShedding`, `healthAwareRouting` and `healthProber`.

1. Timeouts

What it is. A limit on how long a caller waits for a response. [P: AWS Builders' Library, "Timeouts, retries and backoff with jitter"]

How it works in real systems [P]

- Set one on **every** remote call, covering connection and request time, even for calls on the same host.
- Choose it from the downstream latency distribution: pick an acceptable false-timeout rate (example 0.1%) and use the matching percentile (p99.9). Add worst-case network latency for internet clients.
- A too-tight timeout causes failures after deploys (example: a 20 ms timeout included new TLS setup).
- A dead server usually produces **silence**, not an error: the caller burns its whole timeout. This is why latency during an outage walls at the timeout instead of dropping. [K]

Math. If a downstream's latency has CDF F(t), timeout T gives false-timeout rate 1 – F(T). Timeout at p99 means 1% of healthy requests are abandoned. [K]

- Client availability when median latency equals the timeout is about 50%. [P: AWS load-shedding article]

Testable expectations

- A request that outlives its timeout is counted as a timeout and its client-observed latency equals the timeout.
- Blackholed/hung node: failed-request latency clusters at the timeout (p50 of failures ≈ T). A `reject` failure returns fast (about one network hop).
- Timeout raised above the downstream p99.9: false timeouts → about 0.

Simulator scope. Per-node and global `defaultTimeout`. Deadline propagation is [?].

2. Retries, backoff and jitter

What it is. Re-sending a failed call, usually after a delay, to ride out transient faults. [P: Microsoft Retry pattern; AWS]

Strategies [P: Microsoft]: cancel (non-transient), retry immediately (rare fault), retry after delay (busy or connectivity). Delays can be fixed, incremental or exponential. Log early failures as info and only the last as an error.

When not to retry [P]: long-lasting faults, business-logic errors, and as a substitute for scaling. Retry only **idempotent** operations; otherwise a retry can execute the side effect twice (service processed the request but the response was lost). Client 4xx errors should generally not be retried unchanged; 5xx may succeed later.

Delay formulas (base b, cap C, attempt n starting at 0). The AWS blog shows the formulas as images, so these come from knowledge of that post. [K, verify against the AWS Architecture Blog figures]

- No jitter: sleep = $\min(C, b \cdot 2^n)$
- Full jitter: sleep = $\text{random}(0, \min(C, b \cdot 2^n))$
- Equal jitter: $t = \min(C, b \cdot 2^n)$; sleep = $t/2 + \text{random}(0, t/2)$
- Decorrelated jitter: sleep = min(C, random(b, 3·previous_sleep))

Finding [P: AWS blog text]: with 100 contending clients, jitter cut total calls by more than half versus un-jittered exponential backoff and improved completion time; un-jittered was worst on both; Full and Equal used about the same calls; Decorrelated used more calls. Jitter does not change the N^2 growth of total work but reduces it substantially.

Retry amplification (the dangerous part)

- Retries are "selfish": they spend more server capacity to raise one caller's success chance. [P: AWS]
- **Per layer:** with up to r retries, worst-case attempts per call = 1 + r.
- **Across layers multiplies:** 3 attempts per layer over 5 layers = 3^5 = **243x** load on the database. Retry at one point in the stack. [P: AWS]
- Expected attempts per call when each attempt fails independently with probability p and up to r retries are allowed: E[attempts] = `p^0 + p^1 + ... + p^r` = (1 − p^(r+1)) / (1 − p). Example p = 0.5, r = 3: 1.875x load. If p → 1 it tends to 1 + r. [K derived]
- **Retry budget:** allow retries from a token bucket (spend a token per retry, refill slowly); when empty, retry at a fixed low rate. This caps amplification. [P: AWS, the AWS SDK added it in 2016]

Failure modes: retry storm (synchronized retries into a recovering service keep it down), thundering herd, metastable failure (load stays high after the trigger is gone because retries sustain it). [K]

Testable expectations

- Arrivals at a failing downstream ≤ (1 + r) x calls from upstream, and equal that when the downstream is fully down.
- With jitter, retry times of N clients are spread; without, they cluster at $b \cdot 2^n$.
- Non-idempotent operation behind a retry with a lost response produces duplicates unless an idempotency key is used.
- A circuit breaker above a retrying client reduces downstream arrivals after it opens.

3. Circuit breaker

What it is. A wrapper that stops calling a failing dependency so it can recover and the caller fails fast. [P: Martin Fowler]

States and transitions [P: Fowler]

- **Closed:** calls pass; failures are counted. A success resets the count (in the simple version).
- **Open:** calls fail immediately; the protected operation is not invoked.

- **Half-open:** after a reset timeout, a trial call is allowed. Success closes the breaker; failure reopens it and restarts the timeout.

Trip conditions [P]: a failure **count** threshold, or a failure **rate** (for example 50%); different thresholds for different error types; not every error should count. For async work it can trip when a request queue fills; for sync work when a thread pool is exhausted.

Variants [S]: some implementations reopen on any single failed probe; others require N probe successes before closing; the number of trial calls allowed in half-open varies. [?] Owner to fix which the simulator means.

Operational advice [P: Fowler]: log every state change, expose state, alert on trips and resets, treat frequent trips as a deeper problem, allow manual trip/reset, and design clients for the failure (fallbacks: queue work, show stale data).

Caveat [P: AWS]: breakers add modal behaviour that is hard to test and can lengthen recovery.

Math. Fail-fast latency while open ≈ near zero (no call is made), versus a timeout T for each call without a breaker. Load on the dependency while open ≈ 0 plus probe calls. With rate-based tripping over a window of n calls and threshold θ, it opens when failures/n ≥ θ. [K]

Testable expectations

- Backend fully down: after threshold failures the breaker opens; subsequent client latency is far below the timeout and arrivals at the backend drop to about the probe rate.
- After the reset timeout one probe goes through; the backend recovered → closed and traffic resumes; still down → open again.
- Breaker never opens on errors below the threshold (steady 5% error vs 50% threshold stays closed).

4. Rate limiting and throttling

What it is. Controlling how fast requests are sent or accepted so a service stays within capacity. Rate limiting (client or gateway, smoothing outgoing traffic) and throttling (server rejecting excess) are related. [P: Microsoft Rate Limiting pattern]

Algorithms [S: network and API references; formulas derived [K]]

- **Token bucket:** bucket capacity C tokens, refill rate R tokens/s. tokens(t) = min(C, tokens(to) + R·Δt). A request of cost n is admitted only if tokens ≥ n. Allows bursts up to C, long-run rate R. Maximum burst length at peak rate M > R from a full bucket: S = C / (M − R). Total sent in the burst = M·S.
- **Leaky bucket:** output fixed at rate R; queue depth Q(t) = max(0, Q(to) + arrivals − R·Δt), capped at the bucket size; overflow is dropped. Smooths traffic, no bursts out.
- **Fixed window:** count per window, reset at boundaries. Weakness: up to 2x the limit across a boundary.
- **Sliding window (log / counter):** counts over a moving window; avoids the boundary burst at higher memory cost.
- Worked check [S: a GATE exam example]: C = 1 MB, R = 10 MB/s, M = 20 MB/s, burst 12 MB: S = 1/(20 − 10) = 0.1 s sends 2 MB at full rate, the remaining 10 MB at 10 MB/s takes 1 s, total 1.1 s.

Behaviour [P: Microsoft]

- Naive retry-on-error against a throttled service wastes capacity. Example: 10,000 records, service accepts 2,000 per pass: passes of 10,000, 8,000, 6,000, 4,000, 2,000 = **30,000 sends** for 10,000 records (3x) plus 20,000 errors to log.
- Limit by operation count, data volume, or cost units; release in small frequent slices (for example 20 ops every 200 ms for a limit of 100/s) rather than one big batch per second.
- Respond with HTTP 429 and `Retry-After`; coordinate retries with the limiter to avoid retry storms.
- Not suitable when the bottleneck is concurrency or CPU rather than request rate.

Testable expectations

- Offered rate λ below limit R: no rejects. Above: admitted ≈ R (plus the initial burst of C), rejected ≈ λ − R over the long run.
- Token bucket with C tokens, full at start, offered a burst: first C requests admitted at once.
- Leaky bucket: output rate never exceeds R regardless of input.
- Limit of 0: everything rejected (and flagged as a misconfiguration).

5. Bulkhead

What it is. Isolate resources into pools so one failing or hungry consumer or dependency cannot take the rest down. Named after ship hull compartments. Also called cell-based architecture. [P: Microsoft Bulkhead pattern]

How it works [P]

- Partition by consumer load or availability requirement; give each downstream its own connection or thread pool; separate queues per workload; separate instances per tenant.
- Benefits: contains cascading failure, keeps partial function, allows different quality of service per consumer.
- Costs: less efficient resource use (spare capacity in one pool can't help another), more complexity, partition design.
- Combine with retry, circuit breaker and throttling. Use platform limits (Kubernetes CPU/memory limits) rather than re-implementing them.
- Not suitable when lower resource efficiency is unacceptable.

Math. Total capacity c split into partitions $c_1 \ldots c_k$ ($\Sigma c_i = c$). A class with offered load A_i saturates when $A_i \geq c_i \cdot$(target ρ), without taking capacity from others. Cost of isolation: statistical multiplexing lost, so the waiting probability at the same total c is higher than a shared pool (Erlang C, chapter 01 section 5). [K]

Testable expectations

- Overload one class: its latency rises and rejections appear in its partition only; other classes' latency stays flat.
- With no bulkhead the same overload degrades all classes.
- Idle capacity in one partition is **not** used by another (that's the trade-off).

6. Load shedding and goodput

What it is. Rejecting excess requests early, near overload, so accepted requests are served fast. [P: AWS Builders' Library, "Using load shedding to avoid overload"]

Key ideas [P]

- **Throughput** = total request rate sent. **Goodput** = the part handled correctly and quickly enough for the client to use.
- Without shedding, past an inflection point latency rises above client timeouts, requests fail, and the work already done on them is wasted, so **goodput falls as throughput rises**. With shedding, goodput stays roughly flat (shedding has its own cost).
- Choose what to shed by priority: health-check pings first kept, humans over crawlers, later steps of a flow over first steps, within-quota over burst.
- Deadlines: clients send timeout hints; servers drop requests already past their deadline.
- Queues: bound size and max wait; discard stale work; consider LIFO; prefer fast-fail spillover to deep surge queues.
- Proxy max-connection limits are a last resort, set high and below listener threads or file descriptors so health checks pass.
- Keep fast-failed requests out of latency metrics, or they will hide the real latency; shedding can also stop CPU-based autoscaling.
- In deep call graphs retries multiply offered load exponentially.

Backpressure [K]: the downstream signals upstream to slow (429/503, a full queue, TCP window, reactive streams `request(n)`), moving the queue to where it can be bounded.

Testable expectations

- Offered load far above capacity: with shedding, accepted-request p99 stays near unloaded p99 and goodput ≈ capacity; without shedding, p99 → timeout and goodput collapses toward 0.
- Shed requests are counted as rejections, not as latency samples.
- Conservation: offered = served + shed + failed + in flight.

7. Health checks, outlier detection and health-aware routing

What it is. Probing instances so routers and orchestrators stop sending traffic to unhealthy ones. [P: Microsoft Health Endpoint Monitoring pattern]

How it works [P]

- **Shallow (liveness):** process responds, no dependency calls; failing it restarts the process.
- **Deep (readiness):** verifies required dependencies; failing it removes the instance from traffic without restart. Exclude optional dependencies from the decision. Allow for startup before counting failures.
- Evaluate status code, body and latency. Use thresholds (consecutive failures) so transient blips don't trigger action.
- **Detection delay** ≈ probe interval x consecutive failures needed (+ one timeout). With a 1-minute interval and 3 failures, a dead instance keeps receiving traffic for about 2 to 3 minutes. [K derived; matches the Microsoft example of one-minute probes]
- Cached probe results add a staleness window up to the cache TTL.
- **Danger:** deep checks against a slow shared dependency can mark every instance unhealthy at once and remove all of them (a cascading failure caused by monitoring). Mitigate with dependency timeouts, circuit breakers, or "degraded" status. The platform may refuse to remove all instances. [P]
- Failover needs redundant capacity sized to absorb rerouted traffic. [P]

Testable expectations

- A failed backend keeps receiving a share of traffic until the detection delay passes; those requests fail or time out. After removal, the survivors carry n/(n − 1) of their previous load (chapter 01 section 8).
- Recovery: the backend rejoins after the configured number of consecutive successes. [?] Owner to fix this.
- All backends unhealthy: the router's documented behaviour (serve anyway, or reject) must be what the UI shows. [?]

8. How the patterns combine (expected interactions)

Combination	Expected behaviour
Retry above a breaker	After the breaker opens, retries stop reaching the backend; the caller sees fast failures.
Retry above a limiter	Retries consume limiter budget; honour `Retry-After` or the storm continues.
Retry at every layer	Load multiplies (1 + r)^layers.
Timeout shorter than downstream p99	Self-inflicted errors plus wasted downstream work.
Breaker + health checks	Two removal mechanisms; the faster one (breaker, per caller) acts before the slower probe (shared).
Bulkhead + shedding	Shed within each partition by priority; one partition's overload stays local.
Autoscaling + shedding	Shedding can hide CPU load and block scale-out unless the scale signal includes rejects. [P: AWS]

9. Where the simulator differs from this chapter

The chapter describes real systems. These are the known places the simulator is simpler. Each has a tracked test in `e2e/journeys/ch02-reliability.spec.ts`; delete the note when its test starts passing.

#	Chapter says	Simulator does today	Tracked by
1	Retries cover transient faults, including timeouts (section 2)	Retries fire only for `node_failed`, `node_error_rate`, `circuit_breaker_open`, `lock_contended` and `connection_reset`. A timeout (blackholed backend) is **not** retried, so a retry policy gives no protection against a silent dead server.	`R-02c` (parked, decision needed)
2	A configured circuit breaker stops calls to a failing dependency (section 3)	It gates only on `sidecar`, `service-mesh` and `circuit-breaker-controller`. On other nodes the setting is accepted but inert. The validator now warns at import.	`R-03d` (warning is live)
3	Detection delay is about probe interval x consecutive failures (section 7)	A failed backend is removed from routing instantly. No traffic reaches it after the fault.	`R-07c` (expected failure)
4	Retries multiply across layers (section 2): 3 layers of 4 attempts send 4^3 = 64 calls to a failing dependency	Only the node directly upstream of the failure retries; a second and third layer with the same policy add nothing, so the retry-storm hazard cannot arise.	`F-05` (expected failure)

Teaching consequence: a student who adds retries and then kills the backend silently will see no amplification in the simulator, while real systems amplify. Treat results for those scenarios as optimistic

until rows 1, 3 and 4 are closed.

10. Open items

- Verify `[K]` formulas against AWS Architecture Blog jitter figures, and against a token-bucket reference (RFC 2697 / 2698 single/two-rate three-colour marker for the networking form).
- Fetch Nygard, *Release It!* (circuit breaker, bulkhead, timeouts, "stability patterns") and the Resilience4j documentation for default parameter values.
- Owner decisions [?]: half-open probe count and close criterion; deadline propagation; behaviour when all backends are unhealthy; whether the simulated limiter is token bucket or leaky bucket.

Sources

- Martin Fowler, CircuitBreaker [P]
- AWS Builders' Library, Timeouts, retries and backoff with jitter [P]
- AWS Builders' Library, Using load shedding to avoid overload [P]
- AWS Architecture Blog, Exponential backoff and jitter [P, text only; formulas were images]
- Microsoft, Retry pattern [P]
- Microsoft, Bulkhead pattern [P]
- Microsoft, Rate Limiting pattern [P]
- Microsoft, Health Endpoint Monitoring pattern [P]
- Google SRE book, Monitoring Distributed Systems [P]
- Token bucket, Loyola University networking text [S]

03. Probability and distributions

Marks: **[P]** primary, **[S]** secondary (encyclopedia or course text, fetched), **[K]** established knowledge, not yet source-checked, **[?]** needs an owner decision. See README.

Every service time, edge latency and arrival gap in the simulator is a random draw from a distribution. This chapter says what each of the 14 distributions is, what it is for, and what its mean, spread and percentiles must be, so a draw can be checked against theory.

1. What to know about any distribution

Quantity	Meaning	Why the simulator cares
Mean E[X]	long-run average	Sets capacity: mu = 1 / mean service time
Variance, standard deviation	spread	Drives queue waits (Kingman, chapter 01 section 4.3)
Coefficient of variation CV = sd / mean	spread relative to the mean; CV squared is the variability term	CV = 0 constant, CV = 1 exponential, CV > 1 heavy and bursty
Median and percentiles	the value below which p of draws fall	Latency is judged by p95, p99, p99.9, not the mean
Support	which values are possible	Service times cannot be negative
Quantile function Q(p)	inverse of the CDF	Used both to read percentiles and to sample

Rules that hold for every distribution [K]:

- The mean of a sum is the sum of the means. The variance of a sum of **independent** terms is the sum of the variances. Percentiles do **not** add (chapter 01 section 6).
- Multiplying a draw by a constant c multiplies the mean and sd by c, leaves CV unchanged, and multiplies every percentile by c.

2. How a draw is made

Inverse-transform sampling [S: Wikipedia]. If U is uniform on [0, 1] and F is the target CDF, then $X = F^{-1}(U)$ has CDF F, where $F^{-1}(u)$ is the smallest x with $F(x) >= u$. Steps: draw u, apply the inverse CDF, return x. Closed forms exist for the exponential, Weibull, Pareto and uniform. Others use specialised methods (Box-Muller for the normal, rejection or multiplication methods for gamma, beta, binomial and Poisson). [K]

Pseudo-random generators [K]. A seeded generator gives a reproducible stream of uniforms: same seed, same sequence, same run. Good generators pass statistical test batteries; the simulator seeds a small-state generator from a string hash (the code names sfc32 and xmur3). Whether it passes the standard batteries (TestU01 / PractRand) is **[?]**.

Testable expectations

- Same seed and same config produce identical draws. A different seed produces different draws with the same mean and spread.

- Over N draws the sample mean is within about 3 standard errors of the true mean: error is roughly 3 x sd / sqrt(N). This is the right tolerance to attach to every sampling check below.
- Heavy-tailed distributions (Pareto alpha <= 2, log-normal with large sigma) converge slowly; their sample means are noisy even at large N, so use quantiles (median, p99), not the sample mean, as oracles for them.

3. The catalogue

For each: parameters, pdf, moments, percentiles, real-world use, and the simulator's parameter name. Parameter names are taken from the config type; the formulas are the oracle.

3.1 Constant and deterministic

- Every draw equals `value`. Mean = value, variance 0, CV 0. [K]
- Real use: idealised fixed cost, a fixed timer, a calibrated mean when variability is not the point.
- Effect in queues: lowest wait for a given load. M/D/1 waits half as long as M/M/1 (chapter 01 section 4.2).
- Simulator: `constant` and `deterministic` are the same thing.

3.2 Uniform

- Parameters `min`, `max`. pdf = 1 / (max − min) on [min, max]. Mean = (min + max) / 2. Variance = (max − min)² / 12. [K]
- Worked: min 10, max 30: mean 20, variance 33.3, sd 5.77, CV 0.289. Median 20. p99 = 10 + 0.99 x 20 = 29.8.
- Real use: jitter ("0 to 100 ms of random delay"), no knowledge beyond the bounds. Rarely a good latency model.

3.3 Exponential

- Parameter rate lambda (simulator `lambda`, units per millisecond when the value is in ms, so mean = 1 / lambda ms).
- pdf = lambda e^(−lambda x), CDF = 1 − e^(−lambda x). Mean = 1/lambda, variance = 1/lambda², **CV = 1**. [S: Wikipedia]
- Quantile: Q(p) = −ln(1 − p) / lambda. Sampling: x = −ln(U) / lambda. [S]
- **Memoryless:** P(T > s + t | T > s) = P(T > t). The time left does not depend on time already waited. [S]
- Models the gaps in a Poisson process (section 3.7 and chapter 05). Its memorylessness makes M/M queues solvable in closed form.
- Worked: mean 20 ms (lambda = 0.05 per ms): median 13.9 ms, p95 59.9 ms, p99 92.1 ms, p99.9 138.2 ms. The p99 is 4.6 x the mean. [computed]
- Real use: mean service time when little else is known; time to failure of a part with no wear-out. Real latencies are usually less variable in the body and heavier in the tail than exponential.

3.4 Normal

- Parameters `mean`, `stdDev`. pdf = exp(−(x − mean)² / (2 sd²)) / (sd sqrt(2 pi)). [S]
- 68% within 1 sd, 95% within 2, 99.7% within 3. Quantile = mean + sd x z, with z = 1.645 (p95), 1.96 (two-sided 95%), 2.326 (p99), 2.576 (two-sided 99%), 3.09 (p99.9). [S]

- **Support is all reals, so it can produce negative durations.** With mean 20 and sd 10, P(X < 0) = 2.3%. A negative service time is meaningless; a model must truncate or clamp it. Simulator: negative draws are clamped to 0 for service time (so the realised mean is slightly above the stated mean). **[?]** Owner to confirm.
- Real use: sums of many independent small delays (central limit theorem), measurement noise. Poor model for skewed latency.

3.5 Log-normal

- ln X is normal with parameters `mu` and `sigma` (of **ln X**, not of X). [S: Wikipedia]
- Mean = exp(mu + sigma²/2). Median = exp(mu). Variance = (exp(sigma²) − 1) exp(2 mu + sigma²). **CV = sqrt(exp(sigma²) − 1)**, depends only on sigma. mode < median < mean (right-skewed). Quantile Q(p) = exp(mu + sigma x z_p). [S]
- Worked: median 20 ms (mu = ln 20 = 2.996), sigma 0.5: mean 22.66 ms, CV 0.533, p95 45.5 ms, p99 64.0 ms, p99.9 93.8 ms. [computed]
- Real use: the best everyday model for request latency and file sizes: positive, skewed, tail a few times the median, from products of many independent factors. [S]
- Reading config: to hit a target median m use mu = ln m; a target p99 = m x exp(2.326 sigma) fixes sigma.

3.6 Gamma

- Parameters shape k and scale theta. Mean = k theta, variance = k theta², **CV = 1 / sqrt(k)**. [S: Wikipedia]
- k = 1 is exponential (rate 1/theta). Integer k is **Erlang**: the sum of k independent exponentials each with mean theta. [S]
- Worked: k = 4, theta = 5: mean 20, variance 100, CV 0.5. A service made of 4 sequential exponential stages of mean 5 ms each has exactly this distribution.
- Real use: multi-stage processing; waits composed of several phases. Use k to dial CV below 1 smoothly (k = 2: CV 0.71; k = 4: 0.5; k = 16: 0.25).

3.7 Poisson (counts)

- A **discrete** distribution of counts: P(X = k) = lambda^k e^(−lambda) / k!, mean = variance = lambda. [S: Wikipedia]
- Link to the Poisson process: if arrivals in [0, t] are Poisson with mean lambda t for every t, then inter-arrival times are independent exponentials with mean 1 / lambda. [S]
- Superposition: independent Poisson streams add; the sum is Poisson with the summed rate. Thinning: keeping each event with probability p gives a Poisson process of rate lambda p, and the split streams are independent. [S]
- Normal approximation: good for lambda above about 10 with a continuity correction. [S]
- **Using a count as a duration is a modelling choice.** A Poisson draw is an integer, with CV = 1 / sqrt(lambda). Check what unit the node means before using it as a service time. **[?]**

3.8 Binomial (counts)

- n trials each succeeding with probability p. Mean = n p, variance = n p (1 − p). [K]
- Worked: n = 10, p = 0.3: mean 3, variance 2.1. As n grows with n p fixed it approaches Poisson. [K]
- Real use: number of failures among n replicas, number of cache hits in a batch.

3.9 Weibull

- Parameters shape k and scale lambda. CDF = 1 − exp(−(x/lambda)^k). Mean = lambda Gamma(1 + 1/k). Median = lambda (ln 2)^(1/k). Quantile Q(p) = lambda (−ln(1 − p))^(1/k). Variance = lambda² (Gamma(1 + 2/k) − Gamma(1 + 1/k)²). [S: Wikipedia]
- k = 1 is exponential with rate 1/lambda. k = 2 is Rayleigh. [S]
- **Hazard rate** h(x) = (k/lambda)(x/lambda)^(k−1): falling for k < 1 (early failures), constant at k = 1, rising for k > 1 (wear-out). [S]
- Worked: k = 2, lambda = 20: mean 17.72, variance 85.8, median 16.65, p99 42.9. [computed]
- Real use: time to failure and repair; service times with a hard lower shoulder.

3.10 Pareto (Type I)

- Parameters scale x_m (the minimum possible value) and shape alpha. CDF = 1 − (x_m / x)^alpha for x >= x_m. [S: Wikipedia]
- Mean = alpha x_m / (alpha − 1) **only if alpha > 1**; infinite otherwise. Variance finite **only if alpha > 2**; infinite for 1 < alpha <= 2. Median = x_m 2^(1/alpha). Quantile = x_m (1 − p)^(−1/alpha). [S]
- Power-law tail: smaller alpha is a heavier tail. The n-th moment is infinite when alpha <= n. [S]
- Worked: alpha = 2.5, x_m = 10: mean 16.67, variance 222, CV 0.89, median 13.2, p99 63.1, p99.9 158.5. [computed]
- **Config trap.** The simulator names the fields `shape` (alpha) and `scale` (x_m). Mixing them up gives a valid but wrong distribution. Check with the median: x_m x 2^(1/alpha).
- Real use: file sizes, job sizes, request sizes, think times: a few huge items carry much of the load. With alpha <= 2 sample means never settle, which makes a queue fed by it unpredictable.

3.11 Beta

- Support [0, 1]. Parameters alpha, beta. Mean = alpha / (alpha + beta). Variance = alpha beta / ((alpha + beta)² (alpha + beta + 1)). Mode (alpha − 1) / (alpha + beta − 2) for alpha, beta > 1. [S: Wikipedia]
- alpha = beta = 1 is uniform; alpha = beta symmetric about 0.5. [S]
- Real use: a **proportion** (hit ratio, error fraction), not a duration. To use as a time, scale the draw. Used to vary a probability per run. **[?]** what the simulator uses it for.

3.12 Mixture

- A weighted combination of component distributions: draw a component with probability w_i, then draw from it. Weights are non-negative and sum to 1. [S: Wikipedia]
- Mean = sum w_i mu_i. Variance = sum w_i (sigma_i² + mu_i²) − mu². [S]
- Models **bimodal** latency: a fast path and a slow path (cache hit versus miss, warm versus cold start, local versus cross-region).
- Worked: 90% fast (1 ms), 10% slow (50 ms), both constant: mean 5.9 ms, variance 216.1, sd 14.7, **CV 2.49**, CV² 6.2. The p89 is 1 ms and the p99 is 50 ms. A queue fed by this has a Pollaczek-Khinchine wait factor (1 + CV²)/2 = 3.6 x the M/M/1 wait (chapter 01 section 4.2). Lesson: **the mean says 5.9 ms and nothing actually takes 5.9 ms.**
- **Weight rule.** The type says weights should sum to 1. The simulator normalises by the total, so weights 9 and 1 work like 0.9 and 0.1. **[?]** Owner to decide whether a sum other than 1 should be a validation warning.

3.13 Empirical

- Draw from observed data. Two modes in the config: `step` (resample the recorded values) and `linear` (interpolate between them). [K]
- Step: the CDF is the empirical CDF; draws are only the recorded values. Linear: draws can fall between recorded values, which smooths the tail but can never exceed the recorded maximum.
- Real use: replaying a production latency sample. Caveat: a sample has no tail beyond its largest value, so p99.99 of the model equals the sample max, not reality. [K]
- Mean of draws converges to the sample mean (step) or very close to it (linear).

4. Choosing a distribution (what real latency looks like)

Situation	Typical shape	Use
Fixed work, no variability	CV 0	constant
Memoryless service, unknown structure	CV 1	exponential
Several sequential stages	CV < 1	gamma (Erlang), integer shape = number of stages
Typical request latency, skewed with a tail	CV 0.3 to 1, right-skewed	log-normal
Fast path plus slow path	bimodal, CV > 1	mixture
Job or object sizes, heavy tail	power law	Pareto (watch alpha)
Time to failure or repair	hazard varies	Weibull
Network jitter with a known bound	flat	uniform
A measured production sample	whatever it is	empirical

Guide to CV [K]: CV < 1 smoother than Poisson (waits shorter than M/M); CV = 1 exponential; CV > 1 burstier, and the tail dominates waiting.

5. Simulator parameter map (config reference)

Config `type`	Fields	Unit	Real parameter
`constant`, `deterministic`	`value`	ms	the value
`uniform`	`min`, `max`	ms	bounds
`exponential`	`lambda`	per ms	rate; mean = 1/lambda ms
`normal`	`mean`, `stdDev`	ms	mean, sd
`log-normal`	`mu`, `sigma`	ln(ms)	parameters of ln X
`gamma`	`shape`, `scale`	ms	k, theta
`weibull`	`shape`, `scale`	ms	k, lambda
`pareto`	`shape`, `scale`	ms	alpha, x_m
`beta`	`alpha`, `beta`	unitless	proportion on [0, 1]
`poisson`	`lambda`	count	mean count
`binomial`	`n`, `p`	count	trials, success probability
`empirical`	`samples`, `interpolation` (`linear` or `step`)	ms	recorded values

Config `type`	Fields	Unit	Real parameter
`mixture`	`components[{weight, distribution}]`	per component	w_i, component

6. Testable expectations (per distribution)

For N = 200,000 seeded draws, with tolerance about 3 sd / sqrt(N) on the mean and a few percent on quantiles:

- Mean, sd and CV equal the formulas above (exponential 20 ms, log-normal median 20 ms sigma 0.5, gamma k 4 theta 5, Weibull k 2 lambda 20, uniform 10 to 30, mixture 90/10).
- Percentiles: exponential p99 = 92.1 ms for mean 20; log-normal p99 = 64.0 ms for median 20 sigma 0.5.
- Pareto alpha 2.5, x_m 10: median 13.2 and p99 63.1 (use quantiles, not the mean).
- No draw below x_m (Pareto), below min (uniform), or below 0 (service times).
- Mixture proportions: about 90% of draws land in the fast mode.
- Determinism: identical seed gives an identical stream.
- A service time distribution with CV c gives a queue whose mean wait scales with $(1 + c^2)/2$ relative to M/M (chapter 01 test Q-07).

7. Open items

- Fetch and cite: a statistics text for the quantile and moment formulas marked `[K]` (uniform, binomial); the sfc32 and xmur3 generators and their test-battery results.
- Owner decisions `[?]`: normal-draw truncation and clamping (3.4); Poisson and beta as durations (3.7, 3.11); mixture weights that do not sum to 1 (3.12).
- The distribution-sampling flows are in `e2e/journeys/ch03-distributions.spec.ts`; the oracles are in this chapter.

Sources

- Exponential distribution, Wikipedia [S]
- Normal distribution, Wikipedia [S]
- Log-normal distribution, Wikipedia [S]
- Gamma distribution, Wikipedia [S]
- Poisson distribution, Wikipedia [S]
- Weibull distribution, Wikipedia [S]
- Pareto distribution, Wikipedia [S]
- Beta distribution, Wikipedia [S]
- Mixture distribution, Wikipedia [S]
- Inverse transform sampling, Wikipedia [S]

04. Discrete-event simulation

Marks: **[P]** primary, **[S]** secondary (encyclopedia, course notes, fetched or searched), **[K]** established knowledge, not yet source-checked, **[?]** needs an owner decision. See README.

How the simulator produces its numbers, and how much to trust one run. Chapter 01 gives the answers a queue should converge to; this chapter says when a finite run is entitled to be close to them.

1. What a discrete-event simulation is

Definition [S: Wikipedia]. The state of the system changes only at discrete instants called events. Between events nothing happens, so the simulation clock jumps from one event time to the next instead of ticking.

Component	Role
State	Variables that describe the system: queue lengths, busy servers, in-flight requests. A step function of time.
Clock	Current simulated time. It only moves forward, to the time of the next event.
Event list	The pending-events set, a priority queue ordered by time. Each entry has a time and a type.
Random-number generators	Supply the stochastic inputs: inter-arrival times, service times, routing choices, failures.
Statistics	Counters and integrals updated at every event (counts, sums, time-weighted areas).

The loop (next-event time advance) [S].

1. Remove the earliest event from the list.
2. Advance the clock to its time.
3. Run its handler: update state and statistics.
4. Schedule follow-up events, usually after a sampled delay.
5. Repeat until the stop condition is met.

The alternative, **fixed-increment** time advance, splits time into small slices and processes each. It costs work for slices in which nothing happens, so next-event is faster for queueing systems. [S]

Example of events for one request through a server: `request-arrival` (join queue or start service) then `service-complete` (free the worker, start the next waiter, forward the response). A failure injects `node-failure` and `node-recovery` events at configured times. [K]

Testable expectations

- Time never goes backward: every recorded event timestamp is >= the previous one.
- Between two events the state does not change: queue length and busy-worker count are constant over the gap.
- Event count grows with load: roughly proportional to arrivals x (hops x events per hop).

2. Time, units and ties

- The unit of time must be fixed once. The simulator stores time as integer **microseconds**, so no floating-point drift accumulates over long runs. [K]
- Two events can have the same timestamp. The order in which they run must be defined and repeatable (for example by a priority class and then insertion order), otherwise runs with the same seed can diverge. **[?]** Owner to document the tie-break order.
- A service time of 0 ms is legal (a pass-through). It must still be processed as a separate event at the same instant, never skipped.

3. Randomness, seeds and reproducibility

- Every random choice comes from a pseudo-random stream, so a run is a deterministic function of (topology, seed). Same inputs give identical output. [K]
- **Independent streams.** If arrivals, service times and routing all draw from one stream, changing one decision (adding a node) shifts every later draw and changes unrelated outcomes. Separate streams per purpose keep comparisons clean. **Observed:** the simulator draws service times from one shared stream at the moment service starts, so a design with more workers starts services earlier and pairs different draws with different requests. Pairing seeds then does not shrink the noise in a comparison (tracked as `E-13`).
- **Common random numbers (CRN) [S].** To compare two designs, drive both with the same random inputs so a difference in output comes from the design, not from sampling noise. It needs the streams to be synchronised: the same draw must be used for the same purpose in both runs. It is often effective but its benefit is not guaranteed (Glasserman and Yao 1992), and there is no way to know the variance reduction without running both ways. [S]
- A single seeded run is **one sample**, not the truth. Different seeds give different samples whose spread is the sampling error (section 6).

Testable expectations

- Same seed, same topology: byte-identical summary. Different seed: different summary, same expectation.
- Replaying with the same seed after adding an unrelated, disconnected node should not change results if streams are independent; if it does, the generator is a single shared stream. **[?]**

4. Terminating versus steady-state questions

Two different questions, with different ways to answer [K]:

Question	Example	What to do
Terminating: how does the system behave over this specific window	"a 5-minute flash sale starting from empty"	Run the real window from its real starting state. No warmup. Report that window.
Steady-state: long-run average behaviour	"what is mean latency at 80% load"	Discard the start-up transient (warmup), then average.

Using steady-state formulas (chapter 01) to judge a short terminating run is the most common error. A 5-second run starting empty has not reached steady state at high load.

5. Warmup, transient bias and run length

- **Initialisation bias.** A run that starts empty under-counts queue length and waiting time early on, so the averages are biased low. Two fixes: start in a representative state, or run a **warmup** period and delete its data. [S: output-analysis lecture notes]
- **How big is the bias?** At rho 0.9 (M/M/1, mu = 100 per second) requests arriving in the first 5 s from empty wait 71.9 ms on average, against 90 ms in steady state: about 20% low. A separate Monte-Carlo of 20,000 replications gives this; a single 5 s run has a spread of about 40 ms, so hundreds of replications are needed to see it. [computed]
- **How long to warm up.** Pick the warmup from pilot runs: Welch's method averages several replications position by position and the analyst chooses the point where the curve flattens. MSER-5 is an automatic rule that picks the truncation point minimising the confidence-interval width on batch means of size 5. The literature notes many rules have not been thoroughly tested, so check the choice case by case. [S]
- **Rule of thumb.** The transient dies out over a few **relaxation times**. For an M/M/1 queue the relaxation time is about 1 / (mu (1 – sqrt(rho))²). [K, to verify in Kleinrock or Harchol-Balter] At mu = 100 per second this gives 0.12 s at rho 0.5, 0.90 s at rho 0.8, and 15.6 s at rho 0.95. Warmup must grow sharply with utilisation; a fixed 5 s warmup is fine at 80% load and nowhere near enough at 95%. [computed]
- **Run length.** After warmup the measured window must hold enough completed requests that the estimate is stable. At rho 0.6 with 20 ms service, 600 s gives about 18,000 requests. Heavy-tailed service or rho near 1 needs far more.
- **Autocorrelation.** Consecutive waits are positively correlated (a busy period raises many in a row), so N completed requests carry much less information than N independent samples. This is why raw standard errors from one run are too optimistic. [K]

Testable expectations

- A short run from empty reads low versus theory; lengthening the run or adding warmup moves the result toward the closed form (chapter 01 test Q-01 uses 600 s and 30 s warmup).
- The reported "measured window" excludes warmup: measured seconds = duration – warmup.
- A warmup-adequacy check warns when the first part of the window still trends.

6. Output analysis: how sure am I?

For a mean of an output such as average wait [S: lecture notes; formulas K]:

Independent replications. Run n times with different seeds (independent streams). Let xbar be the mean of the n results and s their sample standard deviation. A (1 – a) confidence interval is xbar ± t(n–1, 1–a/2) x s / sqrt(n). Each replication discards its own warmup.

- Illustrative: n = 10, s = 1.3 ms, t(9, 0.975) = 2.262: half-width = 2.262 x 1.3 / sqrt(10) = **0.93 ms**.
- To reach a half-width h, n is about (t s / h)². For h = 0.5 ms with the same s, n is about 30. [computed]

Batch means (one long run). After warmup, split the output into k contiguous, non-overlapping batches of equal length, average each, treat batch means as roughly independent, and apply the same t-interval with k – 1 degrees of freedom. Batches must be long enough that adjacent batch means are nearly uncorrelated. [S]

Rules for comparisons. To decide whether design A beats B, either use CRN and a paired interval on the differences, or use enough independent replications that the intervals separate. Never compare two

single runs and call the smaller number "better". [K]

Testable expectations

- Across 10 seeds of the Q-01 scenario the measured mean waits scatter around the closed form, and the 95% interval contains it in about 95% of repeated experiments.
- Standard error falls as 1 / sqrt(replications).
- A test oracle with a tolerance should be set from this spread, not guessed. Record the replication count and the half-width next to every tolerance. **[?]** Owner to fix the policy.

7. Verification and validation

- **Verification:** does the code do what the model says? Check against closed forms (chapter 01), conservation (arrived = completed + rejected + in flight), determinism, and limiting cases (very low load means no queueing; one server means M/M/1). [K]
- **Validation:** does the model resemble reality? Compare against measured production data and against known real-world behaviour (chapter 22 failure semantics). [K]
- **Invariants worth checking on every run** [K]: time never decreases; no negative queue; busy workers <= c; sum of per-node arrivals respects the topology; utilization in [0, 1]; Little's Law on admitted traffic (chapter 01 section 3).

8. Discrete-event versus fluid (analytic) evaluation

	Discrete-event	**Fluid / analytic**
Unit of work	one request	a continuous rate
Cost	proportional to the number of events	proportional to nodes + edges
Captures	queues, jitter, timeouts, retries, breakers, every trait	steady-state throughput, utilisation, saturation, drops
Fails at	very high rates (millions of events)	anything per-request: tails, retries, bursts

- A fluid model gives, for a node with capacity C and offered rate lambda: carried = min(lambda, C), dropped = max(0, lambda − C), utilisation = lambda / C (may exceed 1 as an offered ratio). [K]
- **The two paths must agree where both apply.** At moderate rates where both are affordable, utilisation and throughput from the fluid model must equal the discrete-event result within noise. This is a strong self-check. The classroom parity tests already do this for utilisation.
- QuickCart at 1,000,000 req/s is evaluated analytically: 8 servers give utilisation 1.25 and 20% loss; 13 servers give 76.9% (chapter 01 section 8).

Testable expectations

- Discrete and analytic agree on utilisation (within a few percent) and on throughput for a stable system at a rate both can run.
- Above the switch threshold the run reports it was analytic, and per-request outputs (traces, tails) are absent or marked unavailable.

9. Stop conditions

- **Duration:** stop at simulated time T. Requests still in flight at the cutoff are neither completed nor failed; they must be reported separately (not silently dropped). [K]

- **Request budget:** generate exactly N source requests, then drain in-flight work. Used for "the sale is N requests".
- **Early halt on saturation:** stop when utilisation or error rate crosses a threshold, so a doomed design reports its verdict at the moment it breaks.

Testable expectations

- Request-budget run generates exactly N requests.
- The in-flight count at the cutoff plus completed plus failed plus rejected equals arrived.
- Early halt: the run ends no later than the first moment the threshold is crossed.

10. Simulator scope

Modelled: next-event loop in integer microseconds, a seeded generator, per-node queues (c servers, finite capacity K), warmup, request-budget and saturation stop conditions, an analytic fall-back above an event-count ceiling. Open **[?]**: tie-break order, independent streams, how warmup length is validated, whether a confidence interval is ever reported to the user.

11. Open items

- Fetch and cite: Law and Kelton, *Simulation Modeling and Analysis* (output analysis, CRN); Kleinrock volume 1 (M/M/1 transient and relaxation time); Welch (1983) on the warmup method.
- Owner decisions: tolerance policy for comparing a seeded run with a closed form (section 6); stream independence (section 3); tie-break order (section 2).

Sources

- Discrete-event simulation, Wikipedia [S]
- Simulation output analysis lecture notes (initialisation bias, Welch, batch means) [S]
- Warm-up methods survey, Warwick [S]
- Variance reduction, Wikipedia (common random numbers) [S]
- Glasserman and Yao, Some guidelines and guarantees for common random numbers, Management Science 1992 [S, cited]

05. Traffic and workload patterns

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge, not yet source-checked, **[?]** needs an owner decision. See README.

What the traffic into a design looks like, how to describe it with numbers, and how each of the simulator's seven patterns should behave. Queueing results (chapter 01) are only as good as the arrival process they are fed.

1. Describing an arrival process

Term	Meaning
Arrival rate lambda	average requests per second over a window
Inter-arrival time	gap between consecutive requests; mean 1 / lambda
Inter-arrival CV (c_a)	sd / mean of the gaps. 0 = perfectly even, 1 = Poisson, > 1 = bursty. The simulator reports it as "Offered Arrival CV".
Peak-to-mean ratio	peak rate / average rate; sizes capacity for bursts
Burstiness / index of dispersion	variance of counts per window divided by the mean count; 1 for Poisson, > 1 for bursty
Arrival process vs service process	independent inputs of a queue; Kingman's formula uses c_a^2 and c_s^2 (chapter 01 section 4.3)

Total arrivals over a window = integral of the rate. For a piecewise-constant rate that is the sum of rate x duration. [K]

2. The seven patterns

2.1 Constant

- Evenly spaced requests: gap = 1 / lambda exactly. c_a = 0, no randomness.
- Total in T seconds = lambda T. Example 100 req/s for 30 s = 3,000 requests.
- Use: idealised baseline, deterministic debugging, isolating service-time variability.
- Real traffic is almost never this smooth; results will read **better** than reality (lower waits at the same load).

2.2 Poisson

- Independent exponential gaps with mean 1 / lambda. Counts in any window are Poisson with mean lambda x window; c_a = 1; variance of counts = mean. [S: Wikipedia]
- Memoryless: the chance of an arrival in the next instant does not depend on the past. Superposition of independent Poisson streams is Poisson with summed rate; random thinning keeps it Poisson. [S]
- The standard model for many independent users acting at random (many small sources). It is the arrival assumption behind M/M/c and M/G/1 formulas.
- Test: count per 1 s window has mean lambda and variance about lambda (index of dispersion near 1); gaps have mean 1/lambda and CV near 1.

2.3 Bursty (on/off)

- Alternates between a **burst** phase at `burstRps` for `burstDuration` and a **normal** phase at base rate for `normalDuration`, repeating. [K, config fields below]
- Mean rate = (burstRps x burstDuration + baseRps x normalDuration) / (burstDuration + normalDuration). Worked: burst 400 rps for 1 s, normal 50 rps for 3 s: mean = (400 + 150) / 4 = **137.5 rps**; peak-to-mean 2.91. [computed]
- Counts per 1 s window alternate 400, 50, 50, 50 (plus noise), so variance between windows is about 22,969 against a Poisson variance of 137.5: the index of dispersion is about 168. This is far burstier than Poisson. [computed]
- Related models: the **on/off source** and the **Markov-modulated Poisson process (MMPP)**, a Poisson process whose rate switches according to a Markov chain; the on/off case is a special case with the off rate often zero. [S: network-traffic-model surveys]
- **Why it matters.** Real network and request traffic is usually burstier than Poisson, at every time scale: Leland, Taqqu, Willinger and Wilson showed Ethernet traffic is self-similar, with no natural burst length, and that aggregating sources does not smooth it. [S: Leland et al., IEEE/ACM ToN 1994] A queue fed by bursty traffic has higher waits than Poisson at the same mean rate (large c_a^2 in Kingman's formula).
- The on/off model has a **fixed** burst length; real burstiness has heavy-tailed durations. Treat results as a lower bound on real burstiness. **[?]**
- **Simulator behaviour (observed):** arrivals inside each phase are **evenly spaced**, not Poisson: per-second counts are exactly 400, 50, 50, 50, 400, ... So a burst only builds a queue when the burst rate exceeds the node's capacity. A 160 req/s burst against 200 req/s of capacity never waits at all; a 400 req/s burst does.

2.4 Diurnal

- Rate varies by hour of day: rate(h) = baseRps x multiplier(h), with 24 hourly multipliers and a `peakMultiplier`. [K, config]
- **Simulator behaviour (observed):** the 24 hourly values are stretched across the whole run, so one simulated day = the run duration. A 60 s run covers all 24 hours at 2.5 s per hour; the first hour's rate applies for the first 2.5 s.
- Daily mean = baseRps x average of the 24 multipliers. Example: multipliers $1 + 0.5 \sin(2\pi (h - 9) / 24)$: mean 1.0, peak 1.5, peak-to-mean 1.5.
- Real daily cycles span roughly 2x to 10x between trough and peak for consumer services, and have weekly and event-driven (campaign, holiday) layers. [K]
- Test: arrivals in hour h are proportional to multiplier(h); the total over the run = baseRps x duration x mean multiplier. Because the day is stretched over the run, results for diurnal runs describe the shape of a day, not real-time seconds.

2.5 Spike

- Base rate with a window starting at `spikeTime` lasting `spikeDuration` at `spikeRps`. [K, config]
- Total = base x T + (spikeRps − base) x spikeDuration. Worked: base 100 rps, spike to 500 for 5 s in a 60 s run = 6,000 + 400 x 5 = **8,000** requests, average 133.3 rps. [computed]
- Real analogues: a flash sale, a viral link, a retry storm after an outage, a cache expiry herd. The chaos "traffic spike" preset multiplies the rate for a window.
- What to watch: queue depth builds during the spike at (spike rate − capacity) per second, and drains afterwards at (capacity − base rate) per second. Recovery time after the spike = backlog / (capacity −

base) where backlog = (spike − capacity) x duration. Example: capacity 300, base 100, spike 500 for 5 s: backlog = 200 x 5 = 1,000 requests, drain time = 1,000 / 200 = **5 s**. [computed; K]
- If the buffer K is smaller than the backlog, the excess is rejected instead of queued.

2.6 Sawtooth

- Rate ramps linearly from the **base rate** (`baseRps`) up to `peakRps` over `rampDuration`, then resets to the base, repeating. [observed]
- Mean over whole ramps = (base + peak) / 2. Example: base 10, peak 200, ramp 10 s: mean 105 rps, 6,300 requests over 60 s. (A ramp from 0 would average peak / 2 = 100.) [computed]
- Use: find the load at which a design breaks (the knee). Record the time at which error rate or latency first exceeds a threshold, then read the rate at that time: rate(t) = peak x t / ramp. [K]
- Also models a slow ramp-up of traffic, such as a launch.

2.7 Replay

- Re-issue a recorded arrival trace (timestamps and types). The only pattern with real burstiness and correlation structure built in. [K]
- Mean rate and request mix equal the trace's. Reproducible for a fixed trace. Limit: cannot extrapolate to a rate or length the trace does not cover.

3. Open, closed and partly open workloads

This is the most common silent error in load testing and simulation. [S: Schroeder, Wierman and Harchol-Balter, NSDI 2006]

Model	Rule	Behaviour under overload
Open	New requests arrive independently of completions, at rate lambda	Offered load stays at lambda; queues grow without bound if lambda > capacity. Realistic for the public internet with many independent users.
Closed	A fixed population of N users; each waits for a response, thinks for Z, then sends again	Self-limiting: slower service means fewer requests. Never overloads. Realistic for a fixed pool of workers or terminals.
Partly open	Users arrive openly, then make several dependent requests in a session	Between the two.

The paper reports that the choice changes results a lot (scheduling policies, response times), and gives eight principles; the choice must match how the real system is used. [S: abstract only fetched]

- **Interactive response time law (closed systems) [S: Jain; Denning and Buzen].** R = N / X − Z, where N users, X throughput, Z think time. It follows from Little's Law applied to the whole loop. Worked: 12 terminals, Z = 20 s, X = 0.5 per second: R = 12 / 0.5 − 20 = **4 s**. With Z = 0 it reduces to Little's Law. Throughput of a closed system is capped: X <= N / (Z + R_min).
- **Coordinated omission [S: Gil Tene].** A load generator that waits for each response before sending the next hides the worst latencies, because the requests that would have been sent during a stall are never sent. The fix is to score each request against when it **should** have been sent. A simulator source with independent arrivals (an open model) avoids it by construction.
- **Test.** In an open source, offered rate equals the configured rate however slow the server is. In a closed source, offered rate falls as response time rises.

4. Request mix, size and key popularity

- `requestDistribution`: a list of request types with weights summing to 1 and a size in bytes; each request gets a type by weighted choice. Expected share of type i = w_i. [K]
- **Zipf popularity (hot keys).** When a keyspace has skew s, the key of rank r is drawn with probability proportional to r^(−s); the chance of the rank-r key is r^(−s) / H(N, s) where H is the sum of r^(−s) over the N ranks. [K] For N = 1,000 keys: with s = 1.0, H = 7.485, so the top key gets **13.4%** of requests, the top 10 get **39.1%**, the top 100 get **69.3%** (uniform would give 0.1%, 1%, 10%). With s = 0.8: top key 6.5%, top 10 23.1%, top 100 52.6%. [computed] Consequence: a cache holding the top 100 keys serves about 69% of requests with s = 1.0. This is why a small cache earns a high hit rate and why one hot partition saturates first (chapter 10, chapter 12).
- Size matters for bandwidth and transfer time: transmission delay = size / bandwidth (chapter 08).
- Sessions: a session id groups one client's requests so reads see their own writes and sticky routing works (chapter 11).

5. How workload shape drives queueing (link to chapter 01)

- At the same mean rate, higher inter-arrival CV gives longer waits: Wq ~ (c_a² + c_s²) / 2 x rho / (1 − rho) x S (Kingman). Constant arrivals (c_a = 0) wait less than Poisson (c_a = 1); bursty (c_a > 1) waits more. [S]
- A capacity plan must cover the **peak** window, not the mean. A design at 70% of capacity on average that sees a 2.9x burst is over capacity for the burst's duration, and builds a backlog it needs time to drain (section 2.5).
- Headroom for variability is the reason for the 60% to 80% target utilisation in chapter 01 section 8.

6. Simulator parameter map

Field	Meaning	Unit
`pattern`	`constant`, `poisson`, `bursty`, `diurnal`, `spike`, `sawtooth`, `replay`	n/a
`baseRps`	base rate	req/s
`bursty.burstRps`, `.burstDuration`, `.normalDuration`	burst rate and the two phase lengths	req/s, ms
`diurnal.peakMultiplier`, `.hourlyMultipliers[24]`	daily curve	multiplier, per hour
`spike.spikeTime`, `.spikeRps`, `.spikeDuration`	spike start, rate, length	ms, req/s, ms
`sawtooth.peakRps`, `.rampDuration`	ramp top and length	req/s, ms
`requestDistribution[]`	`type`, `weight` (sums to 1), `sizeBytes`, optional `metadata`, optional `keyspace{field,size,skew}`	n/a
`stopCondition`	`duration` or `requestBudget` (`maxRequests`), optional `haltOnSaturation`	n/a
`origins`, `sessions.count`	client populations and sessions	n/a

7. Testable expectations

Over a 60 s run at seed S, with tolerance about 3 sd / sqrt(N) on counts:

- **Constant** 100 rps: exactly 6,000 arrivals; inter-arrival CV = 0.
- **Poisson** 100 rps: arrivals 6,000 +/- 3 x sqrt(6,000) = +/- 232; per-second counts have variance about 100; inter-arrival CV about 1 (the app shows 0.99 for 80 rps).
- **Bursty** 400 rps for 1 s then 50 rps for 3 s: mean 137.5 rps; per-second counts alternate high and low; index of dispersion far above 1.
- **Spike** base 100, 500 for 5 s: total 8,000; the rate in the spike window is 5x base.
- **Sawtooth** base 10, peak 200, ramp 10 s: mean 105 rps; the rate at the middle of a ramp is about 105 and it resets to the base every 10 s.
- **Diurnal:** arrivals per hour proportional to the multipliers.
- **Request mix:** observed type shares within sampling error of the weights.
- **Zipf (s = 1, N = 1,000):** top-10 keys receive about 39% of requests.
- **Open versus closed:** offered rate is constant in an open source whatever the server speed.
- **Request budget stop:** exactly N source requests generated.
- Warmup excluded: measured arrivals = rate x (duration – warmup).

8. Simulator scope

Seven patterns, weighted request types, Zipf keyspaces, sessions and a request-budget stop. Sources are **open** (arrivals independent of completions). A closed or partly open source is not modelled. **[?]** How simulated time maps to the hours of the diurnal curve, and whether replay accepts a user trace, are open decisions **[?]**.

9. Findings from the tests

- The backlog arithmetic of section 2.5 holds: base 100, spike 500 for 5 s into a 200 req/s node queues about 1,500 requests (peak queue within 10%) and recovers about 15 s after the spike ends.
- Zipf shares match 1/H, H(10)/H and H(100)/H to within 1 percentage point for s = 1.0 and s = 0.8.

10. Open items

- Fetch and cite: the full Schroeder et al. paper (the eight principles); Fischer and Meier-Hellstern on the MMPP; a source for the real daily-peak ratios (section 2.4).
- Add a closed-population source (N users, think time Z) if teaching closed-loop behaviour matters; then the interactive response time law is directly testable.
- Owner decisions `[?]`: diurnal time mapping; whether to model heavy-tailed burst durations.

Sources

- Poisson distribution, Wikipedia [S]
- Exponential distribution, Wikipedia [S]
- Schroeder, Wierman, Harchol-Balter: Open versus closed: a cautionary tale, NSDI 2006 [P, abstract]

- Leland, Taqqu, Willinger, Wilson: On the self-similar nature of Ethernet traffic, IEEE/ACM ToN 1994 [P]
- Survey of network traffic models, Jain [S]
- Operational laws and the interactive response time law, Jain [S]
- Denning, The science of computing: operational analysis [S]
- Gil Tene, Coordinated omission, Mechanical Sympathy thread [P]
- Dean and Barroso, The Tail at Scale, CACM 2013 [P, abstract]

06. Compute nodes

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 12 `ComputeType`s: `api-endpoint`, `microservice`, `sidecar`, `batch-worker`, `serverless-function`, `faas-background`, `container`, `vm-instance`, `edge-compute`, `gpu-node`, `auth-service`, `search-service`. This chapter covers what a compute node is in a real system, how its capacity is sized, and the math that connects hardware, concurrency and throughput. Queueing results are in chapter 01.

1. What a compute node is

A node that takes a request, spends CPU time and usually waits on something else (a database, a cache, a downstream service), then replies. Capacity is set by **how many requests it can hold in flight** (concurrency) and **how long each holds a slot** (service time). [K]

Real thing	Typical shape	Simulator type
Stateless HTTP/gRPC service	many cheap threads or an event loop, mostly waiting on I/O	`microservice`, `container`, `vm-instance`
Entry point that generates traffic	not capacity-bound; the source of a workload	`api-endpoint`
Proxy attached to a service (Envoy-style)	small, per-request tax on every call	`sidecar`
Queue consumer, cron, ETL step	pulls work, usually CPU-bound, throughput matters more than latency	`batch-worker`, `faas-background`
Function-as-a-service	one request per environment, scales out to many, cold starts	`serverless-function`
Code at the CDN edge	tiny, close to users, short CPU budget	`edge-compute`
Accelerator for model inference	few, expensive, batch-friendly	`gpu-node`
Authentication and search	CPU-heavy per request	`auth-service`, `search-service`

2. Hardware: vCPU, memory and instance families

- A **vCPU is a hardware thread**, not a core: vCPUs = cores x threads per core. An `m5.large` is 1 core x 2 threads = **2 vCPU**, 8 GiB. [P: AWS EC2 instance types]
- M5 sizes: `m5.large` 2 vCPU / 8 GiB, `m5.xlarge` 4 / 16, `m5.2xlarge` 8 / 32 (4 GiB per vCPU, "general purpose"). [P]
- **Burstable (T3):** a baseline CPU level that can burst above it using CPU credits; once credits run out the instance is throttled to baseline. [P: AWS; credit accounting K]
- Families trade CPU, memory and price: compute-optimised (c5) has faster cores and 2 GiB per vCPU; memory-optimised (r5) 8 GiB per vCPU; general purpose (m5) 4 GiB per vCPU. [K]
- Price is roughly linear in vCPU within a family, so doubling size doubles price. [K]
- Purchase model changes price, not performance: on-demand (full), reserved or savings plan (about 40% off for a one-year commitment is typical), spot (deep discount, can be reclaimed). [K; AWS quotes up to roughly 72% and 90% off, verify]

Simulator instance table (these are the config facts; the formulas below are the oracle):

Family	Example	vCPU	RAM	Relative core speed (`perfFactor`)	Price/h
burstable	`t3.medium`	2	4 GB	0.8	0.042
general	`m5.large`	2	8 GB	1.0	0.096
compute-optimised	`c5.large`	2	4 GB	1.3	0.085

3. Concurrency: the one formula that sizes everything

Little's Law applied to a service (chapter 01 section 3): concurrency = arrival rate x time in system.

- AWS Lambda states it directly: **Concurrency = (average requests per second) x (average request duration in seconds)**. [P: AWS Lambda docs]
  - 100 req/s at 1 s each = 100. At 500 ms each = 50. 200 req/s at 250 ms = 50. 5,000 req/s at 200 ms = **1,000**. [P]
- Servers needed = concurrency / concurrency per server. **QuickCart check:** 100,000 req/s x 1.28 ms = **128** concurrent requests, exactly the 128 workers of one server unit. [computed, from the repo's authored contract]
- A node saturates when concurrency demand reaches its workers: capacity = workers / service time. [K]

3.1 How many workers per vCPU?

The question is how many requests one core can juggle. It depends on how much of each request is CPU and how much is waiting.

Sizing formula [S: attributed to Goetz, *Java Concurrency in Practice*; secondary sources only]:

threads = cores x (1 + wait time / service time) (service time here means CPU time)

- Pure compute (wait about 0): threads = cores: **one worker per core**. [S]
- Mostly waiting: many threads per core. Example: 4 cores, 90 ms waiting, 10 ms CPU: 4 x (1 + 9) = **40** threads; with a 0.8 target CPU utilisation, 32. [computed]
- The simulator's two profiles are the two ends of this formula: **cpu-bound = 1 worker per vCPU**; **io-bound = 32 workers per vCPU**, which corresponds to wait/CPU = 31, a request that is about 3% CPU and 97% waiting. [computed from the repo constants] The formula is a starting point: sources caution that there are no ideal numbers and the answer must be measured. [S]
- Events (Node.js, async runtimes) reach high concurrency on few threads because waiting costs no thread. [K]

Testable expectations

- A cpu-bound node on an `m5.large` has 2 workers; an io-bound node has 2 x 32 = 64; with `instanceCount` n multiply by n.
- Max throughput = workers / mean service time: cpu-bound 2 workers at 10 ms = 200 req/s (chapter 01 test Q-04 style).
- Doubling instance count doubles capacity (with perfect scaling; section 5 says when that fails).

3.2 Admission limit K and memory

A node can also run out of **memory** before it runs out of workers: every in-flight request holds memory. Max in-flight K = RAM / memory per request. [K]

- 8 GB at 16 MB per request = 512 in flight; at 2,048 MB per request = 4. A small K forces rejections under overload (chapter 01 section 5.3, flow Q-05).

4. Service time on different hardware

- A faster core serves each request in less time: service time = base time / performance factor. A `c5` core (1.3) turns 10 ms into 7.7 ms for CPU-bound work. [K]
- I/O-bound work barely benefits from a faster core because most of its time is spent waiting. The simulator damps the speed-up for I/O-bound nodes to 25% of the advantage: effective factor = 1 + (perf – 1) x 0.25. [repo constant; physics K]
    - Worked: c5 (1.3) io-bound: 1 + 0.3 x 0.25 = 1.075, so a 10 ms request becomes 9.30 ms. This is exactly the 9.28 ms the UI showed in the finding behind chapter-01's capacity fix.
- **Contention:** if the number of runnable workers exceeds physical cores, CPU-bound work slows because workers time-share cores. Running 64 io-bound workers on 2 vCPUs is fine only because most are waiting. [K]

5. Scaling out and its limits

5.1 Amdahl's law [P: Wikipedia]

If a fraction p of the work can run in parallel on N processors: **speedup = 1 / ((1 − p) + p / N)**; the ceiling is 1 / (1 − p) however large N gets. [S]

- p = 0.95: N = 8 gives 5.93x, N = 32 gives 12.55x, N = 96 gives 16.7x, and the limit is **20x**. [computed]
- It assumes a fixed workload and ignores other bottlenecks (memory, I/O). [S]

5.2 Universal Scalability Law [K, Gunther; Wikipedia page named it but gave no formula]

capacity(N) = N / (1 + alpha (N − 1) + beta N (N − 1)); alpha is contention (serial queueing), beta is coherency (crosstalk cost that grows with N^2). With beta > 0 throughput **peaks and then falls**, at N* = sqrt((1 − alpha) / beta).

- alpha = 0.05, beta = 0.0005: N* = 43.6; relative capacity 5.8 at N = 8, 10.5 at 32, 10.7 at the peak (43.6), 9.3 at 96, 6.5 at 200. [computed]
- Lesson: more machines past the peak make things **worse**, not just no better. Shared locks, a single database, or all-to-all coordination are the usual sources of beta.

5.3 Horizontal vs vertical

- **Scale up** (bigger instance): simple, no coordination, bounded by the largest instance. **Scale out** (more instances): near-linear for stateless work, needs a load balancer (chapter 07), limited by Amdahl and USL terms above. [K]

- Failure: n identical servers each carry n / (n − 1) of the load after one fails (chapter 01 section 8). Capacity plans need N + 1. **Observed (`C-08`):** four servers at 60 req/s each become three at 80 req/s each after one fails, within 3% of the arithmetic.
- **A measurement trap (observed, `C-02` and `C-07`):** throughput measured over a run with a deep queue and a timeout is not the capacity. If the queue can hold more work than the timeout allows, the server spends its capacity on requests that have already expired and successes per second collapse (chapter 02 section 6, goodput versus throughput). Measure capacity with a short queue or read steady-state completions per second.

Testable expectations

- instanceCount 1 to 4 on a stateless cpu-bound node: capacity 4x (no shared bottleneck in the simulator unless one is modelled).
- Adding instances behind a single-worker database stops helping once the database saturates (the shared downstream is the alpha term).

6. Serverless

- A function runs in an **execution environment**, one request at a time; a new environment is created when none is free. [P: AWS Lambda docs]
- **Cold start:** creating an environment runs an Init phase (load code, start runtime, run initialisation) before the first request, so that request is slower. Warm requests reuse the environment. [P]
- Mitigations: **provisioned concurrency** keeps environments pre-initialised (no cold starts for that many concurrent requests, extra cost); **SnapStart** restores from a snapshot (startup "as low as sub-second", supported for some runtimes). [P]
- **Limits [P]:** default account concurrency 1,000 per region; request rate capped at **10 x** the concurrency quota (10,000 req/s by default), so a 20 ms function at 30,000 req/s needs 600 concurrency but is throttled by the request-rate cap; new environments are added at most **1,000 per 10 seconds** per function (or 10,000 req/s per 10 s).
- **Simulator behaviour (observed, flows `C-09` to `C-12`):** the first request pays the cold start (about 850 ms for an 800 ms init plus 50 ms of work), later requests do not, and after an idle gap longer than `idleTimeoutMs` the next burst is cold again. The cold-start state belongs to the **node**, not to each concurrent environment: a burst of 16 simultaneous requests during init records **one** cold start and the rest queue behind it, whereas real Lambda would create about 16 environments that each pay init. So the simulator understates cold-start cost for bursty traffic (`C-12`, tracked as an expected failure). The default concurrency cap is the node's derived worker count.
- Cost is per request and per duration, so low or spiky traffic is cheap and sustained high traffic is expensive compared with a fixed fleet. [K]

Testable expectations

- Concurrency = rate x duration: 5,000 req/s at 200 ms needs 1,000 environments; at the 1,000 cap, 5,001 req/s is throttled. [P]
- The first request after idle pays init time; later requests do not. With provisioned concurrency n, the first n concurrent requests do not.
- When offered concurrency exceeds the cap, the surplus is rejected (`max_concurrency_exceeded` in the simulator), not queued.

- The cap gives an **upper bound** of cap / duration admitted per second, not an exact rate. With random arrivals a cap of 2 slots, 500 ms requests and 20 req/s (10 Erlangs) blocks the Erlang B fraction 50/61 = **82%**, so about 3.6 req/s are served against the bound of 4. [computed, chapter 01 section 5.2]

7. Sidecar, gateway-attached and other small nodes

- A **sidecar proxy** sits beside every service instance and handles its inbound and outbound calls. It adds latency and CPU to every call. Istio's published figures for the Envoy sidecar: about **0.20 vCPU and 60 MB per 1,000 req/s** (1 KB payloads, mutual TLS). That is about 0.2 ms of CPU per request per proxy; a service-to-service call crosses two proxies. [P: Istio performance docs; derivation computed]
- Latency added is reported only in charts on that page, so take it from your own measurement. [P, numbers not extractable]
- Because the cost applies to every hop, a chain of k services each with a sidecar adds k x (two proxy traversals) to the path. [K]

8. Batch workers and background functions

- Pull work from a queue; throughput = workers / service time; latency matters little. Scale by queue depth. [K]
- Backlog arithmetic from chapter 05 applies: if arrivals exceed capacity, queue grows at (arrivals – capacity) per second.

9. GPU nodes

- Few, expensive, high throughput when requests are **batched**: one GPU processes a batch for nearly the cost of one request, so waiting to form a batch trades latency for throughput. [K]
- Capacity planning: tokens per second (LLM) or images per second, not requests. See chapter 18. **[?]** to confirm the simulator models batching.

10. Simulator parameter map

Setting	Meaning	Real counterpart
`resources.instanceType`	catalog SKU: vCPU, RAM, price, `perfFactor`	EC2 instance type
`resources.instanceCount`	number of instances	fleet size
`resources.workloadKind`	`cpu-bound` (1 worker per vCPU) or `io-bound` (32 per vCPU)	thread/worker model
`resources.perRequestMemMb`	memory per in-flight request; K = RAM / this	per-request footprint
`resources.pricingModel`	on-demand, reserved, spot	purchase model (price only)
`queue.workers`, `queue.capacity`	raw concurrency and admission, used only when no instance model is authored	hand-set pool and buffer
`processing.distribution`, `.timeout`	service time and per-request timeout	latency and client timeout

Default profiles per type (the instance and kind a fresh palette node gets): microservice, container, auth, search on `c5.large`; serverless, API gateway, load balancers, CDN, NAT, VPN, reverse proxy, service mesh on `m5.large`; sidecar `t3.medium`; batch worker, auth service, search service, edge compute `c5.large` cpu-bound; GPU node `c5.2xlarge` cpu-bound. Default mean service times by type are in the component spec table (for example relational DB 8 ms, cache 0.1 ms, API gateway 1 ms).

11. Testable expectations (summary)

- Derived workers = vCPU x (1 for cpu-bound, 32 for io-bound) x instanceCount; admission K = RAM / perRequestMem x instanceCount, never below workers.
- Capacity = workers / (service time / effective perf factor); io-bound effective perf factor = 1 + (perf − 1) x 0.25.
- Little: in-flight at steady state = admitted rate x time in system.
- Serverless: concurrency = rate x duration; cold start only on a new environment.
- Cost per hour = instanceCount x price per hour (x purchase multiplier).

12. Open items

- Fetch and cite: Gunther's USL paper (the formula above is from knowledge); Goetz's book; AWS EC2 purchasing options for the real discount ranges; a source for T3 credit accounting.
- Decide `[?]`: whether the simulator should model CPU contention when workers exceed physical cores, and GPU batching.

Sources

- AWS EC2 general purpose instance types [P]
- AWS Lambda concurrency and scaling [P]
- AWS Lambda SnapStart [P]
- Amdahl's law, Wikipedia [S]
- Zalando: how to set an ideal thread pool size (Goetz formula) [S]
- Istio performance and scalability [P]

07. Network nodes and routing

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 17 `NetworkType`s: `load-balancer`, `load-balancer-l4`, `load-balancer-l7`, `global-traffic-manager`, `edge-router`, `nat-gateway`, `transit-gateway`, `vpn-gateway`, `cdn`, `api-gateway`, `service-mesh`, `ingress-controller`, `reverse-proxy`, `high-perf-nic`, `network-policy`, `routing-rule`, `routing-policy`. This chapter covers how each moves requests and the math behind choosing where a request goes. The cost of the links between nodes is chapter 08.

1. What these nodes do

They sit **between** callers and servers and make one of five decisions. [K]

Decision	What it does	Nodes
Spread requests over equivalent servers	pick a target by an algorithm; remove unhealthy ones	`load-balancer`, `-l4`, `-l7`, `ingress-controller`, `reverse-proxy`
Route by content or policy	choose among different services by path, host, header, weight	`api-gateway`, `routing-rule`, `routing-policy`, `ingress-controller`
Cache or offload	answer without reaching the origin	`cdn`, `reverse-proxy`
Connect networks	translate or tunnel between address spaces	`nat-gateway`, `vpn-gateway`, `transit-gateway`, `edge-router`
Add cross-cutting behaviour	encrypt, authenticate, retry, observe on every call	`service-mesh`, `api-gateway`, `network-policy`
Steer globally	send users to a region	`global-traffic-manager` (also chapter 21)

Each is also a **queue with a capacity** (chapter 01): a load balancer that is too small is the bottleneck for everything behind it. The simulator gives each a default instance and a small default service time (L4 balancer 0.15 ms, L7 0.4 ms, reverse proxy 0.5 ms, API gateway 1 ms, NAT 0.5 ms, VPN gateway 2 ms, CDN 2 ms). [repo config]

2. Layer 4 versus layer 7

- **Layer 4 (transport)** balancers route on IP and port (TCP/UDP). They do not look inside the request, so they are fast and cheap, preserve connections end to end, and cannot route by URL. [K]
- **Layer 7 (application)** balancers terminate the connection, parse HTTP or gRPC, and route on path, host or header; they can terminate TLS, retry, rewrite and authenticate. That costs more CPU per request. [K]
- AWS describes the same split: an Application Load Balancer works on requests (layer 7), a Network Load Balancer on connections (layer 4), a Gateway Load Balancer for inline appliances. [P: AWS Elastic Load Balancing docs, structure only fetched]
- Consequence for the model: an L7 node has a **higher service time** than an L4 node at the same load (simulator defaults 0.4 ms vs 0.15 ms), and **only L7 can split by request type**. [K]

Testable expectations

- Same traffic through L4 and L7 with identical capacity: L7 shows the higher per-request latency and utilisation.
- A path-based split works through an L7 node; an L4 node cannot route by request type.

3. Load-balancing algorithms

Choosing a target among n equivalent servers. Each algorithm, the information it needs and where it fails. [P: NGINX docs for the named methods; the rest K unless marked]

Algorithm	Rule	Strength	Weakness
Round robin	take servers in turn	simple, even request counts	ignores that requests differ in cost and servers differ in speed
Weighted round robin	in turn, proportional to weights	handles unequal machines: weights 3, 1, 1 send 60%, 20%, 20% [P]	static; ignores current load
Random	pick uniformly	no shared state	uneven in the short term
Least connections	send to the server with the fewest active connections [P]	adapts to slow requests	needs shared state; "connections" is a proxy for load
Least response time	lowest average latency and fewest active (NGINX `least_time`) [P]	adapts to slow servers	noisy; can pile onto a fast server
Power of two choices (P2C)	pick 2 at random, send to the less loaded	near-optimal balance with almost no state	needs a load signal for 2 servers
IP hash	hash of client address picks the server [P]	same client reaches the same server (affinity)	uneven if few clients; moves many clients when the pool changes
Hash on key	hash of a chosen key (URL, user id)	cache locality, session affinity	hot keys overload one server
Consistent hashing	key and servers on a ring (below)	few keys move when the pool changes	needs virtual nodes to be even
Sticky sessions	remember the first choice per client	stateful apps	defeats balancing; a dead server loses its clients
Broadcast	send to all	replication, fan-out	multiplies load by n

3.1 Why least-loaded beats random: power of two choices [S: Mitzenmacher's thesis, abstract and book chapter]

Throw n balls into n bins. With **one random choice** the fullest bin holds Theta(ln n / ln ln n) balls with high probability. With **two choices** (place in the less full of two random bins) the maximum drops to **ln ln n / ln 2 + O(1)**: an exponential improvement. A third choice only shaves a constant factor. [S]

- n = 1,000: ln n / ln ln n = 3.57 vs ln ln n / ln 2 = 2.79; n = 10^6: 5.26 vs 3.79; n = 10^9: 6.84 vs 4.37. (Asymptotic leading terms, constants omitted.) [computed]
- Practical lesson: **two samples capture almost all the benefit of full knowledge**, which is why NGINX offers `random two` and many proxies use P2C. [P: NGINX]

3.2 Consistent hashing [S: Wikipedia; origin Karger et al. 1997]

Servers and keys are hashed onto a ring; a key belongs to the first server clockwise. [S]

- Adding a server moves only about **1 / (N + 1)** of the keys (those between it and its neighbour); removing one moves about 1 / N. [S]
- **Observed (flows `N-06`, 64 virtual points per server):** growing 9 to 10 servers moved 9.5% of 50,000 keys (theory 10%) against 89.8% for `hash mod n`; growing 4 to 5 moved 17.6% (20%) against 79.7%. Removing a server moved only the keys that server owned.
- **Compare `hash mod n`**: changing n from 9 to 10 leaves a key where it was only about 1 time in 10, so **about 90% of keys move**, against about **10%** on a ring. [computed]
- **Virtual nodes:** with few servers, ring positions cluster and arcs differ widely; hashing each server to many points evens the load and spreads a failed server's keys over many successors instead of doubling one neighbour. [S]
- Used for cache fleets and sharded stores (chapter 12): a moved key is a cache miss, so fewer moves means a smaller miss storm.

3.3 Expected results to test

Round robin over n identical servers with constant arrivals: each gets exactly 1/n. Weighted 3:1:1: shares 0.6, 0.2, 0.2. Random: shares near 1/n with sampling noise of sqrt(p (1 – p) / N).

- Least-connections with one slow server (2x service time): the slow server receives about half the requests of a fast one, so queue lengths even out; round robin leaves the slow server saturated first. [K]
- Hash on key with Zipf keys (chapter 05): the hottest key's server carries at least that key's share. With s = 1.0 over 1,000 keys, one key is 13.4% of traffic, so a server holding it carries more than 13.4% however many servers there are. [computed]
- Sticky: all requests from one client reach one server until it fails. **Simulator keys (observed):** `sticky` pins by `sessionId` (or the canonical key, or `stickyKeyField`); `ip-hash` pins by `clientIp` only, so a workload that sets sessions but no client address is not pinned by `ip-hash`.

4. Health checks and removing servers

See chapter 02 section 7. Detection delay (interval x consecutive failures) is the time a dead server still gets traffic.

- After one of n equal servers is removed, each survivor carries n / (n – 1) of its previous load: 4 servers at 75% become 100%. [K]
- Balancers use two signals: **active** probes and **passive** outlier detection (consecutive failures; NGINX `max_fails` default 1 within `fail_timeout`, then the server is skipped and re-probed with live traffic). [P: NGINX]
- When every target is unhealthy a balancer either serves anyway or fails; the choice should be explicit. **[?]** The simulator **fails**: it rejects with `no_healthy_targets`.
- **Ejection can amplify an overload (observed).** A server is ejected after a single failed attempt, for a window. At a mild overload of 10% (220 req/s into 200 req/s of capacity) a few rejections at each server eject both, and the balancer then rejects about 85% of traffic: goodput 33 req/s against 200 req/s of capacity. With health ejection off the same fleet serves 198.8 req/s. Real balancers soften this: Envoy's **panic threshold** serves to all hosts when too many are ejected; NGINX's `max_fails` and `fail_timeout` (default one failure for 10 s) behave like the simulator. Teaching consequence: a balancer with aggressive passive ejection can turn a slight overload into an outage, which is the retry-storm and metastable-failure story of chapter 22. Tracked as `N-16`.

5. API gateway, ingress controller and reverse proxy

- **Reverse proxy:** accepts requests for backends, can terminate TLS, cache, compress, rate-limit. **Ingress controller:** the same role for a Kubernetes cluster, routing by host and path to services. **API gateway:** adds auth, quotas, request transformation and throttling in front of APIs. [K]
- **Throttling is a token bucket.** Amazon API Gateway limits steady-state rate and a burst per account and region, returns 429 when exceeded; historically 10,000 req/s with a burst of 5,000 (dated, check current quotas). [S: AWS announcement and docs] Token bucket maths: chapter 02 section 4.
- Everything in the request path adds its own latency and its own failure probability; the path's availability is the product of the hops' availabilities. Three hops at 99.9% give 99.7%. [K]

6. CDN and caching proxies

- Users are routed to a nearby **edge**; a hit returns immediately, a miss goes to the **origin** and the response is cached for later requests. [S]
- **Origin shield / tiered cache:** a regional layer between edges and origin; an edge miss asks the shield first, so the origin sees far fewer misses and the shield's cache is warmer because it serves many edges. [S]
- Lifetime is controlled by `Cache-Control: s-maxage` for shared caches (CDN), `max-age` for browsers, `no-store` and `private` disable caching, and unnormalised query strings or varying headers fragment the cache. [S]
- Hit ratio benchmarks vary by source (static content 95% to over 99%, personalised API responses 30% to 70%); treat as ranges, not targets. [S, disputed between sources]

Maths [K]

- Origin request rate = (1 − h) x total rate, where h is the hit ratio. At h = 0.95 the origin sees **5%** of traffic; at 0.99, **1%**. A CDN drop from 99% to 95% quintuples origin load.
- Mean latency = h x t_edge + (1 − h) x (t_edge + t_origin). With t_edge 10 ms, t_origin 120 ms, h = 0.95: **16 ms**; at h = 0.50: 70 ms.
- With a shield of hit ratio h2 on edge misses, origin load = (1 − h) x (1 − h2) x rate.

Testable expectations

- Origin arrivals = (1 − h) x edge arrivals, within sampling error.
- Cold start: hit ratio is low until the cache warms; a flush returns it to zero then recovers (chapter 10).

7. Service mesh and sidecar

- A **service mesh** puts a proxy beside each service to give uniform mutual TLS, retries, circuit breaking and telemetry without changing application code. [K]
- The tax: about 0.20 vCPU and 60 MB per 1,000 req/s per sidecar in Istio's published test; every call crosses two sidecars. [P: Istio docs]
- The simulator's `service-mesh` node is a small queue on the path (default 0.6 ms); behaviours such as retries and breakers are configured as resilience settings on the calling node (chapter 02).

8. NAT, VPN, transit gateway, edge router, NIC, network policy

Node	Real role	What matters for capacity
NAT gateway	lets private hosts reach outside; translates addresses	throughput and connection-table limits, a per-GB data charge
VPN gateway	encrypted tunnel to another network	encryption CPU cost, bandwidth, a single tunnel is a bottleneck
Transit gateway	hub connecting many networks	per-attachment bandwidth, per-GB cost
Edge router	border between networks	routing and bandwidth
High-performance NIC	faster network card (offload, RDMA)	removes the host as a network bottleneck
Network policy	allow or deny rules between workloads	adds a small per-packet check; blocks traffic that should not flow

Numeric limits differ by provider and change; fetch current quotas before using them as oracles. **[?]** All rows [K].

9. Global traffic steering

Directing users to the nearest or healthiest region (geo, latency, weighted and failover policies) is mostly DNS-based; see chapter 21. At the network layer **anycast** advertises one IP from many places so routing delivers users to the nearest. [K]

10. Simulator parameter map

Setting	Meaning
`routingStrategy`	`round-robin`, `weighted`, `random`, `least-conn`, `least-response-time`, `p2c`, `sticky`, `ip-hash`, `broadcast`, `conditional`, `passthrough`
edge `weight`	relative weight for the `weighted` strategy
edge `condition`	expression for `conditional` routing (L7 only in practice)
`healthCheckEnabled`	remove unhealthy targets from routing (default on)
`config.routingKeyField`, `stickyKeyField`	request field used for hash and sticky routing
content routing rules	route by request type, path or header

11. Testable expectations (summary)

- Round robin shares 1/n exactly for even arrivals; weighted shares equal normalised weights.
- P2C maximum queue length is far below random assignment at the same load, and close to least-connections.
- Hash routing: one key always lands on one server; adding a server moves about 1 / (N + 1) of keys with consistent hashing, about N / (N + 1) with modulo.
- Load balancer failure removes its whole subtree from service: a single LB is a single point of failure.
- Per-request latency through a gateway = queue wait + gateway service time; for a lightly loaded L7 node about its 0.4 ms service time.

12. Open items

- Fetch and cite: Mitzenmacher's survey; Karger et al. 1997; Envoy and HAProxy load-balancing docs; Maglev (Google) for hash-based balancing; provider quotas for NAT, VPN and transit.
- Decisions `[?]`: behaviour when all targets are unhealthy; whether least-connections counts connections or in-flight requests.

Sources

- NGINX HTTP load balancing [P]
- Consistent hashing, Wikipedia [S]
- Mitzenmacher, The Power of Two Choices in Randomized Load Balancing (thesis) [S, abstract]
- AWS Elastic Load Balancing [P]
- Istio performance and scalability [P]
- Amazon API Gateway throttling [P]

08. Edges, protocols and network physics

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

An **edge** is the connection between two nodes. It carries a request over some distance, at some bandwidth, through some protocol, and it can lose or fail it. This chapter covers each of those, with the arithmetic. Edge fields in the simulator: `mode`, `protocol`, `latency` (distribution and `pathType`), `bandwidth`, `maxConcurrentRequests`, `packetLossRate`, `errorRate`, `weight`, `condition`, `fanoutFactor`, `connection`, `batching`.

1. The cost of one hop

A request crossing an edge pays, in order: **connection setup** (if no warm connection), **transmission** (putting the bytes on the wire), **propagation** (the signal travelling), possible **queueing** at a busy link, and **retransmission** if packets are lost. [K]

time per hop = setup + queue wait + size / bandwidth + propagation (+ loss recovery)

1.1 Reference latencies [S: "latency numbers" list, circa 2012]

Operation	Time
Main memory reference	100 ns
Send 1 KB over a 1 Gbps network	10 us
Read 4 KB randomly from SSD	150 us
Round trip within the same datacenter	500 us
Read 1 MB sequentially from SSD	1 ms
Disk seek	10 ms
Round trip California to Netherlands and back	150 ms

These are order-of-magnitude anchors; hardware has improved, so use them for ratios (a cross-ocean hop is about 300x a same-datacenter hop), not as exact values.

2. Propagation delay

The signal speed in optical fibre is about c / 1.47, roughly **204,000 km/s**, so **about 4.9 us per km one way** and 9.8 us per km round trip. [S: fibre-latency measurements] (A common round "5 us/km" is one-way.)

- 1,000 km: 4.9 ms one way, **9.8 ms round trip**. [computed]
- New York to London (about 5,570 km great circle): round-trip floor **about 55 ms**; real paths are longer than the great circle and add switching, so measured values are higher. [computed floor]
- Within a region the floor is tens of microseconds; measured **between availability zones in one region is about 0.6 to 1.1 ms** on average (one AWS study across four regions: 0.61 to 1.13 ms), set mostly by switching, not distance. [S: secondary]

- Between two US regions latency is likely above 10 ms; a single-digit-millisecond design target belongs to availability zones, not regions. [S: AWS re:Post, not a measurement]

Testable expectations

- Latency grows with distance by about 4.9 us per km one way; doubling the distance doubles propagation.
- Same-rack < same-datacenter < cross-zone < cross-region < internet, each larger than the last.

2.1 What the simulator uses

Latency per path type is a log-normal (chapter 03) sampled once per hop:

`pathType`	**median**	**mean**	**p99**	**default bandwidth**
`same-rack`	0.30 ms	0.32 ms	0.61 ms	10,000 Mbps
`same-dc`	1.00 ms	1.08 ms	2.54 ms	5,000 Mbps
`cross-zone`	2.01 ms	2.18 ms	5.11 ms	2,500 Mbps
`cross-region`	60.3 ms	63.1 ms	121 ms	1,000 Mbps
`internet`	99.5 ms	137 ms	640 ms	100 Mbps

Comparison with the numbers above [computed]: `same-dc` (median 1.0 ms) is about twice the 0.5 ms datacenter round trip; `cross-zone` (median 2.0 ms) is about twice the measured 0.6 to 1.1 ms inter-AZ averages; `cross-region` (60 ms) is plausible for a trans-continental US pair or a short trans-Atlantic one but too low for far pairs. **[?]** Owner to confirm whether these are meant as one-way or round-trip figures and whether to recalibrate cross-zone. The engine's own comment says the path-type defaults are round-trip figures, yet a hop pays one sample and responses do not traverse edges, so a hop's sample stands for the whole exchange.

3. Transmission delay and bandwidth

transmission time = size / bandwidth. [K]

- 1 KB at 1 Gbps = 8 us (the reference list says 10 us). 1 MB at 100 Mbps = **80 ms**. The simulator uses size_bytes / (Mbps x 125) ms, which gives 0.008 ms for 1,000 bytes at 1,000 Mbps. [computed]
- A link is a **FIFO server**: transfers that arrive while an earlier one is still serialising wait. Throughput can never exceed bandwidth; offered load above it builds a queue exactly like a node (chapter 01). Utilisation of a link = offered bits per second / bandwidth.
- **Bandwidth-delay product** BDP = bandwidth x round-trip time: the data in flight to keep the pipe full. [S: Wikipedia] 1 Gbps x 70 ms = **8.75 MB**; 100 Mbps x 100 ms = 1.25 MB; a 2 Mbps DSL line at 50 ms = 12.5 kB. A TCP window smaller than the BDP leaves the link idle; without window scaling TCP caps at 65,535 bytes. [S]
- Rule: for small requests (a few KB) latency is dominated by propagation; for large transfers (MB) by size / bandwidth.

Testable expectations

- A request of S bytes on a link of B Mbps adds S / (B x 125) ms of transmission delay.
- Offered traffic above the bandwidth makes the link's queue wait grow; delivered rate equals the bandwidth.

- A 1 MB payload on a 100 Mbps edge adds about 80 ms; on a 10,000 Mbps edge about 0.8 ms.

4. TCP, TLS and what a new connection costs

- **TCP** needs a three-way handshake: **1 round trip** before any data moves. [S: RFC 6928 context; K]
- **TLS 1.2** adds **2 round trips**; **TLS 1.3** adds **1**; **TLS 1.3 resumption with 0-RTT** lets the client send data in its first flight (replayable, so only safe for requests that can be repeated). [S: secondary summaries of RFC 8446]
- So the time to the first response byte on a fresh HTTPS connection, with round-trip time R and a server that answers instantly [computed]:

Case	Round trips	At R = 60 ms	At R = 1 ms
Reused connection (keep-alive)	1	60 ms	1 ms
New, TLS 1.3 with 0-RTT resumption	2	120 ms	2 ms
New, TLS 1.3	3	180 ms	3 ms
New, TLS 1.2	4	240 ms	4 ms

- **Slow start:** a new connection starts with an initial congestion window of up to **10 segments (about 14.6 KB)** and roughly doubles it each round trip. [S: RFC 6928] A 100 KB response (about 70 segments) needs about 3 data round trips after the handshake: 10, 20, 40 segments, so about **4 round trips** from a cold start. [computed]
- **Simulator behaviour (observed, flows `E-06`):** with a shared client identity the numbers above hold exactly: per-request TLS 1.2 costs 4 round trips, TLS 1.3 costs 3, resumed TLS 1.3 costs 2 and a resumed TLS 1.2 costs 3; keep-alive and persistent connections pay the handshake only on the first request (4 round trips for TLS 1.2, 3 for TLS 1.3), then 1. **Connections belong to a client**: a request with no client identity is treated as a new client, so without sessions every request pays the full setup even on a keep-alive edge, and resumption never applies. Give the workload a session count to model reuse.
- Lesson: **connection reuse is the biggest latency lever on long paths.** At cross-region distances a new TLS 1.2 connection costs 4x one warm request.

Testable expectations

- With `reuse: per-request` every request pays setup; with `keep-alive` only the first does; `persistent` never closes.
- Setup cost is a multiple of the edge round trip: TCP 1x, plus TLS 1.2 2x or TLS 1.3 1x.
- TLS session resumption reduces the repeat handshake (1.2: one round trip; 1.3: zero).

5. Packet loss

- A lost packet costs a retransmission. Loss also **throttles TCP throughput**: the **Mathis model** (Mathis, Semke, Mahdavi and Ott, 1997) bounds a sustained connection at **rate <= (MSS / RTT) x C / sqrt(p)**, with C about 1.22 (sqrt(3/2)) when every packet is acknowledged, 0.87 with delayed acks. It assumes light to moderate loss and no timeouts; Padhye et al. is more accurate above p of about 2%. [S]
- Worked, MSS = 1,460 bytes, RTT = 50 ms [computed]: p = 0.01% gives **28.6 Mbit/s**; p = 0.1% gives **9.05 Mbit/s**; p = 1% gives **2.86 Mbit/s**, whatever the link capacity. Loss, not bandwidth, can be the limit on long paths.

- A request of k packets is hit if any is lost: P = 1 – (1 – p)^k. With p = 1% and k = 10, **9.6%**. [computed] Large responses suffer more often than small ones.
- Recovery waits for a timeout when there are too few later packets to trigger fast retransmit (typical minimum retransmission timeout around 200 ms on Linux), so a lost packet can cost far more than one round trip. [K]
- UDP does no retransmission: a lost datagram is gone unless the application handles it (QUIC, DNS, games, real-time media). [K]
- **Simulator behaviour (observed, flows `E-08`):** a lost packet adds **one extra edge latency** (a fast-retransmit round trip) and never fails the request: at 20 ms latency and 10% loss about 10% of requests take 41 ms instead of 21 ms, and none fail. The probability a one-packet request is hit is p. The Mathis throughput bound is **not modelled**: loss never lowers delivered bandwidth, and a retransmission timeout (hundreds of ms) is not modelled either, so heavy-loss paths look better in the simulator than in reality (`E-08b`, parked).

6. Protocols

Protocol	Layer and style	Request pairing	Key behaviour
`https` (HTTP/1.1)	text over TCP and TLS	one request in flight per connection	head-of-line blocking per connection; needs many connections for concurrency
`grpc` (HTTP/2)	binary over TCP and TLS	many concurrent streams per connection	multiplexing; RFC 9113 recommends a concurrent-streams limit **no smaller than 100** [S]
`tcp`	raw byte stream	database and cache wire protocols, typically one query at a time per connection	pool size sets concurrency
`udp`	datagrams	none	no handshake, no retransmission, no ordering
`websocket`	long-lived full-duplex over an HTTP upgrade	frames, not request-response	one extra round trip to upgrade, then no per-message setup; connection state held at the server
`amqp`	broker messaging over TCP	channels over one connection	application handshake after TCP and TLS
`kafka`	broker protocol over TCP	produce and fetch requests	batching, acks, partitions (section 8)

6.1 Connection pools and Little's Law

Connections needed = concurrent requests / requests per connection, and concurrent requests = rate x latency (chapter 01).

- HTTP/1.1, 2,000 req/s at 50 ms: 2,000 x 0.05 = **100 concurrent**, so about 100 connections. With HTTP/2 and 100 streams per connection, **1 connection** carries it. [computed]
- A pool smaller than the concurrent demand makes requests wait for a free connection: a queue in front of the link, separate from the server's own queue. That wait must be reported distinctly (the simulator calls it edge connection wait).
- Idle connections time out (keep-alive idle timeout, simulator default 60 s); after that the next request pays setup again. [repo default]

Testable expectations

- Pool of m connections, per-request time W: maximum throughput = m / W for HTTP/1.1 style, m x streams / W for multiplexed.
- With a pool of 10 and 50 ms requests, throughput caps at 10 / 0.05 = **200 req/s** whatever the server can do.

7. Synchronous, asynchronous, streaming and conditional edges

`mode`	**Meaning**	**Effect on the caller**
`synchronous`	wait for the response	latency adds to the caller's time; failure propagates
`asynchronous`	fire and continue (queue, event)	caller returns after handing off; downstream latency is hidden; load still arrives downstream
`streaming`	long-lived, many messages	no per-message setup
`conditional`	taken only if a condition holds	splits traffic by rule

- Asynchronous does **not** remove load: the downstream still receives every message. It removes the caller's wait and decouples availability. [K]
- **Simulator behaviour (observed, flows `E-11`):** an asynchronous edge **forks a side branch** next to the synchronous continuation. The branch still loads its target (every request arrives there) and is reported as its **own request**, so a service with a 3 ms synchronous path and a 200 ms asynchronous side branch shows a blended p99 of about 200 ms while the user-facing path takes about 5 ms. Read per-branch latency, not the blended percentile, for user experience. If a node has **no** synchronous continuation, the asynchronous branch simply is the request and the caller waits. Queue and stream node types are asynchronous **by type** and detach the caller automatically. Two synchronous targets are a split (each request goes to one), not a fork.

8. Batching (Kafka-style)

Producers group records into **batches** to cut per-request overhead. `linger.ms` is how long to wait for more records; `batch.size` is the byte size that triggers an immediate send. A batch is sent when **either** limit is reached. [S]

- Kafka 4.0 changes the `linger.ms` default from 0 to **5 ms** (KIP-1030). [S] (The `batch.size` default, 16 KB in earlier versions, was not confirmed here.)
- **The first record in a batch waits min(linger, time to fill); the average record waits less.** A batch that fills holds records that arrived at different times: with n records at gap g the waits are 0, g, …, (n − 1) g, an average of (n − 1) g / 2. Five records at 1,000 req/s (g = 1 ms) fill in 5 ms but wait 2 ms on average, which is what the simulator shows (`E-12`); a record alone in its batch waits the whole linger (about 10 ms at 10 req/s with `linger.ms = 10`).
- Added latency for the first record = min(linger, time to fill the batch). At 10,000 records/s of 1 KB on one partition, a 16 KB batch fills in 1.6 ms, so `linger.ms = 5` never waits the full 5 ms; at 100 records/s the batch never fills and every record waits the full linger. [computed]
- Trade: larger batches give higher throughput and compression ratios at the cost of added latency per record.

9. Fan-out

A fan-out factor f multiplies the load a request creates downstream: one post written to f followers is f writes. Downstream arrival rate = f x source rate. A factor of 500 on 100 posts/s is 50,000 writes/s, which can saturate a node that the source rate alone never would. [K, chapter 24]

10. Errors on the wire

- `errorRate` is the chance a request over this edge fails (for example a reset or refused connection); `packetLossRate` is the chance a packet is lost and must be retransmitted. They are different failures: errors fail requests, loss delays them. [K]
- A failure rate e per hop gives a path success rate of the product of (1 – e) over hops. Five hops at 1% each: 95.1%. [computed: 0.99^5]

11. Simulator parameter map

Edge field	Unit	Meaning
`latency.distribution`	ms	per-hop delay sample (default is the log-normal for the path type)
`latency.pathType`	n/a	`same-rack`, `same-dc`, `cross-zone`, `cross-region`, `internet`
`bandwidth`	Mbps	link capacity; transmission = bytes / (Mbps x 125) ms
`maxConcurrentRequests`	count	limit on in-flight requests on this edge
`protocolOverheadMs`	ms	per-request protocol cost: https 0.5, grpc 0.2, tcp 0, udp 0, websocket 0.1, amqp 1, kafka 2
`packetLossRate`, `errorRate`	0 to 1	loss and error probabilities
`fanoutFactor`	multiplier	deliveries per request
`connection.reuse`	`per-request`, `keep-alive`, `persistent`	connection lifecycle
`connection.tls`	`none`, `1.2`, `1.3`	handshake version
`connection.maxConnections`, `maxStreamsPerConnection`	count	pool and multiplexing limits
`batching.lingerMs`, `maxBatchBytes`	ms, bytes	Kafka batching (kafka edges only)

Connector mode. In the default environment connections are plain wires and every field above is ignored at run time (the importer says so; chapter-level flows `M-01` to `M-05`). Test edge physics in the network environment.

12. Testable expectations (summary)

- Two nodes, edge latency L ms, service time S ms, no queueing: end-to-end latency = L + S + protocol overhead (network mode).
- Error rate e on the only edge: failed fraction e.
- Offered traffic above `bandwidth`: delivered rate equals bandwidth; link queue wait grows.
- New connection costs 1 (TCP) + 2 (TLS 1.2) or 1 (TLS 1.3) round trips; a reused connection costs none.
- Pool of m connections at W seconds each caps throughput at m / W.

- Batching: record latency added = min(linger, batch fill time).
- Fan-out f multiplies downstream arrivals by f.

13. Open items

- Fetch and cite: RFC 8446 and RFC 9113 text; Mathis et al. 1997; Kafka producer configuration page for defaults; AWS Network Manager inter-region latency figures; QUIC (RFC 9000) for the combined handshake.
- Decisions `[?]`: one-way or round-trip meaning of path-type latency; whether to recalibrate `cross-zone`; how response transfer should be charged since responses do not traverse edges.

Sources

- Latency numbers every programmer should know (list, circa 2012) [S]
- Bandwidth-delay product, Wikipedia [S]
- RFC 6928: Increasing TCP's Initial Window [P]
- Mathis, Semke, Mahdavi, Ott: The macroscopic behavior of the TCP congestion avoidance algorithm, CCR 1997 [P]
- Dissecting latency in the Internet's fiber infrastructure [S]
- KIP-1030: change defaults, including linger.ms [P]
- RFC 9113: HTTP/2 (SETTINGS_MAX_CONCURRENT_STREAMS) [P, guidance read via mailing-list copy]

09. Storage and databases

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 19 `StorageType`s: `relational-db`, `nosql-db`, `object-storage`, `block-storage`, `distributed-file-system`, `in-memory-cache`, `search-index`, `time-series-db`, `columnar-db`, `graph-db`, `vector-db`, `data-warehouse`, `data-lake`, `kv-store`, `archive-storage`, `schema-registry`, `cdc`, `backup-service`, `kms-storage`. Caching is chapter 10; replication and consistency chapter 11; sharding chapter 12; the change stream and backup flows touch chapter 13.

1. What a data store does and what limits it

A store holds data durably and answers reads and writes. Its limits are always one of: **I/O operations per second** (IOPS), **bytes per second** (throughput), **CPU** (query execution, compression), **memory** (working set), **connections**, or **capacity** (bytes). A request is slow when it waits behind one of those. [K]

Anchor numbers [S: "latency numbers" list circa 2012; order of magnitude only]:

Operation	Time
Main memory reference	100 ns
Read 4 KB randomly from SSD	150 us
Round trip within a datacenter	500 us
Read 1 MB sequentially from SSD	1 ms
Disk seek	10 ms
Read 1 MB sequentially from spinning disk	20 ms

Rules of thumb that follow: memory is about 1,500x faster than an SSD random read; sequential access is far cheaper than random on every device (1 MB sequential from SSD in 1 ms against about 256 random 4 KB reads at 150 us each = 38 ms); and a network hop to the store costs more than the SSD read behind it.

2. I/O capacity: IOPS, throughput and queue depth

- **IOPS x I/O size = throughput.** Amazon EBS states it directly: throughput in MiB/s = IOPS x I/O size in KiB / 1,024. [P: AWS EBS docs]
- **gp3** (general purpose SSD): baseline **3,000 IOPS and 125 MiB/s**, independent of volume size; provision up to 80,000 IOPS and 2,000 MiB/s; throughput scales 0.25 MiB/s per provisioned IOPS (2,000 MiB/s needs 8,000 IOPS); single-digit-millisecond latency; no burst credits. [P]
- **gp2:** 3 IOPS per GiB (minimum 100, maximum 16,000), bursting to 3,000 IOPS from a credit bucket of 5.4 million credits. Burst duration = credits / (burst IOPS – baseline IOPS). A 100 GiB volume (300 IOPS baseline) bursts 5.4 M / 2,700 = **2,000 s** and needs 18,000 s to refill; a 750 GiB volume (2,250 baseline) bursts 7,200 s. [P, computed]
- Worked: 3,000 IOPS at 16 KiB = **46.9 MiB/s**, so the IOPS limit binds first; to reach 125 MiB/s you need 8,000 IOPS at 16 KiB, 500 IOPS at 256 KiB, or 32,000 IOPS at 4 KiB. Small random I/O is IOPS-limited, large sequential I/O is throughput-limited. [computed]

- **Queue depth (Little's Law, chapter 01).** A device that serves one 150 us random read at a time does 1 / 150 us = **6,667 IOPS** at queue depth 1; SSDs serve many in parallel, so throughput rises with queue depth until the device saturates. IOPS = outstanding requests / latency. [K, computed]
- Saturation shows as latency rising with queue depth, exactly like any queue: expect the hockey stick (chapter 01 section 4.1).

Testable expectations

- A store with capacity C ops/s and service time S has C = workers / S; offered load above C builds a queue and then rejects.
- Throughput never exceeds IOPS x I/O size; the binding limit is whichever is lower.

3. Relational databases

- **Row storage on B-trees.** An index is a B-tree whose nodes are disk blocks; a lookup reads about **log_b(N)** blocks for fan-out b. N = 10^6 rows at b = 100 is 3 reads (about 20 for a binary search); N = 10^9 at b = 100 is 5 levels (4.5 rounded up); at b = 500, 4. Search, insert and delete are O(log N). [S: Wikipedia B-tree; levels computed] Upper levels are cached in memory, so a lookup is often one disk read.
- **Write-ahead logging (WAL).** A change must not reach a data file until the log record describing it is flushed. Commit needs only a sequential **fsync of the log**, not every dirtied data page; one fsync can commit many transactions (group commit). After a crash the log is replayed (roll-forward). Checkpoints bound recovery time. [P: PostgreSQL docs]
- **ACID transactions**, locks or MVCC for isolation, constraints, joins and a query planner. The cost: coordination, which limits write scaling to one primary (chapters 11 and 12).
- **Connection pools.** Each connection holds memory and a process or thread. Connections needed = queries per second x time per query (Little): 500 req/s x 20 ms = **10 connections**; far more than the database has cores usually only adds contention. [computed]
- Typical default in the simulator: 8 ms per point read, 16 ms per write (a log flush plus the data change), 42 ms per scan. [repo: storage profile]

4. NoSQL and the LSM-tree

- **Write path of an LSM-tree** (Cassandra, RocksDB, HBase): append to a write-ahead log, insert into an in-memory sorted buffer (memtable); when full, flush **sequentially** to disk as an immutable sorted file; updates write new entries and deletes write **tombstones**. **Compaction** merges files in the background. [S: Wikipedia LSM-tree]
- **Compaction trade-off:** leveling (one run per level) gives fewer files to read but more rewriting (higher write amplification); tiering (several runs per level) gives less rewriting but reads must check more files. Write amplification under leveling is O(T x L / B) for size ratio T, L levels and B entries per page. [S]
- **Bloom filters** let a lookup skip files that cannot contain the key. False-positive rate p = (1 – e^(–kn/m))^k: 10 bits per key with k = 7 gives **0.82%**; 8 bits with k = 6 gives 2.2%. [K, computed]
- LSM favours **write-heavy** workloads (sequential I/O); mixed workloads can suffer because compaction competes for I/O and invalidates cached data. B-trees favour point reads and updates in place. [S]
- Real effect on capacity: effective device write rate = application writes x write amplification. A 10 MB/s write load at amplification 10 needs 100 MB/s of disk. The simulator models this with a write-

amplification setting on storage capacity. [K, repo config]
- Typical defaults: 4 ms for a NoSQL point read and 7 ms for a write; 1 ms for a key-value read. [repo: storage profile]

5. Object, block and file storage

- **Object storage (S3):** flat namespace of bucket + key; whole-object PUT and GET over HTTP; very high durability and capacity; **strong read-after-write consistency** for PUT and DELETE in all regions (since 2020): a read after a successful PUT returns the new data, a list after a PUT shows it; updates to one key are atomic; there are **no atomic updates across keys**; concurrent writes to a key resolve last-writer-wins. Bucket configuration remains eventually consistent. [P: AWS S3 docs]
- Latency is tens of milliseconds per request (simulator default: a 24 ms read and a 36 ms write); throughput scales by issuing many parallel requests; per-request overhead makes many small objects costly. [K]
- Durability design target of 11 nines for standard classes. [K, verify against the S3 storage-class page]
- **Block storage (EBS-like):** a raw volume attached to one instance; IOPS and throughput limits as in section 2. **Distributed file system (HDFS-style):** large files split into blocks replicated across nodes; high sequential throughput, poor for small files. [K]
- **Archive storage:** cheap, slow retrieval (minutes to hours), minimum storage durations. **Backup service:** copies on a schedule; recovery point objective (RPO) = time since last backup; recovery time objective (RTO) = restore duration = data size / restore rate. [K] Worked: 2 TB restored at 200 MB/s takes 10,000 s, **2.8 hours**. [computed]

6. Analytical stores

- **Row vs column layout:** columnar stores read only the columns a query touches and compress each column well (similar values together). Vendor figures: 5 to 10x compression against 1.5 to 3x for row storage (rough). Columnar suits aggregations over many rows; point lookups and single-row updates are slower because a row must be reassembled. [S: columnar storage guides]
- **Data warehouse:** columnar, SQL, structured, loaded in batches (ETL, chapter 18); queries scan large ranges (simulator default: an 18 ms read and a 28 ms write). **Data lake:** raw files (often Parquet) in object storage, schema applied on read; cheap, flexible, slower (simulator default: 22 ms and 26 ms). [K]
- **Scan vs point read** is a real difference of orders of magnitude: a scan of n rows costs about n x per-row cost, a point read costs about log N index reads. The simulator's storage profile gives reads and scans different service times. [K, repo config]
- **Time-series databases** store timestamped samples append-mostly with time-range queries and downsampling; **search index** (inverted index) maps terms to documents, fast for text queries and slow to update; **graph databases** follow edges cheaply but need whole-graph partitioning thought. [K]

7. Vector databases

- Store embeddings and answer **nearest-neighbour** queries. Exact search is O(N x dimensions); approximate indexes trade recall for speed.
- **HNSW** (Malkov and Yashunin): layered proximity graphs; search starts at a sparse top layer and descends, giving about logarithmic cost in practice. Parameters: **M** (links per node; 16 to 32 typical,

more for high recall and high dimension, costs memory), **efConstruction** (build candidate list; larger is better recall, slower build) and **ef** (query candidate list; larger is higher recall and higher latency). The logarithmic claim comes from secondary sources, not the paper's proofs. [S: implementation guides]
- Capacity planning is memory-bound: vectors x dimensions x 4 bytes plus graph links. 10 M vectors of 768 float32 dimensions is about **30.7 GB** before index overhead. [computed]

8. Schema registry, CDC, KMS storage

- **Schema registry:** stores message schemas and compatibility rules (backward, forward, full) so producers and consumers evolve safely; a small, read-heavy, cacheable service. [K]
- **CDC (change data capture):** reads the database's change log and publishes row changes as events; keeps derived systems (cache, search, analytics) in sync without dual writes. Ordering is per key or per partition (chapter 13). [K]
- **KMS storage:** durable store for keys, with access control and audit; reads are latency-sensitive and cached by clients. Wrapping and unwrapping keys is a CPU-bound call (chapter 15). [K]

9. Capacity, replication and cost arithmetic

- Raw size = rows x bytes per row; index overhead adds roughly 20% to 50%; **replication multiplies stored bytes**: 200 GB at 3 replicas is 600 GB. 10^9 rows x 200 B = 200 GB. [K, computed]
- Working set vs memory: if the hot set fits in memory most reads never touch disk (chapter 10); if not, hit ratio and tail latency fall sharply.
- Per-operation cost differs by store model: provisioned instance-hours, per-request (consumption) pricing, and per-GB storage and egress. The simulator's cost model has instance-based, per-million-request and per-GB lines. [repo]

10. Simulator parameter map

Setting	Meaning	Real counterpart
default storage profile (point read / write)	relational 8 / 16 ms, NoSQL 4 / 7, key-value 1 / 2, object 24 / 36, search index 10 / 20, time-series 6 / 11, graph 9 / 13, vector 11 / 18, warehouse 18 / 28, lake 22 / 26; scans cost 3 to 6 times a read	typical request time; replaces an authored processing time
`resources`	instance type and count; io-bound by default for data nodes	database host
`dataModel`, storage profile	different read and scan service times	point read vs scan
`storageCapacityGb`, `initialStorageGb`, `storageWriteAmplification`	capacity, initial data, amplification	disk size and write amplification
`readLatency`, `writeLatency`	per-operation service time distributions	read/write cost split
`replicationRole`, `replicaMembers`, `replicationLagMs`, `writeAckPolicy`	primary/replica model	chapter 11

Setting	Meaning	Real counterpart
`routingKeyField` and shard settings	which shard owns a key	chapter 12

11. Testable expectations (summary)

- Capacity = workers / service time; a database of 4 workers at 8 ms serves 500 queries/s.
- Reads cost less than scans in a profile that distinguishes them; the ratio equals the configured service-time ratio.
- Storage fills at write rate x write amplification x bytes per record; capacity exhaustion rejects writes.
- Restore time = data size / restore rate.
- Little: connections in use = query rate x query time.

12. Open items

- Fetch and cite: S3 durability and per-prefix request rates (current), the HNSW paper's complexity statements, an LSM write-amplification source with concrete numbers (RocksDB wiki), DynamoDB-style per-partition limits (chapter 12).
- Decisions `[?]`: whether the simulator should model index depth (reads costing more as data grows) and compaction stalls.

Sources

- B-tree, Wikipedia [S]
- Log-structured merge-tree, Wikipedia [S]
- PostgreSQL: write-ahead logging [P]
- Amazon S3: what is S3 and its consistency model [P]
- Amazon EBS general purpose SSD volumes [P]
- Latency numbers every programmer should know (circa 2012) [S]
- Columnar storage guide (MotherDuck) [S, vendor]

10. Caching

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

A cache keeps a copy of data closer or cheaper to reach than its source, so most requests never touch the source. The simulator's cache node models a **hit ratio**, a **hit latency**, a **miss path to a backing store**, and the failure and warm-up behaviour. Storage is chapter 09; CDNs and proxies chapter 07; consistency of cached data chapter 11.

1. What a cache buys

Effective latency = h x t_cache + (1 − h) x (t_cache + t_backing), where h is the hit ratio. [S: Wikipedia states the hit ratio; the average-access formula is the standard model]

- t_cache 1 ms, t_backing 8 ms: h = 0.9 gives **1.8 ms**; h = 0.99 gives 1.08 ms; h = 0 gives 9 ms (the cache is pure overhead). [computed]
- **Backing-store load = (1 − h) x request rate.** At h = 0.95 the store sees 5% of the traffic; 0.99 sees 1%. Going from 0.99 to 0.95 multiplies store load by 5 (chapter 07 section 6).
- **The mean hides the tail.** At h = 0.9 the slowest 10% of requests are all misses, so p90 and above are the **backing store's latency**, not the cache's. A cache improves the median far more than the p99. [K]

Testable expectations

- Backing-store arrivals = (1 − h) x cache arrivals (within sampling error).
- Mean latency follows the formula; p50 equals the hit latency when h > 0.5.
- h = 0 (or a flushed cache) makes every request pay both hops.

2. What decides the hit ratio

Three things: **cache size relative to the working set**, **how skewed popularity is**, and **how fresh data must be** (TTL and invalidation). [K]

2.1 Popularity: Zipf (chapter 05 section 4)

If request probability of the rank-r key is proportional to r^(−s), a cache that **perfectly holds the C hottest keys** (the best possible static policy, LFU-like) hits with probability H(C, s) / H(N, s). For N = 1,000 keys:

s	C = 10	C = 100	C = 300
1.0 (classic web)	39.1%	69.3%	83.9%
0.8	23.1%	52.6%	72.5%

(Perfect-LFU ceilings.) To reach a 90% ceiling at s = 1.0 you need **473 of the 1,000 keys**: skew helps, but a long tail still has to fit. [computed]

2.2 Eviction policies [S: Wikipedia cache replacement policies]

Policy	Evicts	Strength	Weakness
LRU	least recently used	adapts to shifting popularity	not scan-resistant: one pass over cold data flushes useful items
LFU	least often used	keeps persistently popular items	holds on to formerly popular items too long
FIFO	oldest inserted	cheap	ignores use
Random	random item	no bookkeeping	average
Bélády (optimal)	used farthest in the future	the best possible	needs the future; only a benchmark
ARC / 2Q	balances recency and frequency	self-tuning, scan-resistant	more state
TTL-based	expired first	bounded staleness	must still pick among unexpired

Redis offers approximated LRU and LFU (`allkeys-lru`, `allkeys-lfu`) and TTL-based `volatile-ttl`; the eviction policy is set by `maxmemory-policy`. [S: Redis docs, policy list only]

2.3 How well does LRU do? The Che approximation [S: Che et al. 2002; arXiv 1202.3974]

For independent requests with rate q_i for item i, an LRU cache of capacity C behaves as if each item stays a "characteristic time" t_C after its last request, where t_C solves **C = sum over i of (1 − e^(−q_i t_C))**, and the hit probability of item i is **1 − e^(−q_i t_C)**; the overall hit ratio is the request-weighted sum. It is accurate for large catalogues of items that each have a small share of popularity, and exact in the limit under Zipf popularity. [S]

- N = 1,000 keys, s = 1.0: LRU hit ratio **20.9%** at C = 10, **57.7%** at C = 100, **77.1%** at C = 300, against the ideal 39.1%, 69.3% and 83.9%. [computed]
- s = 0.8: 8.2%, 37.8% and 62.3%, against 23.1%, 52.6% and 72.5%. [computed]
- Reading: **a real LRU cache holding 10% of the keys gets about 58% of the requests, not the 69% of a perfect one**: the 11-point gap is the price of not knowing the future. Lower skew (smaller s) hurts more.

Testable expectations

- Derived-LRU cache hit ratio for Zipf traffic matches the Che prediction within a few percentage points and always sits below the ideal top-C share.
- Hit ratio rises with cache size, with diminishing returns; and rises with skew s.
- A uniform workload (s = 0) gives hit ratio about C / N.

3. Cache patterns

Pattern	Read	Write	Notes
Cache-aside (lazy loading)	app checks cache, on miss loads from store and fills the cache	app writes the store, then deletes or updates the cache	most common; app owns the logic; races between a fill and an update leave stale data
Read-through	cache loads from store itself on miss	as above	simpler callers; cache must know the store

Pattern	Read	Write	Notes
Write-through	n/a	writes go to cache and store together [S]	cache always fresh; every write pays both
Write-back (write-behind)	n/a	writes go to the cache first and reach the store later, when the entry is evicted [S]	fast writes; **data loss if the cache fails before flushing**
Write-around	n/a	writes go straight to the store, not the cache [S]	avoids filling the cache with write-once data

Keeping a cache consistent with its store is hard; the safe, simple defaults are short TTLs and invalidate-on-write. [K]

4. Time to live and staleness

- A TTL bounds how stale an entry can be: maximum staleness = TTL. A shorter TTL means more misses: if a key is requested at rate r and cached for TTL seconds, it causes at most **one miss per TTL** to refresh; with r x TTL requests per refresh the **hit ratio of that key is 1 − 1 / (r x TTL + 1)**. A key hit 10 times per TTL has hit ratio 91%. [K, derived]
- Synchronised TTLs (everything cached at once, same expiry) expire together; add **jitter** to TTLs to spread refreshes. [K]
- `Cache-Control` for shared caches: `s-maxage`; browsers `max-age`; `no-store` disables caching. [S, chapter 07]

5. Cache stampede (thundering herd)

When a popular entry expires or the cache is flushed, **many requests miss together and all recompute**, overloading the store, which slows recomputation, which causes more misses: congestion collapse and a hit ratio that can fall to zero. [S: Wikipedia cache stampede]

- Example: a page that takes 3 s to render and is requested 10 times per second causes about 3 x 10 = **30 simultaneous recomputations** at expiry. [S]
- **Mitigations [S]:**
  - **Locking / request collapsing:** the first miss recomputes; others wait for it or are served a stale value. Cost: an extra write per lock, lock-holder failure handling, TTL tuning.
  - **External recomputation:** a separate refresher updates entries before they expire.
  - **Probabilistic early expiration (XFetch):** each request recomputes early with probability rising towards expiry: recompute when now − delta x beta x ln(U) >= expiry, with U uniform on (0, 1), delta the measured recompute time and beta about 1. The early-recompute offset is exponential with mean delta x beta, so refreshes spread out.
  - **Leases (Facebook memcache):** the cache hands one client a short-lived token to fill a missing key and tells others to wait; a delete invalidates the token. Reported to cut database query rate from about 17,000 to 1,300 per second in one case (figure from secondary summaries). [S]
- **Request collapsing arithmetic:** with collapsing, a miss storm of k concurrent requests for one key produces **1** backing-store request instead of k.

Testable expectations

- After a flush, backing-store load spikes toward the full request rate and decays as the cache refills; with request collapsing the spike is at most one request per key.
- Without collapsing, concurrent misses for one key each reach the store.

6. When the cache fails

- A **cache that is down serves nothing**: every request becomes a miss, so store load jumps from (1 – h) x rate to the **full rate**. At h = 0.95 that is a **20x** load increase, usually enough to take the store down too. This is why caches must not be the only capacity (plan the store for the cold-cache case or protect it). [K, computed]
- A down cache must **not** keep answering its remembered hits (a bug this project fixed): after the failure time the hit counter must stop.
- **Fallback policy:** on a cache failure a client may fail, fall back to the store (risking overload), or serve stale data. The default is a decision. **[?]**
- Warm-up after restart or flush: hit ratio starts at 0 and recovers as popular keys reload; recovery time depends on request rate and key popularity. [K]

7. Simulator parameter map

Setting	Meaning
`cacheModel`	`declared-rate` (a fixed hit rate you set) or `derived-lru` (hit ratio emerges from an LRU over the workload's keys)
`cacheHitRate`	the declared hit probability (declared-rate model)
`cacheHitLatencyMs`	time to answer a hit
`cacheRamMb`, `valueSizeBytes`	cache capacity in entries = RAM / value size (derived-LRU model)
`requestCollapsing`	collapse concurrent misses for the same key into one backing request
`ttlSeconds`, `cacheEngine`	expiry and engine (redis, memcached)
cache flush fault	empties a cache at a time; declared-rate caches miss every arrival for the re-warm duration; derived-LRU caches re-warm from traffic
node failure	a failed cache answers no hits

8. Testable expectations (summary)

- Backing arrivals = (1 – h) x requests; latency follows the formula.
- Derived-LRU hit ratio under Zipf tracks the Che approximation and stays under the ideal top-C share.
- Cache size C and skew s each increase the hit ratio monotonically.
- Flush: hit ratio goes to 0 then recovers; request collapsing caps the stampede at one backing request per key.
- Cache down: backing load jumps to the full request rate; no hits are counted after the failure.

9. Open items

- Fetch and cite: Che et al. 2002; Vattani, Chierichetti and Lowenstein on XFetch; the Facebook memcache paper (leases, 17,000 to 1,300); the Redis eviction page's approximated-LRU section.
- Decisions `[?]`: default fallback behaviour for a down cache; whether write-back with data loss on failure should be modelled.

Sources

- Cache stampede, Wikipedia [S]
- Cache (computing), Wikipedia [S]
- Cache replacement policies, Wikipedia [S]
- A versatile and accurate approximation for LRU cache performance (arXiv 1202.3974) [S]
- Redis key eviction [P, list only]
- Scaling Memcache at Facebook (NSDI 2013) [P, abstract; leases via secondary summaries]

11. Replication, consistency and consensus

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

Covers the replication model, what a reader is allowed to see (consistency models), quorums, consensus (Raft), leader election, and locks and leases: `etcd-consul-kv`, `leader-election`, `distributed-lock`, `coordination-service`, and the replica behaviour of the storage nodes in chapter 09.

1. Why replicate, and what it costs

- **Durability** (a copy survives a failed machine), **availability** (another copy keeps serving) and **read scale** (replicas answer reads). The price: copies can **disagree**, and keeping them in step costs latency.
- The unavoidable trade: a write acknowledged to the client before all copies have it is fast but exposes **stale reads** and risks **lost writes** on failover; waiting for the copies is safe but slow, and unavailable if a copy is unreachable. [K]

2. Primary-replica replication

One **primary** accepts writes and ships them to **replicas**. Choices:

Mode	The primary acknowledges the client when…	Write latency	Loses data on primary crash?	Stale reads?
Asynchronous	it has applied the write locally	the primary's own time	**yes**, writes not yet shipped	yes, by the replication lag
Semi-synchronous	at least one replica has **received** it (relay log)	primary + one round trip	no, if that replica survives	**still yes**: received is not applied
Synchronous / quorum	a required set of replicas has applied it	primary + slowest required replica	no	no on those replicas

[S: MySQL replication discussions; Percona] Important detail: **semi-sync is a durability feature, not a freshness feature**: a replica acknowledges after writing to its relay log, not after applying, so it can be far behind in apply yet still acknowledge. If replicas are slow or down the primary falls back to async after a timeout (default 10 s in MySQL). [S]

Replication lag is how far behind a replica is. Monitoring examples: warn at about 10 s, critical at about 60 s (rules of thumb). [S]

Math [K]

- Write latency, quorum: with write quorum W of n copies (including the primary), the write waits for the **(W − 1)th fastest follower**. Round trips to followers of 1, 5 and 80 ms: a 2-of-3 quorum commits in about **1 ms** (the fastest follower); waiting for all takes **80 ms**. Five copies with follower round trips 1, 2, 60 and 70 ms and a majority of 3: the second-fastest follower, **2 ms**.
- Staleness: a read arriving t after a write sees the old value if replication lag > t. With a fixed lag of 100 ms and reads uniformly spread over the 500 ms after a write, **20%** are stale. [computed]

- Failover: a failed primary is replaced by promoting a replica; during promotion writes are rejected, and under async replication the writes the replica never received are lost (the **recovery point**). Window = detection time + promotion time. [K]

Testable expectations

- Async: write latency = primary only; reads from a replica can be stale for up to the lag; failover can lose the last (lag x write rate) writes.
- Quorum: write latency rises with the slower required replica; reads from required replicas are never stale.
- A configured failover window rejects writes for exactly that long, then service resumes.

3. Consistency models

What a read is allowed to return. From strongest to weakest, with the guarantees as the models are usually defined [S: Jepsen consistency map, relationships only; definitions K]:

Model	Guarantee	Availability under partition
Linearizable	every operation appears to take effect at one instant between its start and end, in real-time order; a read returns the latest completed write	unavailable (needs a majority)
Sequential	all processes see one order consistent with each process's own order, not necessarily real time	unavailable
Causal	if one operation could have caused another, everyone sees them in that order	sticky available
Read your writes	a client sees its own earlier writes	sticky available
Monotonic reads	a client never sees data go backwards	sticky available
Eventual	if writes stop, replicas converge	total

(Availability classes: *total* = any live node can answer; *sticky* = available if the client keeps talking to the same node; *unavailable* = needs a majority.) [S: Jepsen] Strict serializability is the transactional counterpart (serializable and linearizable). Snapshot isolation allows write skew. [S: Jepsen]

How databases deliver stronger reads from replicas [S: MySQL/ProxySQL/MongoDB mechanisms]: a replica read can wait until it has applied a given position (MySQL `WAIT_FOR_EXECUTED_GTID_SET`, MongoDB `afterClusterTime`, Raft ReadIndex), or the read goes to the primary for a short window after the user's own write ("sticky primary", often 1 to 2 s). Each costs latency or primary load.

Testable expectations

- Eventual reads from a lagging replica can return an old version; read-your-writes reads never return a version older than the session's own acknowledged write; strong reads never return anything but the newest committed version.
- Stronger models cost more read latency or capacity: a read that waits for the replica to catch up holds a worker for the wait.
- Leader reads are always up to date.

4. CAP and PACELC

- **CAP [S: Wikipedia]:** during a network **partition** a replicated store must choose between **consistency** (every read returns the latest write or an error) and **availability** (every non-failing node answers, possibly with old data). It is not "pick two of three at all times": without a partition you can have both. CAP consistency (linearizability) differs from ACID consistency, and CAP availability differs from "high availability". Many databases are configurable between the two.
- **PACELC (2010) [S]:** if **P**artitioned choose A or C; **E**lse (normal operation) choose **L**atency or **C**onsistency. The second half is the one you pay every day: stronger consistency costs round trips.

5. Quorums

For N replicas, write to W and read from R: **R + W > N** guarantees every read quorum overlaps every write quorum, so a read sees the latest acknowledged write (if no failures reshape the quorum). [S: Dynamo and course notes]

- N = 3, R = 2, W = 2: overlap (2 + 2 > 3). R = 1, W = 3: overlap, reads fast and writes need all. R = 1, W = 1: **no** overlap (2 is not > 3), fast and stale. [computed]
- **Sloppy quorums and hinted handoff** (Dynamo): when a replica is down, writes go to the next healthy nodes with a hint to hand them back; availability up, but **R + W > N no longer guarantees overlap** during failures, and divergent versions need reconciliation (vector clocks, last-writer-wins). [S]
- **Availability of a quorum.** If each replica is up independently with probability p, a quorum of k of n is available with probability the sum over i >= k of C(n, i) p^i (1 – p)^(n – i):

Cluster	Needs	p = 0.99 each	p = 0.999 each	Tolerates
1 node	1	0.99	0.999	0
3	2	**0.999702**	0.999997	1
5	3	**0.99999015**	0.99999999	2
7	4	0.99999966	1.0	3

[computed] Odd sizes are standard: 4 nodes need 3 for a majority and tolerate only 1, the same as 3 nodes.

6. Consensus: Raft

- **What it does:** a set of servers agree on one ordered log of commands so a replicated state machine behaves as one; it keeps going while a **majority** is available and **never returns an incorrect result** (it may stall instead). A cluster of n servers tolerates **floor((n – 1) / 2)** failures: 3 tolerates 1, 5 tolerates 2, 7 tolerates 3. [S: raft.github.io; derivation computed]
- **Leader election:** followers that hear no heartbeat within a **randomised election timeout** become candidates and request votes; a majority makes a leader. Typical timeouts are **150 to 300 ms**, randomised to avoid split votes, and must be much larger than the time to broadcast a heartbeat and get replies (typical broadcast time 0.5 to 20 ms). [S: Wikipedia Raft; secondary]
- **Commit rule:** an entry is committed once stored on a majority; the leader replies to the client then. **Commit latency = one round trip to the quickest majority** = the fastest follower in a 3-node cluster, the second-fastest in 5 (plus a disk sync on each). [S, K]

- **Read consistency:** a leader read is linearizable only if it confirms it is still leader (a heartbeat round, a lease, or ReadIndex). [K]
- **Availability gap during failover:** time without a leader = failure detection (up to one election timeout) + election. Plan for hundreds of milliseconds to seconds of write unavailability. [K]
- Used by etcd and Consul (`etcd-consul-kv`, `coordination-service`); ZooKeeper uses a similar protocol (ZAB). Metadata that must be correct (locks, leader identity, configuration) belongs on such a store, not on a cache. [K]

Testable expectations

- With a majority up, writes commit; with a majority down, writes are refused (never silently accepted).
- Writes to a 5-node cluster commit after the second-fastest follower acknowledges.
- Killing the leader makes writes unavailable for about the election timeout, then they resume with a new leader.

7. Locks, leases and fencing

- A **lease** is a lock that expires after a duration, so a crashed holder does not block everyone forever.
- **The failure:** holder A takes a lease, **pauses** (garbage collection, VM stall) past its expiry, B takes the lease, A wakes still believing it holds it, and both write. Leases alone do not give mutual exclusion under pauses or clock drift. [S: Kleppmann on Redlock]
- **Fencing tokens:** the lock service issues a monotonically increasing number with each grant; the **protected resource** rejects any write carrying a smaller number than one it has already seen. Needs the resource to cooperate. [S]
- **Efficiency vs correctness:** for an efficiency lock (avoid duplicate work, an occasional double run is merely wasteful) almost any lock is fine; for a correctness lock (two holders corrupt data) use a consensus-backed service (ZooKeeper, etcd) and fencing. Redlock's safety depends on timing assumptions; Redis's author and Kleppmann agree fencing is stronger. [S]
- **Contention math:** with n clients competing for one lock held for h seconds, the maximum rate of lock-protected work is 1 / h; a lock turns a parallel system into a serial one at that point (Amdahl, chapter 06). [K]

8. Leader election

A group picks one leader for a role (primary, scheduler, singleton job). It is a consensus problem: use the consensus store's lease (etcd, ZooKeeper ephemeral nodes) rather than rolling your own. Safe use requires that the leader stops acting when it can no longer prove it holds the lease, and that downstream accepts only the newest leader (fencing). [K]

9. Conflict resolution when writes are accepted in several places

Multi-leader and leaderless systems must reconcile concurrent writes: **last-writer-wins** (simple, loses data), **vector clocks** (detect concurrency, return siblings to the application), and **CRDTs** (merge deterministically). [S: Dynamo; K]

10. Simulator parameter map

Setting	Meaning
`replicationEnabled`, `replicationRole`	`primary` / `leader`, `replica` / `follower`
`replicaMembers`	the followers of a primary
`replicationLagMs`	delay before a follower applies a committed write
`writeAckPolicy`	when a write is acknowledged: `primary` (the primary alone) or `quorum`
`consensusProtocol`, `conflictResolution`	named for the replicated store
`failoverUntilMs`	the simulated time (ms from the start of the run) until which the node rejects traffic while a promotion completes
`consistencyModel`	`eventual`, `monotonic-reads`, `read-your-writes`, `strong` for follower reads
`readLatency`, `writeLatency`	separate read and write service times
lock settings	`leaseMs`, `acquireMs`, `fencing`, `lockKeyField`

The simulator tracks a **version per key**: each write the leader finishes commits a new version; a follower applies it `replicationLagMs` later; a read returns the newest version its node has applied. It counts stale reads, read-your-writes violations and monotonic-read violations, and checks single-key register linearizability on the recorded history. [repo]

11. Testable expectations (summary)

- Follower read staleness: fraction of reads that return an old version = P(lag > time since write); zero for a strong model.
- A strong follower read waits for the replica and costs extra service time equal to the wait.
- R + W > N configurations show no stale reads while all replicas are healthy; R + W <= N shows stale reads.
- A majority quorum is available with the probability in section 5; fewer than a majority up means writes are refused.
- Lock with fencing: a write carrying an older token is rejected; without fencing, a delayed holder can overwrite a newer one.

12. Open items

- Fetch and cite: the Raft paper (election timeout, broadcast time condition); Jepsen model definitions; the Dynamo paper's section 4; Kleppmann's *Designing Data-Intensive Applications* chapters 5 and 9; Gilbert and Lynch's CAP proof.
- Decisions `[?]`: whether the simulator should model election and leader unavailability for quorum stores, and sloppy quorums.

Sources

- The Raft consensus algorithm [P]
- CAP theorem, Wikipedia [S]
- Jepsen: consistency models [P, relationships only]
- Dynamo (Princeton course notes on the SOSP 2007 paper) [S]

- Percona: MySQL high availability, stale reads and how to fix them [S]
- Kleppmann: how to do distributed locking (via summaries) [S]

12. Sharding and partitioning

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

Splitting data or traffic across machines so no single one carries it all: `sharding`, `hashing`, `shard-node`, `partition-node`, and the key-routing in chapter 07. Replication (copies of the same data) is chapter 11; this chapter is about dividing **different** data.

1. Why shard, and the cost

A single node has a ceiling on storage, writes per second and memory. **Sharding** divides the keyspace so each node holds and serves a part. [K]

- **Capacity scales with the number of shards only if load is even.** Total capacity = shards x per-shard capacity **at best**; in practice the limit is the **hottest shard**.
- **Costs:** queries that span shards (fan-out, section 5), no cross-shard transactions without extra protocol, rebalancing work, operational complexity. Do not shard until one node cannot cope. [K]

2. How keys map to shards

Scheme	Shard of a key	Good at	Bad at
Hash	hash(key) mod n, or a hash range	even spread of many keys	range queries scan every shard; changing n moves almost every key with plain `mod`
Consistent hash	next node clockwise on a ring (chapter 07)	few keys move on resize; virtual nodes even the load	still uneven under skew
Range	contiguous key ranges	range scans stay on one shard	**hot spots** on monotonic keys (timestamps, auto-increment ids)
Directory / lookup	a table maps key (or key range) to shard	flexible placement, rebalancing	the directory is a dependency and must itself scale
Geo / tenant	by region or customer	data locality, isolation	uneven tenants

[K] Resize cost: adding a shard moves about **1 / (N + 1)** of the keys with consistent hashing (9 to 10 shards: 10%), but about **N / (N + 1)** with `hash mod n` (90%). [chapter 07, computed]

3. Choosing a shard key

A good key has **high cardinality** (many distinct values), **even access** (no value much hotter than others), and matches **how queries filter** so most queries touch one shard. [S: AWS DynamoDB partition-key guidance, K]

- Bad: a date as the key when all today's writes share one value (one hot shard); a boolean; a country in a global app dominated by one country.
- Fix hot keys by adding a **suffix** (key#0..key#k) and writing to a random one, then reading all k and merging: trades read cost for write spread. [K]
- A **composite key** (tenant + id) isolates tenants and spreads load within one. [K]

4. Hot keys and skew

Real access is skewed (Zipf, chapter 05). **Hashing the key spreads many keys evenly but cannot split one key.**

- With Zipf s = 1.0 over 1,000 keys, the hottest key is **13.4%** of traffic. Over 20 shards, an even share is 5%, so the shard holding that key carries **at least 2.7x** its fair share however well the rest is balanced. [computed, chapter 05]
- **Per-key limits are real:** DynamoDB throttles one partition above about **3,000 reads and 1,000 writes per second**; adaptive capacity and split-for-heat absorb moderate imbalance but **not** a single key above that ceiling. A key needing 25,000 writes/s needs the load spread over at least **25** key values. [S: AWS DynamoDB docs]
- **Mitigations:** key suffixing, caching the hot items (chapter 10), replicating hot shards for reads, splitting the hot range.

Testable expectations

- Hash routing over many uniformly distributed keys gives shard shares of 1/n within the sampling error plus the ring's own spread: a consistent-hash ring with V virtual nodes per shard gives each shard an arc with relative standard deviation about 1/sqrt(V) (Karger et al.; Dynamo), so V = 64 gives a standard deviation of about 12.5% of the even share (individual shards can be 25% off) and V = 256 about 6%. Under Zipf, the busiest shard's share is at least the hottest key's share.
- The busiest shard saturates first; total throughput is bounded by the hottest shard's capacity divided by its share of traffic: with a hottest shard of capacity C carrying share f, system throughput <= C / f. Worked: C = 1,000 req/s per shard and f = 0.134 gives at most **7,463 req/s** in total, however many shards exist. [computed]

5. Scatter-gather queries and the tail

A query that must consult all n shards finishes when the **slowest** responds, so its latency is the **maximum** of n shard latencies.

- If shard latencies are independent exponentials with mean mu, **E[max] = mu x H_n** where H_n is the nth harmonic number. n = 1: 1.0 x mu; n = 10: **2.93 x**; n = 100: **5.19 x**; n = 1,000: **7.49 x**. For large n, H_n is about ln n + 0.577. [S: order statistics of exponentials; computed]
- The **p99 of the maximum** grows too: for exponentials p99 of max = mu x (–ln(1 – 0.99^(1/n))): n = 1: 4.6 x; n = 10: 6.9 x; n = 100: **9.2 x**; n = 1,000: 11.5 x. [computed]
- And a rare slow shard becomes the common case: if each shard is slow with probability q, a query across n shards is slow with probability 1 – (1 – q)^n; q = 1%, n = 100: **63%** (chapter 01 section 6, *The Tail at Scale*). [K]
- **More shards do not make a scatter-gather query faster**; past a point they make its tail worse. Hedged requests, partial results and targeting only the shards that hold the data reduce it. [K]
- The simulator models the consequence by adding a penalty drawn as the maximum of `shardCount` exponential samples of mean `perShardLatencyMs`. [repo]

Testable expectations

- Mean added latency of a scatter-gather over n shards = mu x H_n; it grows about logarithmically in n.

- Doubling shards does not halve query time; it adds about mu x ln 2 to the mean.

6. Rebalancing and resharding

- Adding capacity means moving data: bytes moved = data x fraction moved; time = bytes / transfer rate while still serving traffic. 200 GB, 10% moved at 100 MB/s: **200 s** of background copy. [computed]
- **Many small partitions** (more logical partitions than machines, e.g. 1,024 virtual shards over 10 machines) make rebalancing move whole partitions without re-hashing keys. [K]
- **Hot shard splitting:** a shard whose traffic exceeds a threshold relative to the average (the simulator's `hotShardThresholdRatio`, with a minimum sample count) is split. [repo]
- During a rebalance, routing must stay consistent (a key never served from two places) and a moved key's cache entries are cold. [K]

7. Partitions in messaging (preview of chapter 13)

A Kafka-style topic is divided into **partitions**; ordering holds **within** a partition, a key always maps to the same partition, and parallelism of consumers is capped by the partition count. Choosing the partition count is choosing the maximum parallelism: more partitions raise throughput and rebalance cost. [K, chapter 13]

8. Cross-shard operations

- A **transaction touching several shards** needs a protocol (two-phase commit, sagas): extra round trips, blocking on failure, a coordinator. Cost grows with shards touched. [K]
- **Joins** across shards require shipping data or co-locating related rows (same shard key). Choose the key so related data lives together. [K]
- **Secondary indexes** are either local (query every shard) or global (a separate sharded index, updated asynchronously). [K]

9. Simulator parameter map

Setting	Meaning
`routingKeyField`	request field whose value picks the shard
`missingRoutingKeyPolicy`	what to do when a request has no key
`autoShardRebalance`, `hotShardThresholdRatio`, `hotShardMinSamples`, `shardSplitKeyField`	automatic hot-shard splitting
`shardCount`, `perShardLatencyMs`	scatter-gather fan-out and per-shard mean latency
workload `keyspace {field, size, skew}`	the keys requests carry and their Zipf skew
routing strategies `ip-hash`, `sticky`, consistent hash	how a node maps a key to a target (chapter 07)

10. Testable expectations (summary)

- Even keys: shard shares 1/n within 3 standard errors. Zipf keys: hottest shard >= hottest key's share.
- System throughput under skew <= capacity of the hottest shard / its traffic share.
- Scatter-gather mean = mu x H_n, p99 as above.
- Adding a shard moves about 1/(N + 1) of the keys (consistent hashing) or N/(N + 1) (`mod`).
- Splitting a hot shard lowers its share.

11. Open items

- Fetch and cite: the DynamoDB partition-key page (limits change); the Dean and Barroso paper's fan-out numbers; Vitess or Citus docs for resharding mechanics.
- Decisions `[?]`: how a split shard's keys are divided in the simulator and whether splitting is instantaneous.

Sources

- DynamoDB partition-key design (uniform load) [P]
- Consistent hashing, Wikipedia [S]
- Probable Approximate Coordination (expected maximum of exponentials) [S]
- The Tail at Scale, CACM 2013 [P, abstract]

13. Messaging and streaming

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 7 `MessagingType`s: `queue`, `pub-sub`, `stream`, `event-bus`, `event-sourcing-store`, `message-broker`, `task-queue`. Related: change data capture (chapter 09), request batching and Kafka edges (chapter 08), consumers as compute nodes (chapter 06), and idempotency (chapter 20).

1. Why put a broker between services

A **broker** decouples producer and consumer in time, rate and failure: the producer hands off and continues; the consumer processes when ready; a burst waits in the buffer instead of overwhelming the consumer. [K]

- Price: another system to run, **at-least-once** delivery needing idempotent consumers, ordering limits, and added end-to-end latency (chapter 08: an asynchronous edge hides the downstream wait from the caller but not the load).
- **Backpressure** is the alternative to an unbounded buffer: a bounded queue pushes back on producers (reject, block, or shed) so memory does not grow without limit. [K]

2. The shapes

Type	Delivery	Consumers see	Retention	Typical use
Queue (work queue)	each message to **one** consumer	competing consumers share the work	until acknowledged, then deleted	job processing, task queue
Pub/sub	each message to **every** subscriber	independent copies per subscription	usually short	notifications, fan-out
Stream / log (Kafka-style)	appended to a partitioned **log**; consumers track an **offset**	each consumer group reads the whole log independently	time or size based (days), replayable	event streaming, CDC, analytics
Event bus	routing by event type or rule to interested parties	subscribers by pattern	short	domain events
Event-sourcing store	the log **is** the system of record; state is a fold over events	any consumer can rebuild state	forever	auditable state, replay
Message broker	queues and exchanges with routing (AMQP-style)	per binding	until consumed	enterprise integration
Task queue	one message to one worker with retries and scheduling	workers	until done	background jobs

[K] The simulator treats queue, stream, broker and background-worker node types as **asynchronous boundaries** (chapter 08): the producer is acknowledged at append and consumption is an independent flow. [repo]

3. Delivery guarantees

Guarantee	Meaning	How it happens
At most once	may lose, never duplicates	commit the offset / delete the message **before** processing; a crash after loses the message
At least once	never loses, may duplicate	commit / delete **after** processing; a crash after processing but before the commit redelivers
Exactly once	each message's effect happens once	at-least-once **plus** idempotent processing or transactions

[S: Kafka consumer-semantics guides] "Exactly once" is usually **effectively once**: a Kafka producer with idempotence and transactions gives exactly-once between Kafka topics, but effects on an external system need idempotent consumers (an idempotency key, chapter 20). A third-party claim that SQS FIFO is exactly-once is not in AWS's own wording; FIFO gives strict ordering per message group and deduplication within a window. [S]

3.1 Visibility timeout and redelivery (SQS style)

A received message is **hidden** for the visibility timeout (default **30 s** in SQS); if the consumer has not deleted it by then, it becomes visible again and another consumer receives it. [P: AWS SQS docs]

- If processing takes longer than the timeout, the message is **redelivered while the first consumer is still working**: a duplicate by design. Standard SQS queues can also deliver a message more than once within the timeout, so consumers must be idempotent. [P]
- If processing time T is exponential with mean 20 s and the timeout is 30 s, the chance a message exceeds the timeout is e^(−30/20) = **22%** (22% of messages will be duplicated). [computed] Set the timeout above a high percentile of processing time (chapter 03).
- **Dead-letter queue (DLQ):** after `maxReceiveCount` failed receives (for example 5) the message is moved to a DLQ so one **poison message** cannot loop forever, loading the consumers 5 times and then stopping. [P/K]

4. Consumer groups, partitions and ordering (Kafka-style)

- A topic has **p partitions**. Each partition is an ordered log. **Ordering is guaranteed only within a partition.** A message key hashed to a partition keeps all messages of one key in order. [S/K]
- A **consumer group** shares the work: each partition is read by **exactly one** consumer in the group at a time, so **parallelism is capped by p**: more consumers than partitions leaves the extras idle. Different groups each read everything independently. [K]
- **Offsets:** a consumer's position in each partition; committing it is what defines at-most-once or at-least-once (section 3). [S]
- **Rebalance:** when consumers join or leave, partitions are reassigned and consumption **pauses** briefly; frequent rebalances look like a throughput dip. [K]
- **Producer acknowledgement:** acks = 0 (do not wait: fastest, may lose), 1 (leader only: loses if the leader dies before replicating), all (every in-sync replica: safest, slowest). [K]
- **Partition count** is chosen for the maximum parallelism and throughput you will need; it is awkward to change later (it breaks key-to-partition mapping). Chapter 12 section 7.

5. Queue maths

Every consumer set is a queue (chapter 01). Let producers arrive at rate lambda and consumers drain at rate c x mu.

- **Backlog grows at (lambda − c mu)** when lambda exceeds consumer capacity; it drains at (c mu − lambda) when capacity exceeds arrivals. Producers at 1,200/s against a 1,000/s consumer fleet for 60 s leave **12,000 messages** of lag; if capacity then rises to 1,500/s with arrivals back at 1,000/s, it drains at 500/s in **24 s**. [computed]
- **Lag in time = lag in messages / consume rate.** 12,000 messages at 1,000/s is 12 s behind real time. **Consumer lag** (messages behind the latest offset) is the key health metric. [K]
- **Consumers needed = arrival rate x processing time / target utilisation** (Little's Law): 10,000 msg/s x 50 ms / 0.7 = **714** concurrent workers. [computed]
- **Stability:** a queue is stable only if lambda < c x mu. Below that, delay stays bounded and follows the formulas of chapter 01; above it, an unbounded backlog (or rejected messages if bounded) is the only outcome.
- **Little's Law on the buffer:** messages in queue = arrival rate x time in queue. 1,000/s with a 5 s wait holds 5,000 messages.
- **Storage:** retention storage = rate x size x retention x replication: 10,000 msg/s x 1 KB x 7 days x 3 replicas = **18.1 TB**. [computed]
- **Fan-out amplification:** pub/sub to s subscriptions multiplies delivered load by s: 1,000 msg/s to 5 subscribers is 5,000 deliveries/s. [computed]
- **Batching:** grouping messages cuts per-message overhead but adds up to `linger` of latency (chapter 08 section 8).

Testable expectations

- Producer acknowledgement time is independent of consumer speed; the **consumer backlog** is what grows when consumers are slow.
- Lag after t seconds = (lambda − capacity) x t when overloaded; zero when capacity exceeds arrivals.
- Produced = consumed + still in the log (conservation); in a stream, committed + lag = produced.
- With p partitions, a group of more than p consumers has idle members and no extra throughput.
- Messages per key are consumed in order; across partitions no order is promised.
- Redelivery fraction = P(processing time > visibility timeout).

6. Event sourcing, replay and snapshots

- The **event log is the source of truth**; current state = fold of events. To recover, **replay** events from the start or from a **snapshot**. [K]
- Replay time = events replayed x time per event. 10^6 events at 0.01 ms each = **10 s**; with a snapshot every 10,000 events at most 10,000 are replayed: **0.1 s**. The snapshot interval trades storage and write cost for recovery time. [computed]
- Events are immutable; corrections are new events; schemas evolve through a registry (chapter 09). [K]

7. Windowing and stream processing

Streaming analytics groups events into **windows**: tumbling (fixed, non-overlapping), sliding (overlapping), session (gaps). Events in a tumbling window of w seconds at rate r: r x w = 500/s x 10 s = **5,000**. Late events need a watermark or a lateness bound; state grows with window length and keys. [K, chapter 18]

8. Change data capture

CDC turns a database's change log into a stream, keeping derived stores in step without dual writes. Ordering is per key or partition; consumers must be idempotent and handle redelivery; a slow consumer lags without affecting the database. [K, chapter 09]

9. Failure modes

Failure	Effect	Defence
Consumer slower than producers	backlog and lag grow, then storage or memory limit	scale consumers (to partition count), shed or back-pressure, alert on lag
Poison message	endless redelivery, blocks a partition (ordering)	DLQ after maxReceiveCount
Visibility timeout too short	duplicates	timeout above processing p99, idempotent consumers
Broker down	producers cannot enqueue	producer-side buffering, retries with backoff and idempotence, replication
Rebalance storm	repeated pauses	stable membership, static assignment
Hot partition	one partition saturates its single consumer	better key, more partitions (chapter 12)

10. Simulator parameter map

Setting	Meaning
node type `queue`, `stream`, `message-broker`, `pub-sub`	asynchronous boundary; producer acknowledged at append
`queue.workers`, `processing.distribution` on the queue	consumer concurrency and processing time (backlog grows when slow)
`deliverySemantics`	at-most-once, at-least-once or similar delivery setting
`visibilityTimeoutMs`, `maxReceiveCount`, `dlqNodeId`	redelivery, poison-message limit, dead-letter target
`partitionCount`, `partitionKeyField`	stream partitions and the key that selects one
`consumerGroup`, `consumerGroupMode`	competing consumers within a group, independent groups
`retentionMs`	how long the log is kept
`consumerOrdering`	`parallel`, `per-partition`, `per-key`
batching `batchSize`, `batchWindowMs`, `fixedCostMs`	micro-batching of consumer work
`snapshotEvery`, `replayCostPerEventMs`	event-sourcing replay cost
`windowMs`	streaming window length
dedup `dedupKeyField`, `dedupWindowMs`	idempotency at the consumer (chapter 20)

Known probe result recorded in the project: a stream with 200 produced and a slow consumer shows 99 committed and 101 lag, summing to the produced count. A queue's metrics count the producer-side ack and the consumer-side delivery separately, which reads as roughly double the arrivals and half the service time. [repo: `docs/engine-probe-findings.md`, section 3]

11. Testable expectations (summary)

- Producer latency excludes consumer processing; consumer backlog grows at (arrival – drain) when drain < arrival.
- Conservation: produced = committed + lag (+ dead-lettered).
- Redelivery share = P(T > visibility timeout); a message is dead-lettered after maxReceiveCount failures.
- Ordering: within a partition or key preserved; across partitions not.
- Consumers beyond the partition count add no throughput.
- Fan-out: delivered load = subscriptions x produced load.
- Replay time = events since last snapshot x per-event cost.

12. Open items

- Fetch and cite: Kafka's own design documentation (delivery semantics, consumer groups); SQS developer guide pages for `maxReceiveCount` and FIFO deduplication; the Kafka transactions design doc; a source for rebalance cost.
- Decisions `[?]`: whether the stream and queue counters should be labelled "produced" and "delivered" (carried over from the engine probe notes).

Sources

- Amazon SQS visibility timeout [P]
- Amazon SQS queue types [P]
- Delivery semantics for Kafka consumers (Conduktor) [S]
- Interpreting Kafka's exactly-once semantics [S]
- KIP-1030: Kafka defaults [P]

14. Orchestration and infrastructure

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 13 `OrchestrationTypes`: `kubernetes-cluster`, `container-registry`, `service-registry`, `tool-registry`, `config-store`, `secrets-manager`, `cluster-autoscaler`, `agent-orchestrator`, `orchestrator-scheduler`, `ci-cd-runner`, `iac-engine`, `container-runtime`, `provisioner`. Horizontal scaling of one service (autoscaler) lives here too; per-instance capacity is chapter 06.

1. What an orchestrator does

An orchestrator keeps a **desired state** (N replicas of a workload, each needing some CPU and memory) true on a fleet of machines. It is a set of control loops: observe, compare with the desired state, act. Every loop has a **period** and a **reaction lag**, so a response to a load change is never instant. [K]

2. Horizontal autoscaling (HPA)

The Kubernetes Horizontal Pod Autoscaler computes the replica count from the ratio of the observed metric to its target: [P: Kubernetes HPA docs]

desiredReplicas = ceil(currentReplicas x currentMetricValue / desiredMetricValue)

- **Sync period:** the controller runs every **15 s** by default. [P]
- **Tolerance:** if the ratio is within **10%** of 1.0 (0.9 to 1.1), nothing happens. [P]
- **Stabilization window:** scale-down looks back **300 s** and takes the highest recommendation, so a brief dip does not remove capacity; scale-up has **0 s**. [P]
- **Default rate policies:** scale-up may add the larger of **100% of current replicas or 4 pods every 15 s**; scale-down may remove **100% per 15 s** (held back by the 300 s window). [P]
- **Missing or not-ready pods:** when scaling up they are assumed to use **0%** of the target, when scaling down **100%**, which damps the decision. A just-started pod is ignored for CPU for 30 s (initial readiness delay) and 5 min (CPU initialisation period). [P]
- Utilization is measured against the pod's **resource request**, not the node. [P]

Worked: 4 replicas at 90% against a 70% target: ceil(4 x 90 / 70) = ceil(5.14) = **6**. 10 replicas at 30%: ceil(10 x 30 / 70) = **5**, but only after the 300 s window agrees. 4 replicas at 76%: ratio 1.086, inside tolerance, **no change**. [computed]

Time to scale out from 2 replicas to over 100 at the default policy: each step adds max(current, 4): 2 to 6 to 12 to 24 to 48 to 96 to 192, **6 steps = 90 s** of controller time, plus pod start-up and readiness for each wave. A spike faster than that overloads the service in the meantime. [computed]

Why autoscalers oscillate: a lag in the loop plus a metric that responds to the action (more replicas lowers utilization) gives overshoot; the stabilization window and tolerance are the damping. [K]

3. Cluster autoscaling and node provisioning

The Cluster Autoscaler adds nodes when pods are **unschedulable** and removes nodes that are under-used. [P: autoscaler FAQ]

- Scan interval **10 s**. Scale-up simulates a template node to see whether it would let the pending pods run. [P]
- A node is a scale-down candidate when its requested CPU and memory are under **50%** of allocatable (`--scale-down-utilization-threshold`, default 0.5) and every pod on it can move; it is removed after being unneeded for **10 minutes**. [P]
- A new node is expected within **15 minutes** (`--max-node-provision-time`); on GCE it usually takes **3 to 4 minutes** until pods can schedule, about 5 minutes end to end. [P]

So a traffic spike that needs new machines is served by the existing fleet for several minutes: the burst must be absorbed by headroom, queues or shedding, not by scaling. Scale-down is slow on purpose (10 minutes) to avoid flapping. [K]

4. Scheduling and bin packing

The scheduler places a pod only on a node with enough free **CPU and memory** (and satisfying constraints). A pod that fits nowhere stays **Pending** and adds no capacity. [P]

- **Scoring strategies:** `LeastAllocated` spreads load across nodes; `MostAllocated` prefers highly utilized nodes (packs, helping scale-down of idle nodes); `RequestedToCapacityRatio` applies a configurable shape. Per-resource utilization is requested x 100 / capacity; the node score is the weighted average sum(score_i x weight_i) / sum(weight_i). [P: Kubernetes bin-packing docs]
- **Fragmentation:** pods of 3 vCPU on 8 vCPU nodes fit **2 per node**, stranding 2 vCPU (25%). 10 such pods need **5 nodes**, not the 3.75 that the sums suggest. 3 nodes of 4 vCPU hold only three 3-vCPU pods, not four. [computed]
- **Packing vs spreading:** packing needs fewer nodes (cost, scale-down) but concentrates pods, so one node failure takes a larger share of a workload; spreading across nodes and zones costs capacity but limits blast radius. [K]
- Optimal bin packing is NP-hard; first-fit-decreasing uses at most about 11/9 of the optimum plus a constant. [K]

5. Node failure and recovery

A node that stops reporting is marked **NotReady** after the node-monitor grace period (**40 s**), and its pods are evicted when their default not-ready toleration (**300 s**) runs out, so replacement pods start only after about **340 s**; then each needs scheduling room, an image pull and start-up. [S: Kubernetes taints docs, kube-controller-manager defaults; the 40 s and 300 s values were not on the fetched page text and are [K]]

During the 40 s before NotReady the dead pods still sit in service endpoints, so a share of requests equal to (pods on the failed node / ready pods) errors out. A workload with all replicas on one node is down for the whole 340 s plus start-up. [K, computed]

Image pull: time = image size / bandwidth: 500 MB at 1 Gbps (125 MB/s) is 4 s; layers cached on the node cost nothing. A new node pulls every image cold. [K]

6. The rest of the control plane

Component	What it does	What it costs the request path
Container registry	stores images; serves pulls	nothing at request time; adds to pod start-up (pull time); an outage blocks scale-out and replacement but not running pods
Service registry	lookup of healthy instances	a stale entry sends traffic to a dead instance until its TTL or health check expires
Config store, secrets manager	serve configuration and secrets	read at start-up or refreshed on an interval; if read per request, it is a dependency with its own latency and availability
CI/CD runner	executes builds and deploys	a queue: builds arrive at rate lambda, each takes a runner for service time s; M/M/c applies (chapter 01)
IaC engine, provisioner	create machines and resources	provisioning latency (VM boot about 1 to 2 min, managed database 5 to 20 min) [K]
Agent orchestrator, tool registry	route work among AI agents and tools	chapter 18

CI runner example: builds at 6 per hour, 10 minutes each, 2 runners: offered load a = 1 Erlang, utilization 0.5, Erlang C = 1/3, mean wait = C / (c x mu - lambda) = (1/3) / (12 - 6 per hour) = **3.3 minutes**. [computed]

7. Failure modes

- **Pending pods:** not enough room (fragmentation, node limit reached): the scale-up asked for does not happen. Visible as desired replicas above ready replicas.
- **Autoscaler lag:** metric window plus sync period plus pod start-up: the service runs hot for minutes after a step in load.
- **Flapping:** scale-down too eager; fixed by the stabilization window.
- **Thundering start-up:** many replicas starting together all pull images and warm caches at once (chapter 10).
- **Cascading eviction:** a node under memory pressure evicts pods, which land on other nodes and push them over.

8. Simulator parameter map

Setting	Meaning	Real counterpart
`autoscaleMinInstances`, `autoscaleMaxInstances`	bounds on instance count; max enables the loop	HPA minReplicas / maxReplicas
`autoscaleTargetUtilization` (0.7)	target utilization	HPA target
`autoscaleCooldownMs` (5000)	loop period	sync period (15 s)
`autoscaleStep` (1)	instances added or removed per tick	HPA rate policy
`placementStrategy`	`spread` or `bin-pack`	scheduler scoring
`podStartupMs` (5000)	time from placement to ready	image pull + start
`rescheduleDelayMs` (340000)	failure detection + eviction	40 s + 300 s
`clusterMaxMachines`, `machineProvisionMs` (90000)	node limit, boot time	Cluster Autoscaler
`machineFailureAtMs`, `machineFailureCount`, `machineRecoveryAtMs`	scripted machine loss	node failure

Setting	Meaning	Real counterpart
`scheduledOn`	the cluster a workload is placed on	namespace / node pool

The simulator's autoscaler adds or removes `autoscaleStep` instances per tick toward the target band (scale up above the target, scale down below half of it) rather than computing the proportional formula, so it converges linearly, not in one calculation. [repo]

9. Testable expectations (summary)

- Proportional rule: replicas needed = ceil(load / (target utilization x capacity per replica)); a controller at that count sits at or just under the target.
- Inside the tolerance band no scaling; scale-down is slower than scale-up.
- Capacity follows demand with a lag of at least one period plus start-up; a faster spike than the lag overloads the service.
- A pod that fits on no node adds no capacity; fragmentation makes cluster capacity lower than the arithmetic sum.
- A machine failure removes its pods at once and replacements arrive after detection plus start-up; recovery time is at least the detection delay.
- Bin-packing uses fewer machines than spreading for the same workload; a machine loss under bin-packing removes a larger share.
- Node provisioning time delays scale-up for a cluster that must grow.

10. Open items

- Fetch and cite: the Kubernetes taints page's "taint based evictions" section and the node-monitor flags; the Cluster Autoscaler provisioning figures for other clouds.
- Decisions `[?]`: whether the simulator's step autoscaler should follow the proportional HPA formula with tolerance and stabilization; whether scale-down should have a stabilization window.

Sources

- Horizontal Pod Autoscaling, Kubernetes [P]
- Resource bin packing, Kubernetes [P]
- Taints and tolerations, Kubernetes [P]
- Cluster Autoscaler FAQ [P]
- kube-controller-manager reference [P]

15. Security and identity

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 11 `SecurityType`s: `iam-rbac`, `waf`, `firewall`, `bastion-host`, `certificate-authority`, `secrets-rotation`, `kms-security`, `dlp-inspection`, `identity-provider`, `siem`, `privilege-escalation-control`. Related: TLS handshake round trips (chapter 08), rate limiting and abuse control (chapter 02), certificates and DNS (chapter 21), third-party identity (chapter 20).

1. Security as a cost on the path

Security controls are not free boxes: each one adds latency, consumes CPU, can **reject** traffic (rightly or wrongly) and can fail. A security hop therefore shows up in the same three places as any other component: latency, capacity and error rate. [K]

Control	Cost on a request	Can reject	Failure presents as
Authentication (token check)	local signature verification (microseconds to a millisecond) or a remote call to the identity provider (network round trip)	401 (no or bad credential)	login outage, if every request needs a live provider call
Authorization (IAM, RBAC)	policy evaluation, usually microseconds	403	wrong permission denied or granted
WAF	inspects headers and body; cost grows with request size and rule count	yes: blocks matches	false positives block legitimate users
Firewall, security groups	per-packet rule match, negligible in hardware	yes	silent drops look like timeouts
TLS termination	handshake CPU and round trips (chapter 08)	handshake failures	expired certificate = hard outage
KMS, encryption	an encrypt or decrypt call; hardware-backed, quota-limited	throttling	KMS throttling stalls every dependent request
DLP inspection	scans payloads for sensitive data; cost grows with payload bytes	yes	large uploads slow down
SIEM	ingests security events asynchronously	no	dropped or delayed events, not user-visible errors

2. Authentication and authorization

- **Authentication** proves who the caller is; **authorization** decides what they may do. 401 means no valid identity, 403 means identified but not permitted. [K]
- **Token validation.** A signed token (for example a JWT) is verified **locally** with the issuer's public key: cheap, no network, but revocation is not seen until expiry. An opaque token needs **introspection**: a call to the identity provider per request, unless cached. [K]
- **Expiry and scopes.** A token has a lifetime; an expired one is rejected; a token missing a required scope is rejected with 403. [K]
- **Policy evaluation (AWS IAM).** The default is **deny**; an allow in an identity-based or resource-based policy grants access; an **explicit deny overrides any allow**; permissions boundaries and

organization service-control policies **intersect** with identity permissions (the action must be allowed by all). [P: AWS IAM policy evaluation logic]
- **RBAC** maps users to roles and roles to permissions; least privilege means each role has only what it needs. [K]

Remote-validation arithmetic (Little, chapter 01): 2,000 requests/s each calling an identity provider that answers in 5 ms keeps **10 introspection calls in flight** and makes the provider a dependency of every request. Caching decisions for 60 s with 1,000 distinct tokens cuts that to about 1,000 / 60 = **17 calls/s**. [computed]

3. WAF and inspection

A web application firewall matches requests against rules. The OWASP Core Rule Set uses **anomaly scoring**: rules do not block individually; each match adds points to a score and a request is blocked when the score reaches a threshold (defaults **5 inbound**, **4 outbound**). **Paranoia levels** 1 to 4 enable progressively more rules: higher levels catch more attacks and produce more false positives, and common practice is to start with a very high threshold and lower it while tuning. [S: OWASP CRS documentation]

- **Cost:** per-request inspection latency that rises with the number of rules and with body size. [K]
- **False positives:** legitimate requests blocked. If a fraction f of legitimate traffic is blocked, availability for those users is 1 - f however healthy the backend. [K]
- **False negatives:** attacks that pass. The reason a WAF is one layer, not the defence. [K]
- Expected blocked traffic = block rate x requests (a Bernoulli thinning); expected added latency = inspection time x passed requests. [computed]

4. Keys, certificates and secrets

- **Envelope encryption.** A key-management service protects a **root key** that never leaves hardware (AWS KMS: FIPS 140-3 Level 3 modules); data is encrypted with a **data key**, which is itself encrypted by the root key. The expensive KMS call is made per data key, **not per request**. [P: AWS KMS developer guide]
- **KMS on the hot path.** If every request makes a KMS call, request rate becomes KMS rate and per-account quotas (thousands of requests per second, shared across keys) become the capacity limit; caching data keys for seconds to minutes removes that. [K: quota values are provider- and region-specific and not verified here]
- **Certificate authority.** Issues certificates with a validity period; clients check expiry and revocation (CRL or OCSP, an extra lookup unless stapled). An expired certificate fails every new connection. [K]
- **Secrets rotation.** Rotating without downtime needs an **overlap**: the old and new secret both valid until all clients have refreshed. Rotation faster than client refresh causes authentication failures. [K]
- **Bastion host, privilege-escalation control:** access paths for operators; they add steps (latency for humans) and an audit trail, not request-path cost. [K]

5. Failure modes

- **Identity provider outage:** every new login and every uncached introspection fails; locally validated tokens keep working until they expire.

- **Over-eager WAF:** after a rule update a share of legitimate traffic gets 403; error rate rises with no backend fault.
- **KMS throttling:** latency and errors on every encrypt or decrypt path.
- **Certificate expiry:** all new TLS connections fail at one instant.
- **Revocation lag:** a revoked token is still accepted until its cache or lifetime runs out.

6. Simulator parameter map

Setting	Meaning
`authenticationEnabled`, `authTokenField`, `authStatusField`, `authRequiredScopes`	token present, valid, unexpired and carrying the scopes; otherwise rejected with a missing/invalid/expired reason
`inspectionMs`, `blockRate`	WAF-style scan latency per request and a fraction blocked (`inspection_blocked`)
`cryptoMs`	CPU cost of an encrypt/verify/sign per request, added to service time

[repo] The trait takes the credential from request metadata, so a workload models a population of valid, expired and invalid callers.

7. Testable expectations (summary)

- A request without a credential is rejected; an expired or invalid one is rejected with its own reason; a missing scope is rejected.
- The fraction blocked by an inspection hop equals the configured block rate within sampling error; passed requests pay the inspection time.
- Added path latency equals the sum of the inspection and crypto costs configured on the path.
- A remote-validation dependency adds its latency to every request; a cache removes it for repeated tokens.
- An explicit deny overrides an allow; with no allow the default is deny.
- A security hop with a block rate lowers goodput by that fraction while leaving the backend's load lower by the same amount.

8. Open items

- Fetch and cite: JWT (RFC 7519) validation rules; OCSP and CRL behaviour; the KMS request-quota table; CRS performance measurements.
- Decisions `[?]`: whether a KMS node should enforce a request quota (the trait comment says quota-limited; the cost is modelled, the quota is not); whether authentication should model a remote identity-provider call with caching.

Sources

- AWS IAM policy evaluation logic [P]
- AWS KMS developer guide: overview [P]
- OWASP CRS: anomaly scoring [S]
- Handling false positives with the OWASP ModSecurity Core Rule Set (netnea) [S]

16. Observability

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 9 `ObservabilityTypes`: `centralized-logging`, `distributed-tracing`, `metrics-store`, `alerting-hook`, `dashboard`, `rum-monitoring`, `health-check-manager`, `safety-observability-mesh`, `profiling-service`. The simulator's own output metrics are defined in chapter 27; health-check detection delay in chapters 02 and 07.

1. Logs, metrics, traces

Signal	What it is	Cost driver	Good for
Metrics	numeric time series (counters, gauges, histograms)	cardinality (distinct label combinations)	alerting, trends, SLOs
Logs	discrete events with text	volume (bytes per event x events)	detail about one event
Traces	the path of one request across services as spans	spans per request x request rate	where time went
Profiles	where CPU or memory goes inside a process	sampling frequency	optimising code

[K] Observability is the ability to ask new questions of a running system from its outputs; monitoring is watching known failure signals.

2. The four golden signals

Google's SRE book names four: [P: SRE book, Monitoring Distributed Systems]

- **Latency:** time to serve a request; track successful and failed requests **separately**, since a fast error can flatter an average.
- **Traffic:** demand, for example requests per second.
- **Errors:** rate of failed requests, explicit, implicit (wrong content with a success code) or by policy (slower than the commitment).
- **Saturation:** how full the most constrained resource is; systems degrade before 100%.

Averages hide tails: a 100 ms mean can coexist with requests taking 5 s; record **histograms** (counts per bucket, roughly exponential boundaries) and read percentiles. [P]

3. Percentiles from histograms

A histogram stores counts per bucket. `histogram_quantile` assumes observations are spread evenly inside the bucket containing the quantile and **interpolates linearly**; the error is bounded by the **width of that bucket**. With buckets at 200 ms and 300 ms, a true p95 of 220 ms can only be placed in that 100 ms range (the page's example estimated about 295 ms). [P: Prometheus histograms and summaries]

- **Never average percentiles** across instances: the result is statistically meaningless. Sum bucket counts across instances (same boundaries), then compute the quantile. [P]

- Summaries compute quantiles in the client: cheap to query, but not aggregatable. [P]

4. SLIs, SLOs, error budgets and burn rate

- **SLI:** a measured ratio, for example good requests / total requests. **SLO:** the target for it over a window, for example 99.9% over 30 days. The **error budget** is 1 - SLO. [K]
- Budget in time: 30 days = 43,200 min, so 99% allows **432 min (7.2 h)**; 99.9% allows **43.2 min**; 99.99% allows **4.32 min**. [computed]
- **Burn rate** is how fast the budget is consumed relative to the SLO: 1 means the whole budget lasts exactly the window. Budget consumed = burn rate x alert window / period. Time to exhaustion = period / burn rate: burn rate 2 gives 15 days, 10 gives 3 days, 1,000 gives 43 minutes. [P: Google SRE workbook, Alerting on SLOs]
- **Error rate that burns at rate B** = B x (1 - SLO). For a 99.9% SLO, burn rate 14.4 is an error rate of **1.44%**. [P]
- **Multiwindow, multi-burn-rate alerts** (99.9% SLO, 30 days): [P]

Severity	Long window	Short window	Burn rate	Budget consumed
Page	1 hour	5 minutes	14.4	2%
Page	6 hours	30 minutes	6	5%
Ticket	3 days	6 hours	1	10%

The alert fires only when **both** windows exceed the threshold: the long window gives significance, the short window makes the alert reset quickly after the problem ends. [P]

Alerting philosophy: page on **symptoms** (users affected), use causes for debugging; every page must be urgent, actionable and need human judgement. [P: SRE book]

5. Sampling, volume and cost

- **Head sampling** decides at the root span, cheaply (dropped traces create no spans) but blind to the outcome; **tail sampling** decides after the trace completes, so it can keep every error and slow trace, but must buffer all spans in a collector. Sampling by trace id lets every service reach the same decision without coordination. [S: Uptrace, OpenSearch, Datadog guides]
- Dapper kept overhead low and deployment ubiquitous by sampling and by instrumenting a small number of common libraries. [P: Dapper paper abstract]
- **Capture probability.** With head sampling rate p, the chance of capturing at least one of k occurrences of a rare event is 1 - (1 - p)^k. At p = 1%, seeing it with 99% probability needs k >= **459** occurrences. [computed]
- **Volume.** Log bytes/day = events/s x bytes x 86,400: 5,000 req/s x 1 KB is 5 MB/s = **432 GB/day**. Trace storage = request rate x spans per trace x bytes per span x sampling rate. [computed]
- **Cardinality.** Metric cost grows with the product of label value counts; a per-user label turns a few series into millions. [K]
- **Observer effect.** Synchronous logging or export adds latency to the request; asynchronous export through a bounded buffer drops events under overload instead, and the application is unaffected. [K]

6. Health checks and real-user monitoring

- A health check's **detection time** is interval x consecutive failures needed (chapter 02): 10 s x 3 = up to 30 s of traffic to a dead target. [K]
- **RUM** measures what users experience in the browser (page load, errors) and is the only signal that includes the last mile; it is sampled and sent asynchronously. [K]
- **Continuous profiling** typically costs a few percent CPU at sampling rates of tens of Hz. [K]

7. Failure modes

- **Telemetry pipeline overload:** collectors drop events; a dashboard goes blank or flat exactly during the incident.
- **Averaged percentiles** or too-wide buckets hide a tail problem.
- **Alert fatigue:** thresholds on causes, not symptoms.
- **Cardinality explosion:** the metrics store runs out of memory or the bill spikes.
- **Blind spots:** sampling that drops the rare error you care about.

8. Simulator parameter map

Setting	Meaning
`telemetryAsyncIngest`	exporter sends through a bounded buffer; drops instead of rejecting
`telemetrySampleRate`	head sampling: only this fraction of events is exported
`telemetryIngestRps`	rolling 1 s ingest ceiling; beyond it events are dropped
`telemetryDropped` counter	events lost; completes the export branch so user requests never fail
global `traceSampleRate`	fraction of requests traced in the output

[repo] Dropped telemetry never appears as a failed user request, which matches fire-and-forget exporters.

9. Testable expectations (summary)

- With sampling rate p, exported events = p x events within sampling error; dropped = events past the ingest ceiling.
- Telemetry overload never raises the application's error rate or latency (fire-and-forget).
- Burn rate = observed error rate / (1 - SLO); time to exhaustion = window / burn rate.
- A percentile computed from bucketed counts is within the bucket width of the true value.
- Probability of capturing a rare event at sampling rate p over k occurrences is 1 - (1 - p)^k.
- Health-check detection delay is interval x failures.

10. Open items

- Fetch and cite: the OpenTelemetry sampling specification; the Dapper paper's measured overhead and sampling rate; Prometheus native-histogram accuracy.
- Decisions `[?]`: whether the simulator should compute burn rate and SLO status directly in the output.

Sources

- Monitoring Distributed Systems, Google SRE book [P]

- Alerting on SLOs, Google SRE workbook [P]
- Histograms and summaries, Prometheus [P]
- Dapper, a large-scale distributed systems tracing infrastructure [P]
- OpenTelemetry sampling: head-based and tail-based (Uptrace) [S]
- Sampling strategies (OpenSearch) [S]

17. DevOps and delivery

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 7 `DevopsTypes`: `artifact-repository`, `build-system`, `feature-flag-service`, `deployment-controller`, `chaos-engineering-framework`, `policy-as-code`, `pipeline-secrets`. Fault injection mechanics are chapter 22; CI/CD runners as queues are chapter 14.

1. What delivery changes

Every release is a controlled change to a running system. The delivery strategy decides how much of the fleet sees the change at once (the **blast radius**), how fast a bad change is noticed, and how fast it is undone. [K]

Expected harm from a bad release = share of traffic exposed x request rate x time until rollback. [K]

2. Deployment strategies

Strategy	Mechanism	Extra capacity	Rollback	Risk
Recreate	stop all old, start all new	none	redeploy old	downtime during the gap
Rolling	replace instances gradually	surge pods	roll forward or back, gradually	mixed versions run together; capacity dips if unavailable > 0
Blue-green	a full second environment; the router switches all traffic at once	a **whole second environment**	switch the router back, instantly	everyone is exposed at the cut-over; transactions made meanwhile need handling
Canary	send a small share to the new version, then widen	a small number of instances	route the share back	needs good metrics to decide

- **Blue-green** (Fowler): two near-identical environments, one live and one for testing the next release; after testing the router sends all traffic to the new one and the old one idles; rollback is switching back. Separate schema changes from application releases so one database works with both versions. [S: martinfowler.com]
- **Canary** (Fowler): deploy to infrastructure with no traffic, route a small set of users, then add servers and users gradually until everyone is on the new version; roll back by rerouting. A router picks the users: random sample, employees first, or by geography. [S: martinfowler.com]
- **Rolling update** (Kubernetes Deployment): `maxSurge` and `maxUnavailable` default to **25%**; surge rounds **up** and unavailable rounds **down**. With 3 replicas: surge 1, unavailable 0, so there are always at least 3 ready and at most 4 running. [S, K: the Strategy section was not on the fetched text]

Rolling worked example: 10 replicas, surge 25% (3), unavailable 25% (2): capacity never falls below **8 of 10**; each wave replaces up to about 5 pods and waits for readiness, so rollout time is roughly waves x (start-up + readiness delay). [computed]

Blue-green cost: 2x the steady-state instances while both exist; hours of overlap multiply the instance bill by the overlap fraction. [K]

3. Canary arithmetic

- Canary at weight w of R requests/s receives w x R. At 5% of 1,000 req/s that is **50 req/s**. [computed]
- Bad requests until rollback = w x R x t. A canary that fails everything for 60 s at 5% and 1,000 req/s causes **3,000** failed requests, against 60,000 for an all-at-once release. [computed]
- **How long to detect a regression.** To tell an error rate p1 from p2 with about 80% power at 5% significance, each arm needs roughly n = 16 x p(1 - p) / delta^2 requests, with p the mean rate and delta the difference. Detecting 1% vs 2% (p = 1.5%, delta = 1%): n = 16 x 0.015 x 0.985 / 0.0001 = **about 2,400** canary requests, so **47 s** at 50 req/s. A latency regression needs more samples because latency is noisy. [K statistics, computed]
- A canary that is too small (w very low) or too short never collects enough data; one that is too large exposes too many. [K]
- Promotion needs automatic checks on the golden signals (chapter 16) compared with the baseline, not just "no crashes". [K]

4. Feature flags

A flag separates **deploy** from **release**: code ships dark and is turned on for a share of users. [K]

- **Evaluation** is local (an in-memory rule check, microseconds) when the SDK streams rules from the flag service; it is a **remote call per request** if the service is queried synchronously. [K]
- **Percentage rollouts** hash a stable user id into a bucket, so a user keeps the same experience as the percentage rises. [K]
- **Failure:** if the flag service is down, clients use cached rules or a hard-coded default; a default of "on" for a risky feature is a latent incident. [K]
- Flags make rollback seconds, not minutes, but accumulate as debt if not removed. [K]

5. Build, artifacts, policy and secrets

Component	Behaviour	What to model
Build system	runs jobs on runners	a queue; wall time shortened by parallel steps only up to the serial fraction (Amdahl, chapter 06)
Artifact repository	stores build outputs and images	download time in deploys; an outage blocks releases but not running traffic
Policy as code	checks changes against rules before apply or at admission	a latency cost at admission; a wrong rule blocks all deploys
Pipeline secrets	credentials the pipeline uses	leak risk; rotation (chapter 15)

6. Chaos engineering

Run experiments to find weaknesses before they find you. The method: define a **steady state** metric, hypothesise it holds under a fault, inject a realistic fault with a **limited blast radius** and an **abort condition**, then compare. Typical faults: kill an instance, add latency, drop dependency calls, fail a zone. [K: Principles of Chaos Engineering, not fetched]

The simulator's fault list is the same set (chapter 22), so a chaos experiment is a topology plus a fault and a pass/fail check on the steady-state metric.

7. Failure modes

- **Bad release at 100% exposure:** error rate jumps at once; recovery time equals rollback time.
- **Canary without comparison:** a regression that is small relative to noise is promoted.
- **Mixed versions:** during a rolling update both versions serve; incompatible schema or API changes break one of them.
- **Capacity dip:** `maxUnavailable` too high lowers capacity during a rollout and overloads the rest.
- **Stuck rollout:** new pods never become ready, so the rollout halts with a reduced fleet.

8. Simulator scope

The delivery components are mostly modelled as nodes on the path (latency, capacity, failure) and as scenarios using faults and instance counts. A rollout over time, percentage routing between two versions and an automatic promotion decision are expressed with routing weights and scheduled faults. [?] Owner to confirm what a "deployment controller" node should simulate by itself.

9. Testable expectations (summary)

- Failed requests during a bad canary = weight x rate x detection-to-rollback time.
- Blue-green needs double the instances while both are live.
- During a rolling update capacity is never below replicas - maxUnavailable.
- Detection time of a canary regression falls as canary weight rises (more samples per second).
- A flag evaluated remotely adds its latency to each request; a local flag does not.

10. Open items

- Fetch and cite: the Kubernetes Deployment strategy section; Argo Rollouts or Flagger analysis; Principles of Chaos Engineering; DORA metrics.
- Decisions `[?]`: what the deployment-controller and feature-flag nodes should do inside the engine beyond routing.

Sources

- BlueGreenDeployment, Martin Fowler [S]
- CanaryRelease, Martin Fowler [S]
- Deployments, Kubernetes [P]

18. Data infrastructure and AI

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 5 `DataInfraTypes`: `etl-pipeline`, `streaming-analytics`, `feature-store`, `memory-fabric`, `model-serving`; plus the AI-related `llm-gateway` (integration) and `agent-orchestrator`. Vector search indexes are chapter 09; queues and streams are chapter 13.

1. Batch versus streaming

	Batch (ETL)	Streaming
Unit of work	a bounded dataset	an unbounded flow of events
Freshness	period of the schedule (hourly, daily)	seconds
Cost shape	throughput: data size / processing rate	sustained rate x per-event cost, always on
Failure	re-run the job	resume from an offset; exactly-once needs care

Batch time = data size / throughput: 500 GB at 250 MB/s is **2,000 s (33 min)**; a daily job must finish before the next run starts or it queues behind itself. [computed] **Backfill** (reprocessing history) competes with the live pipeline for the same resources. **Idempotent** loads make re-runs safe. [K]

2. Windowing

Streaming aggregates group events into **windows**: [P: Apache Flink documentation]

- **Tumbling:** fixed size, no overlap; each event is in exactly one window.
- **Sliding:** fixed size, with a slide step; when the slide is smaller than the size windows overlap, so each event falls in size / slide windows (10-minute windows sliding every 5 minutes put each event in 2). [P example, K arithmetic]
- **Session:** windows that close after a gap with no events.
- **Event time vs processing time.** Event-time windows fire when the **watermark** passes the window end; by default **late** events (behind the watermark) are **dropped**, and **allowed lateness** (default 0) keeps the window open longer and can fire it again. [P]

Worked: 1,000 events/s into 5-minute tumbling windows = **300,000 events** per window, one result every 300 s. The same stream in 10-minute windows sliding every 5 minutes does about **2x the work and holds 2x the state**, and emits a result every 5 minutes. [computed]

Latency of a windowed result is at least the window length (plus watermark delay): a 60 s window cannot give a sub-minute answer. [K]

3. Feature stores

A feature store serves the same features for **training** and for **inference**: [S: Feast documentation]

- **Offline store** (warehouse or lake): historical features for training and batch scoring.

- **Online store:** a low-latency key-value store holding the latest feature values for real-time prediction.
- **Materialization** copies features from batch sources into the online store; freshness is limited by how often it runs, and **training/serving skew** appears when online and offline values differ. [S, K]
- **Point-in-time correctness:** a training row must only use feature values that were known at that row's timestamp, or the model leaks the future. [K]

4. Model serving

- **Latency** = queueing + batch wait + inference time + pre/post-processing. [K]
- **Dynamic batching** groups requests to use the accelerator efficiently: throughput per replica = batch size / batch time. Batch of 8 at 40 ms gives **200 req/s**; waiting up to 10 ms to fill a batch adds up to 10 ms to the first request. Bigger batches raise throughput and latency together. [computed]
- **GPU memory** limits batch size and the number of models resident; loading a model from storage is a cold start (seconds to minutes). [K]
- **Autoscaling** models is slow because replicas start slowly (chapter 14); keep warm capacity for bursts. [K]

5. Large-language-model serving and gateways

- **Latency of a response** = time to first token + (output tokens - 1) x time per token. 500 output tokens at 20 ms per token is about **10 s**; the first token (prefill) depends on the prompt length, the rest on the output length. A chatty prompt is slow regardless of request rate. [K]
- **Concurrency** (Little, chapter 01): 10 requests/s x 10 s = **100 concurrent streams**; capacity is bounded by accelerator memory for the key-value cache of each active sequence. [computed, K]
- **Cost** = input tokens x price_in + output tokens x price_out; output is usually priced higher. Rate limits are often in tokens per minute as well as requests per minute. [K]
- **LLM gateway:** a proxy in front of providers adding routing, caching, rate limiting, fallbacks and token accounting; it adds a hop and can cache identical prompts (chapter 10). [K]
- **Agents:** an orchestrator calls a model and tools in a loop. Sequential steps **add** their latencies (5 steps x 4 s = 20 s); parallel tool calls cost the **maximum** of the branches, which grows like H_n for n independent exponential-like calls (chapter 12). Each retry or extra step multiplies token cost. [K]
- **Semantic caching** trades hit ratio against wrong answers for near-duplicate prompts. [K]

6. Failure modes

- **Late data** silently dropped by a window: totals disagree with the batch result.
- **Backlog in the stream** when the processing rate falls below the arrival rate (chapter 13); lag grows without bound.
- **Training/serving skew:** online features computed differently from offline ones.
- **Cold model:** the first request after scale-out waits for the model to load.
- **Token blow-up:** long prompts or loops in an agent multiply latency and cost.
- **Provider rate limit:** requests fail with 429 at a tokens-per-minute ceiling.

7. Simulator parameter map

Setting	Meaning
`windowMs`	processing-time tumbling window length; one aggregate per window
`msPerToken`, `outputTokens`	latency added per request = msPerToken x outputTokens
model serving and stream nodes	queue and service time like any node; batching via the batching settings (chapter 13)

[repo] Windowing is processing-time only; event-time windows, watermarks and late-data handling are deliberately not modelled. Token cost latency does not distinguish prefill from decode.

8. Testable expectations (summary)

- One aggregate per window; count per window = rate x window length.
- Token latency = per-token time x output tokens (+ fixed cost); doubling output tokens doubles the generation part.
- Concurrency of streams = rate x response time.
- A sliding window's work and state grow by size / slide over a tumbling window of the same size.
- A batch job that takes longer than its period builds a backlog.

9. Open items

- Fetch and cite: Flink watermark and lateness pages in full; the Dataflow model paper; LLM serving papers (vLLM / PagedAttention) for the key-value cache and throughput figures; a model-serving batching reference.
- Decisions `[?]`: whether to model event-time windows and late data; whether to split first-token and per-token latency.

Sources

- Windows, Apache Flink [P]
- Feast: concepts overview [S]

19. Real-time and media

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 6 `RealTimeTypes`: `websockets-gateway`, `push-notification-service`, `transcoder`, `signaling-server`, `sfu-mcu`, `webrtc-mesh`. Related: the TLS and connection set-up round trips (chapter 08), fan-out through a topic (chapter 13), memory-bound capacity (chapter 06).

1. Long-lived connections

A request/response service holds a connection for milliseconds; a real-time gateway holds it for **minutes to days**. Capacity is therefore set by **how many connections it can hold**, not by requests per second. [K]

- **WebSocket** starts as an HTTP request with `Upgrade: websocket`; the server answers `101 Switching Protocols`. The upgrade is one request/response **after** TCP (and TLS for `wss`), so a new connection costs the TCP and TLS round trips of chapter 08 plus one more. [P: RFC 6455]
- **Frame overhead:** 2 to 14 bytes per message (2-byte base header, +2 or +8 for long payloads, +4 for the masking key); client-to-server frames are always masked, server-to-client never. [P]
- **Control frames:** ping (0x9), pong (0xA) and close (0x8), each at most 125 bytes. Either side may start the close handshake. [P]
- **Keep-alive:** idle connections are dropped by NATs and load balancers after tens of seconds to minutes, so clients and servers send periodic pings. [K]

2. Connection capacity arithmetic

Quantity	Formula	Worked
Memory-bound connections	RAM / memory per connection	16 GB / 32 KB = **524,288** per instance [computed]
Heartbeat load	connections / interval x cost per ping	100,000 / 30 s = 3,333 pings/s x 0.01 ms = 33 ms of CPU per second = **0.033 cores** [computed]
Reconnect storm	clients x handshake cost / time to reconnect	100,000 clients each needing a 2 ms TLS handshake = 200 s of CPU; across 16 cores **12.5 s** of full load [computed]
Concurrent connections	arrival rate x session length (Little)	50 new users/s x 600 s = **30,000** held [computed]

- A fleet that cannot hold the offered connections **refuses the excess**: the limit is the smaller of the RAM cap and any per-instance connection cap. [K]
- **Thundering herd on restart:** when a gateway restarts, every client reconnects at once; jittered exponential backoff on the client spreads the load (chapter 02). [K]
- **Sticky routing.** A connection lives on one instance, so a load balancer's per-request balancing does not balance a long-lived connection; load follows the connection count at connect time. [K]

3. Push and fan-out

- **Fan-out cost:** pushing one event to N held connections costs N x (send cost); at 0.01 ms per send, **1,000,000 recipients = 10 s of CPU**, or 0.6 s spread over 16 cores. A single popular event can saturate a gateway even if request rate is low. [computed]
- **Delivered load = events x recipients**, not events (compare chapter 13 topics). [K]
- **Mobile push** (APNs, FCM) is a call out to the platform provider: asynchronous, rate-limited, best effort, with no delivery-time guarantee. It is an external dependency (chapter 20). [K]

4. Media pipelines

- **Transcoding** turns one input into several renditions (a **bitrate ladder**, for example 1080p, 720p, 480p, 360p). It is CPU or GPU bound; capacity = cores / (CPU-seconds per second of video x renditions). Live transcoding must run at **real time or faster** (real-time factor <= 1) or the stream falls behind. [K]
- **Signalling server:** exchanges the small messages (offer, answer, ICE candidates) that set up a call; low rate, latency-sensitive, no media. [K]

5. Multi-party video: mesh, SFU, MCU

N participants, each sending one stream of bitrate b:

Topology	Upload per client	Download per client	Server	Scales to
Mesh (peer to peer)	(N - 1) b	(N - 1) b	none	about 4 to 5 people
SFU (selective forwarding)	b (or a simulcast set)	(N - 1) b	forwards, no decode: egress N (N - 1) b	tens of people
MCU (mixing)	b	b (one mixed stream)	decodes N, encodes mixed streams: heavy CPU	constrained clients, legacy

[S: vendor and community summaries; no primary measurement found] Worked at b = 1.5 Mbps: mesh with N = 5 uploads **6 Mbps** per client; an SFU with N = 5 sends 5 x 4 x 1.5 = **30 Mbps** out; with N = 50 and everyone visible that is 50 x 49 x 1.5 = **3,675 Mbps**, which is why large calls show only a few active speakers or use simulcast layers. [computed]

6. Failure modes

- **Connection exhaustion:** RAM or file-descriptor limit reached; new connections are refused while existing ones work.
- **Reconnect storm** after a gateway or network blip.
- **Slow consumer:** a client that cannot keep up forces the server to buffer or drop; per-connection send buffers must be bounded.
- **Head-of-line blocking** on one TCP connection delays all messages behind a lost packet.
- **Media quality collapse** when the uplink cannot carry the stream: the sender must lower bitrate.

7. Simulator parameter map

Setting	Meaning
`heldConnections`	connections offered to the gateway fleet

Setting	Meaning
`memPerConnectionKb` (32), `maxConnectionsPerInstance`	memory and cap per instance; the held count is the smaller of the offered, cap and RAM limits
`heartbeatIntervalMs`, `heartbeatCostMs` (0.01)	keep-alive pings; cores kept busy = held / interval x cost
`pushRecipients`, `pushSendMs` (0.01)	recipients per pushed event and CPU per send

[repo] The connected share (held / offered) scales delivered pushes; excess connections are refused.

8. Testable expectations (summary)

- Held connections = min(offered, per-instance cap x instances, RAM / memory per connection).
- Heartbeat cores = held / interval x cost; they reduce the capacity left for requests.
- One event to N recipients costs N x send time; delivered messages = events x recipients x connected share.
- Offered connections above the cap are refused, not queued.
- Mesh upload grows linearly in N - 1; SFU upload stays constant; SFU egress grows as N (N - 1).

9. Open items

- Fetch and cite: RFC 6455 ping/pong and keep-alive sections; a primary source for SFU/MCU scaling (the mediasoup or Jitsi architecture docs); APNs and FCM rate limits; a transcoding real-time-factor benchmark.
- Decisions `[?]`: whether `sfu-mcu` and `webrtc-mesh` should model per-participant bandwidth or only a processing cost.

Sources

- RFC 6455: The WebSocket Protocol [P]
- WebRTC network topology: mesh, SFU, MCU (Ant Media) [S, vendor]
- WebRTC multiparty video alternatives (BlogGeek.me) [S]
- SFU vs MCU vs mesh (Fora Soft) [S, vendor]

20. Integration

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 5 `IntegrationType`s: `webhook-gateway`, `llm-gateway`, `third-party-api-connector`, `payment-gateway`, `third-party-auth`. Related: retries and breakers (chapter 02), idempotency at consumers (chapter 13), LLM latency (chapter 18), authentication cost (chapter 15).

1. A dependency you do not control

A third-party call is a node whose **latency, availability, rate limit and failure modes belong to someone else**. You cannot add capacity to it, and its slowness becomes yours. [K]

- **Latency adds:** your response time includes theirs; your p99 is at least their p99 on the path. [K]
- **Availability multiplies:** dependencies in series must all be up. Five dependencies at 99.9% each give 0.999^5 = **99.50%**, not 99.9%. [computed]
- **Rate limits:** providers answer **429 Too Many Requests**, often with a `Retry-After` hint; the client must back off (chapter 02), queue, or shed. [K]
- **Timeouts and breakers:** set a timeout well below your own caller's deadline and open a circuit breaker on repeated failure (chapter 02) so a slow provider does not hold all your workers (Little: concurrency = rate x their latency). [K]
- **Fallbacks:** a cached answer, a second provider, or degraded behaviour. [K]

2. Payments and idempotency

A payment call that **times out** leaves the client not knowing whether the charge happened. Retrying without protection can charge twice. The fix is an **idempotency key**: [P: Stripe API docs]

- The client sends a unique key (a v4 UUID is suggested, up to **255 characters**) in the `Idempotency-Key` header with each `POST`; `GET` and `DELETE` are idempotent by definition and ignore it.
- The server saves the **status code and body** of the first request made with that key, once execution has begun, and returns the saved response to a retry, **including a 500**.
- Keys can be pruned after **24 hours**; a reused key after that makes a new request.
- If a retry's parameters differ from the original's, the server **returns an error** rather than guessing.
- Requests that failed validation or conflicted with a concurrent request are not saved, so they can be retried.

Duplicate-charge arithmetic: with a timeout rate t on charge calls where the server had in fact processed the request, the fraction of charges duplicated by a blind retry is t x (share processed before the timeout). With idempotency keys the duplicate count is **zero** for retries inside the key lifetime. [K, computed]

Reconciliation: when an outcome stays unknown, query the provider for the transaction by key before retrying, and run a periodic reconciliation job that matches your records to the provider's. [K]

3. Webhooks

A webhook is an **outbound HTTP call from a provider to you** when something happens. Stripe's behaviour is a representative reference: [P: Stripe webhooks documentation]

- **Success is a 2xx** returned quickly, **before** slow work; a 3xx redirect counts as failure; a timeout counts as failure.
- **Retries:** delivery is retried with **exponential backoff for up to three days** in live mode (a few retries over hours in test mode). Delivery is therefore **at least once**.
- **Duplicates happen:** keep the ids of processed events and skip repeats. **Order is not guaranteed:** events like `invoice.created` and `invoice.paid` can arrive in either order, so fetch current state instead of trusting arrival order.
- **Process asynchronously:** put the event on a queue and return 200; a burst (for example subscription renewals at the start of a month) can overwhelm a synchronous handler.
- **Verify the sender:** an HMAC-SHA256 signature over `timestamp.body` with a secret, and a timestamp **tolerance of 5 minutes** against replay.

Endpoint arithmetic: if a provider sends bursts of B events in a second and your handler takes s seconds with c workers, the backlog drains in B s / c seconds; without a queue the excess fails and is retried later, adding to the next burst. [computed]

4. Authentication with a third party (OAuth/OIDC)

- A sign-in redirects the user to the provider and back (several round trips and a token exchange), an interactive cost paid once per session. [K]
- Access tokens are short-lived; a **refresh token** gets a new one. Validating a token locally (signature) is cheap, introspecting it is a network call per request unless cached (chapter 15). [K]
- A provider outage blocks new sign-ins; existing valid tokens keep working until they expire. [K]

5. LLM gateways

A gateway in front of one or more model providers: routing, caching, rate-limit handling, fallback to another provider, token accounting. Latency and cost are per token (chapter 18); a provider's tokens-per-minute limit is a rate limit as above. [K]

6. Failure modes

- **Slow provider** (not down): workers pile up waiting; without a timeout and breaker the whole service stalls (the bimodal-latency case of chapter 22).
- **Rate limited:** 429s; retries make it worse (retry amplification).
- **Duplicate side effects:** retries or redelivered webhooks without idempotency.
- **Out-of-order events** applied as if in order.
- **Provider schema or version change** breaks parsing.
- **Signature or clock-skew failures** reject genuine webhooks.

7. Simulator parameter map

Setting	Meaning
`externalLatencyMs`	provider latency added to each call on `third-party-api-connector`, `payment-gateway`, `third-party-auth`, `webhook-gateway` and push services
`dedupKeyField`, `dedupWindowMs`, `storeLookupMs`	idempotency manager: which request field is the key, how long a key is remembered, the lookup cost
`commitOutcomeJournal`, `reconcileUnknownOutcomes`, `externalReconciliationMode`, `reconciliationProbeResult`	unknown-outcome handling after a timeout
`resilience.retry`, circuit breaker, timeout	client-side protection (chapter 02)

[repo] The idempotency manager records the decision for each request (first, duplicate in window, expired window).

8. Testable expectations (summary)

- A third-party node adds its configured latency to each call; end-to-end latency is the sum along the path.
- Availability of a series of dependencies is the product of their availabilities.
- A duplicate key inside the window is recognised and not executed again; a key after the window is a new request.
- A retry without a key on a timeout can execute the effect twice; with a key it cannot.
- A webhook endpoint that responds slower than the provider's timeout is retried; duplicates are possible, so the effect count must be guarded by an event id.
- A rate-limited provider returns 429 above its limit; naive retries raise the offered rate.

9. Open items

- Fetch and cite: RFC 6585 (429) and RFC 9110 (`Retry-After`); the IETF idempotency-key header draft; an OAuth 2.0 (RFC 6749) summary; one payment provider's timeout guidance.
- Decisions `[?]`: whether a provider's rate limit should be a setting on third-party nodes (the engine models its latency only).

Sources

- Idempotent requests, Stripe API [P]
- Receive Stripe events in your webhook endpoint [P]

21. DNS and certificates

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The 3 `DnsTypes`: `dns-authoritative-server`, `internal-dns`, `certificate-distro`. Related: global traffic management and geo latency (chapter 07), TLS handshake round trips (chapter 08), certificate authority and key management (chapter 15).

1. Name resolution

A client asks a **recursive resolver**, which walks root, top-level domain and the **authoritative** server for the zone, and caches the answer for its **TTL**. [K]

- A cold lookup takes about **3 round trips** (root, TLD, authoritative) plus the client's own hop to the resolver; a cached lookup costs only the hop to the resolver (or nothing, from the operating-system cache). [K]
- **TTL** is an unsigned 32-bit number of seconds, valid range **0 to 2^31 - 1**; it is a **maximum**, not a promise: "The TTL specifies a maximum time to live, not a mandatory time to live", so implementations may cap it. All records in one record set must carry the same TTL. [P: RFC 2181]
- **Negative answers** (name does not exist) are cached too, using the zone's SOA minimum. [K]

Cache miss ratio. For one resolver cache and requests for a name arriving at rate lambda (Poisson), each time the cached answer expires the next request misses and refills it for TTL seconds; a cycle holds on average 1 + lambda x TTL requests, so

miss ratio = 1 / (1 + lambda x TTL) [derived]

Worked: lambda = 1/s, TTL = 30 s: 1/31 = **3.2%**. A rarely requested name, lambda = 0.01/s, TTL = 30 s: 1/1.3 = **77%**. Expected resolution latency = miss ratio x cold-lookup time. [computed]

2. TTL and failover

DNS-based failover works only as fast as clients' caches expire. [K]

- After the authoritative answer changes, each cache keeps the old answer for up to TTL: the **worst case is TTL**, the mean over caches is about TTL / 2.
- Total time to move traffic after a failure = **detection time** (health-check interval x consecutive failures) + **TTL propagation**. A 10 s check with 3 failures and a 60 s TTL: 30 s to detect, then up to 60 s (mean 30 s) for caches to drop the old answer: up to **90 s** of traffic to a dead endpoint, **60 s** on average per client. [computed]
- Lowering the TTL ahead of a planned change shortens it; very low TTLs raise load on the authoritative servers: queries per second to the authoritative server is roughly (resolvers x names) / TTL. [K]
- Some resolvers and clients ignore low TTLs, so any failover design must tolerate a longer tail. [K]

3. Routing policies

Route 53, as a reference, offers these policies: **simple** (one resource), **failover** (active-passive on health), **geolocation** (by user location), **geoproximity** (by resource location with a bias), **latency** (the Region with the best latency), **IP-based** (by client address range), **multivalue answer** (up to eight healthy records chosen at random) and **weighted** (proportions you set). [P: Route 53 developer guide]

- **Weighted:** share of resolutions for a record = its weight / sum of weights. Because answers are cached, the share of **traffic** follows the share of **resolver cache fills**, so a few large resolvers can skew it. [K]
- **Failover:** serve the primary while its health check passes, else the secondary. [P]
- **Latency / geolocation:** choose the target by measured latency or by user location, an input of the client's resolver address unless EDNS client subnet is used. [K]

4. Certificates

- A certificate binds a name to a public key, signed by a certificate authority; clients check **name, validity period and chain** (and revocation). [K]
- **Lifetime:** Let's Encrypt certificates last **90 days**, renewed about every **60 days**; short-lived certificates last **6 days**, renewed every **3 days**. Lifetimes cannot be adjusted. [P: Let's Encrypt FAQ]
- **Renewal margin:** the window between the renewal point and expiry is the time automation has to recover from a failure: 30 days for the 90-day default, 3 days for the 6-day certificate. If renewal fails for longer than the margin, **every new TLS connection fails at expiry**. [computed]
- **Distribution:** a renewed certificate must reach every terminator (load balancers, CDN edges, gateways). Time to fully deploy = nodes / rollout parallelism x time per node; until then some nodes still serve the old one. [K]
- **Revocation:** CRLs and OCSP add a lookup unless the server staples the response. [K]

5. Failure modes

- **Stale cache after a change:** clients reach the old address until TTLs run out.
- **Authoritative server down:** uncached names fail to resolve; cached names keep working until their TTL ends (so outage impact lags by up to TTL).
- **Resolver overload or poisoning** (see chapter 15).
- **Expired certificate:** a hard, simultaneous outage for all new connections; monitoring expiry dates is a standard control.
- **Partial certificate rollout:** some endpoints serve the new certificate and some the old; clients that pin or compare see mismatches.

6. Simulator parameter map

Setting	Meaning
`dnsRoutingPolicy`	`simple`, `weighted`, `failover`, `latency-based` or `geolocation` on `internal-dns` and `global-traffic-manager`
`dnsCacheTtlSeconds` (30)	how long an answer is cached per host or request type
`dnsGeoTargets`	origin to target mapping for geolocation
edge `weight`	weight of a weighted answer; an unset or non-positive weight counts as 1
`regionLatencyMs`	extra latency by origin region on CDN, traffic manager and edge router

[repo] `certificate-distro` carries no behaviour of its own: no expiry, renewal or rollout is modelled.

7. Testable expectations (summary)

- Weighted resolution shares equal weight / sum of weights within sampling error; an unset weight counts as 1.
- A cached answer is reused for the TTL and re-resolved afterwards: resolutions per name over a run = run time / TTL.
- Cache miss ratio follows 1 / (1 + lambda x TTL).
- After a primary fails, failover moves traffic only once detection and TTL have elapsed.
- Geolocation sends each origin to its mapped target; `[?]` what an unmapped origin does needs a defined answer (a default record).
- A certificate past its expiry fails new connections; renewal margin = lifetime - renewal point.

8. Open items

- Fetch and cite: RFC 1034 and 1035 (resolution algorithm, negative caching); the Route 53 weighted and failover pages; the Let's Encrypt post on 90-day lifetimes; a measurement of resolvers ignoring TTLs.
- Decisions `[?]`: whether certificate expiry and rollout should be modelled (the node type exists but is inert).

Sources

- RFC 2181: Clarifications to the DNS specification [P]
- Choosing a routing policy, Amazon Route 53 [P]
- Let's Encrypt FAQ [P]

22. Failure semantics and chaos

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

How components fail, how failure spreads, and how to inject it. Builds on chapter 02 (timeouts, retries, breakers, load shedding), chapter 07 (health checks), chapter 11 (failover) and chapter 14 (node and machine loss).

1. How a component fails

Failure	What the caller sees	Latency shape
Fail-stop (clean crash, connection refused)	immediate error	fast: one round trip
Crash-restart	errors, then success after restart; in-flight requests lost	fast errors, then a cold start (caches empty)
Blackhole (packets vanish, host or path dead)	nothing; the client waits	**the full client timeout**
Hang (accepts, never answers)	connection opens, no response; backlog fills then overflows	timeout wall; capacity also lost as workers are held
Degraded (slow, partial)	success but slower, or some requests fail	service time multiplied for a fraction of requests
Gray failure	works for some observers, fails for others	looks healthy to the checker, unhealthy to the application

- A real server failure is usually **silence**: clients keep the connection open and burn their whole timeout, so latency during an outage **walls at the timeout** rather than dropping. A fast refusal models only a TCP reset or a failure hidden by a load balancer. [K, consistent with chapter 02]
- **Gray failure** (Huang et al., HotOS 2017) is defined by **differential observability**: the system's failure detectors report healthy while an application is experiencing failure. Subtle faults like severe slowness, random packet loss, flaky I/O and memory thrashing cause most major availability incidents in cloud systems, more than fail-stop failures. [S: abstract and summaries]
- **Crash-restart** loses in-flight state; whether requests held at the moment of failure are reset or resume at recovery depends on the system (a hung connection may resume, a restarted process cannot). [K]

2. How failure spreads

A **cascading failure** grows over time through positive feedback: a partial failure raises the chance that other parts fail. [P: Google SRE book, Addressing Cascading Failures]

- **Overload on the survivors.** Losing one of N equal servers moves load to the rest: survivor utilization = rho x N / (N - 1). Cluster B lost, cluster A goes from 1,000 to 1,200 QPS and its frontends start failing; a service healthy at 10,000 QPS that fails at 11,000 may need load cut to about 1,000 QPS to recover when only 10% of servers are healthy. [P, computed]

- **Retry amplification.** Retries at several layers multiply: four attempts per layer across three layers is 4^3 = **64** attempts on the database. [P] A backend limit of 10,000 QPS with a steady 10,100 QPS of demand gets pushed over by its own retries. [P]
- **Bimodal latency.** With 1,000 threads, if 5% of requests hang for a 100 s deadline, 5,000 threads would be needed; only about 19.6% of requests are served, roughly 80% see errors, though just 5% actually failed. [P]
- **Resource exhaustion:** CPU, memory, threads and file descriptors run out, so in-flight requests, missed deadlines and failed health checks feed on each other (a garbage-collection death spiral). [P]
- **Cold caches:** a restarted node or cluster comes back with empty caches and overloads its backends (chapter 10). [P]

Metastable failure (Bronson et al., HotOS 2021): a system is in a vulnerable state, a **trigger** causes temporary overload, and a **sustaining effect** (work amplification from a common-case optimisation such as retries or caching) keeps it failed **after the trigger is gone**. The sustaining loop, not the trigger, is the root cause; retries are the leading example. Practitioners have used other names: retry storm, death spiral, persistent congestion. [S: abstract and summaries]

Mitigations (SRE book): [P]

- Load test to failure; plan **N+2** capacity.
- **Load shedding:** reject early (HTTP 503 above an in-flight threshold); use LIFO or CoDel queueing so stale requests are dropped.
- Keep queues short relative to the thread pool (about 50% or less); a queue 10x the thread count can turn a 100 ms request into 1.1 s.
- **Retry discipline:** randomized exponential backoff with jitter, a per-request cap, a per-process retry budget, retry at one layer only.
- **Deadline propagation** and **fail fast** when a backend is known to be down.
- Separate **liveness** (restart me) from **readiness** (send me traffic) so health checks do not restart a service that is merely overloaded.
- Recovery: add capacity, pause batch work, drop traffic hard (to 1%) and ramp up, fix the trigger.

3. Correlated failure and blast radius

- **Fault domains:** a region, availability zone or subnet failing takes every component placed in it; a component stays down until **every** domain containing it has recovered. [repo]
- **Blast radius** is the share of users or capacity one fault can affect: cells, shards and zones shrink it. Losing one of 3 zones removes 1/3 of capacity and raises survivor load to 1.5x. [computed]
- **Shared dependencies** (a DNS server, a certificate, a database) turn independent components into one failure. [K]
- **Common-mode failure:** the same bad deployment or configuration reaches every replica (chapter 17). [K]

4. Chaos experiments

Define a steady-state metric, hypothesise that it holds under a realistic fault, inject the fault with a limited blast radius and an abort condition, compare (chapter 17). A useful experiment names the fault type, the target, the start, the duration and what recovery looks like. [K]

5. Simulator parameter map

Setting	Meaning
`node-failure` fault, `mode`	`reject` (instant `node_failed`), `blackhole` (client waits its timeout), `hang` (backlog fills to capacity then overflows), `degraded` (serving, with a service-time change)
`inFlightPolicy`	`reset` or `hang`: what happens to queued and in-service requests at failure
`recoveryPolicy`	`resume` or `reset`: what happens to held requests at recovery (hang only)
`degradation { fraction, serviceTimeMultiplier }`	share of requests affected and how much slower
`atMs`, `durationMs`, `duration: permanent / fixed`	when the fault starts and for how long
fault `targetId` = location	fails every component in a region, availability zone or subnet
`cache-flush` fault	empties a cache (chapter 10)
cluster `machineFailureAtMs`	scripted machine loss (chapter 14)

[repo] The default for chaos injection is a **silent dead server**: new arrivals are blackholed, in-flight requests hang, and recovery resets any still-held requests. The legacy path is an instant rejection.

6. Testable expectations (summary)

- `reject` returns fast (about one edge hop); `blackhole` and `hang` wall at the client timeout.
- `degraded` raises latency by the multiplier for the stated fraction of requests and loses none.
- Removing 1 of N equal servers raises survivor utilization to rho x N / (N - 1).
- A fault on an availability zone fails exactly the components placed in it, and a component in two domains stays down until both recover.
- Retries at k layers multiply load by the product of attempts; with retries off the same fault does not cause collapse.
- A recovered node starts with cold state (empty cache, no warm connections).
- Fault duration bounds the outage; a permanent fault never recovers.

7. Open items

- Fetch and cite: the full metastable-failure paper and the OSDI 2022 follow-up ("Metastable Failures in the Wild"); the gray-failure paper's own definition; AWS Builders' Library on avoiding fallback and on static stability.
- Decisions `[?]`: whether health checks should be able to see a gray failure (a check that passes while requests fail) and whether a metastable-failure scenario should be a built-in template.

Sources

- Addressing Cascading Failures, Google SRE book [P]
- Metastable Failures in Distributed Systems, HotOS 2021 (summary by Marc Brooker) [S]
- Metastable Failures in the Wild, OSDI 2022 [S]
- Gray Failure: The Achilles' Heel of Cloud-Scale Systems, Microsoft Research [S]

23. Capacity planning and cost

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

Sizing a system for load and failure, and pricing the result. Builds on chapter 01 (utilization and queueing, the QuickCart sizing), chapter 06 (instances and scaling limits), chapter 14 (autoscaling) and chapter 22 (survivor load).

1. Utilization targets and headroom

Latency rises sharply as utilization approaches 1 (chapter 01), so a system is sized to run **well below** full: typical targets are 60% to 80% busy at the planning peak, lower for latency-sensitive tiers. [K]

- Servers needed = ceil(peak rate / (per-server rate x target utilization)). QuickCart: 1,000,000 req/s at 100,000 per server and a 0.8 target needs **13** servers (12 give 83.3%, 13 give 76.9%). [computed, chapter 01]
- **Headroom** is the unused share, 1 - utilization; it is the price of absorbing bursts, failures and forecast error. [K]

2. N+1 and N+2

Sizing so the fleet still meets its target after losing k servers:

normal utilization <= target x (N - k) / N

Worked: 12 servers, target 80% after one failure: normal utilization <= 0.8 x 11/12 = **73.3%**; after losing it, the 11 survivors run at 80%. For k = 2 on 12 servers: 0.8 x 10/12 = **66.7%**. Losing one of 3 zones: survivors run at 1.5x, so a 3-zone fleet should run at no more than 2/3 of the target to survive a zone loss. [computed] The Google SRE book recommends testing to failure and planning **N+2** where an update or a failure can overlap. [P: SRE book, Addressing Cascading Failures]

3. Instance sizing and cost per request

- **Capacity of one instance** = servers x (1 / service time) (workers per vCPU and perf factor from chapter 06), times the utilization target.
- **Cost per request** = price per hour / (requests per hour per instance).

Worked with the simulator's catalogue prices: an `m5.large` (2 vCPU, $0.096/h) running 100 ms requests serves 2 / 0.1 s = 20 req/s at full load, **14 req/s at 70%**: 50,400 requests per hour, so **$0.096 / 50,400 = $1.9 per million requests**. [computed from the catalogue]

- Within one instance family price grows about linearly with size (`m5.xlarge` is $0.192 for 4 vCPU), so larger instances do not make a request cheaper; they cut the number of nodes (and per-node overhead) and raise the blast radius of one failure. [K, catalogue]
- **Scaling limits** (Amdahl and USL, chapter 06) mean that past some size extra vCPUs add little; the cheapest design may use more, smaller instances. [K]

4. Pricing models

Model	Real discount vs on-demand	Commitment and risk	Simulator multiplier
On-demand	0	none	1.0
Reserved / savings plan	up to about 66% to 72% for a one or three year term	pay whether used or not	0.6
Spot	up to about 90%	the provider can reclaim the instance with about a two-minute warning	0.3

[S: third-party summaries of AWS pricing; the AWS pricing pages themselves were not fetched, so treat the percentages as indicative]

- **Reserved break-even:** you pay the reserved rate for every hour, so it beats on-demand when the instance would otherwise run more than (reserved multiplier) of the hours: at a 0.6 multiplier, above **60% utilization of the hours**. [computed]
- **Spot** suits stateless, fault-tolerant and batch work; the effective cost is price x (1 + overhead of interruptions and retried work). A workload that loses 10% of its work to interruptions at 0.3 pays 0.33 of on-demand. [computed]
- **Autoscaling** saves the idle share of a peaky load: a service that needs 10 instances for 4 hours a day and 3 the rest averages (10 x 4 + 3 x 20) / 24 = **4.2** instances, not 10. [computed]

5. Forecasting and when to buy more

With demand growing by a fraction g per period and current utilization u, the time until a target utilization u* is reached is

t = ln(u* / u) / ln(1 + g) [derived]

Worked: 50% utilization today, 10% monthly growth, 80% target: ln(1.6) / ln(1.1) = **4.9 months**; to reach 100%: ln(2) / ln(1.1) = **7.3 months**. Capacity must be ordered before this date, allowing for provisioning time (chapter 14: minutes for a node, weeks for hardware). [computed]

- **Load tests** find the real per-server limit; planning from a spreadsheet of CPU alone misses memory, connections, locks and downstream limits. [K]
- Plan for the **peak**, not the mean: the diurnal ratio of peak to mean (chapter 05) sets the idle share of a statically sized fleet. [K]

6. Where the money goes

Line	Driver
Compute	instance hours x price x pricing multiplier
Storage	GB-months, requests, IOPS provisioned (chapter 09)
Network	egress GB, cross-zone transfer, load balancer hours
Managed services	per request or per hour (consumption pricing)
Observability	log and trace volume (chapter 16: 5 MB/s of logs is 432 GB/day)

Cost is a function of the design, so it can be a constraint in a question: "meet the SLO under $X per hour". [K]

7. Simulator parameter map

Setting	Meaning
`resources.instanceType`, `instanceCount`	what is bought
`resources.pricingModel`	`on-demand` (1.0), `reserved` (0.6), `spot` (0.3)
total cost	sum of price per hour x multiplier x instance count over priced nodes, USD/hour
volume nodes	an estimate from throughput and bytes until a run supplies measured data
budget	a cap on cost used for grading

[repo] A node with no instance model is unpriced and contributes $0. The simulator's reserved and spot multipliers (0.6 and 0.3, "about 40% and 70% off") are more conservative than the real maximum discounts.

8. Testable expectations (summary)

- Total cost = sum over nodes of price x multiplier x count; doubling the count doubles the cost; reserved = 0.6x and spot = 0.3x of on-demand.
- Servers needed follows ceil(peak / (capacity x target)); after losing k of N servers the survivors' utilization is rho x N / (N - k).
- A fleet sized only for the mean load fails the peak; a fleet sized with N+1 headroom survives one failure at or under the target.
- Cost per request falls as utilization rises (until latency degrades) and is the same across sizes in a family at equal utilization.
- Scaling out raises cost linearly while capacity grows sub-linearly past the USL peak.

9. Open items

- Fetch and cite: AWS EC2 pricing pages for reserved, savings plans and spot; the AWS Well-Architected cost optimisation pillar; Google SRE workbook chapter on capacity.
- Decisions `[?]`: whether the pricing multipliers should follow real discounts; whether to model egress and cross-zone transfer cost.

Sources

- Addressing Cascading Failures, Google SRE book [P]
- Instance prices: the simulator's own catalogue (`src/engine/catalog/instanceCatalog.ts`) [repo]
- Spot and reserved discount summaries: third-party AWS pricing guides, listed in the search results for this chapter [S, to be replaced by AWS pages]

24. Architecture and system-design patterns

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

The patterns a design is assembled from, what each costs, and what the simulator can show about it. Components are in chapters 06 to 22; the trade-off tables for choosing between patterns are chapter 25.

1. Structure

Pattern	Idea	Gains	Costs
Layered	presentation, application, data in tiers; each calls the one below	clear separation	every request crosses every tier; a slow lower tier slows all
Monolith	one deployable	simple to run, no network hops between modules, local transactions	scales as one unit; one bad release affects everything
Microservices	many small independently deployed services	independent scaling and release; fault isolation	network hops, partial failure, distributed transactions, operational load
Event-driven	components communicate through events on a broker (chapter 13)	decoupling in time, easy fan-out, buffering of bursts	eventual consistency, harder tracing, ordering and duplicate handling
Sidecar / service mesh	cross-cutting concerns (TLS, retries, telemetry) in a proxy beside each service	uniform policy	an extra hop and CPU per call (chapter 07)
Cell-based	many isolated full copies of the workload, a thin router in front	failure and bad releases stay inside one cell	duplicated capacity, cross-cell operations are hard

Microservice arithmetic [K, computed]: each synchronous hop adds its latency and its failure probability. A request crossing 6 services at 99.9% each is available 0.999^6 = **99.4%**, and its latency is the **sum** along the path. Splitting a monolith therefore costs availability and latency unless the calls are cached, asynchronous or parallel.

Cell-based architecture: the workload is partitioned by a key that matches the grain of the service (customer id, resource id); a cell router sends each request to its cell; each cell is independent and shares no state. With 10 cells and 100 requests, a failure in one cell leaves 90% of requests unaffected; it does not necessarily need more hosts (30 hosts can become 3 cells of 10). [P: AWS Well-Architected, cell-based architecture]

2. Data and consistency patterns

- **CQRS** (Command Query Responsibility Segregation): separate models for updating and for reading, possibly separate databases. It helps when reads and writes have very different shapes or scale, and Fowler advises caution: it adds complexity, usually brings eventual consistency, and fits a minority of domains; apply it to specific bounded contexts, not the whole system. [S: martinfowler.com]

- **Saga:** a business transaction across services, each with its own database, as a sequence of local transactions. **Choreography:** each service's event triggers the next. **Orchestration:** a central orchestrator tells each service what to do. If a step fails, **compensating transactions** undo the earlier steps; there is no automatic rollback and concurrent sagas can see intermediate data (lack of isolation). [S: microservices.io]
- **Transactional outbox:** to avoid the **dual-write problem** (update the database and publish a message atomically), write the message to an outbox table in the same database transaction; a relay publishes it by polling or tailing the log. Delivery is **at least once**, so consumers must be idempotent (chapter 20). [S: microservices.io]
- **Event sourcing:** the log of events is the record; state is a fold over it; replay cost is events since the last snapshot x cost per event (chapter 13). [K]
- **Read replicas, caching, sharding:** chapters 10 to 12.

3. Traffic and scaling patterns

Pattern	Mechanism	Arithmetic
Queue-based load leveling	a queue absorbs bursts so the consumer runs at a steady rate	backlog grows at (arrival - drain); drain time = backlog / (drain - base) (chapter 05)
Fan-out on write	when a user posts, copy it into each follower's feed	writes per second = posts/s x followers; a celebrity with 1,000,000 followers posting once per second causes **1,000,000 feed writes/s**
Fan-out on read	build the feed at read time from the accounts followed	reads per timeline = accounts followed; cost moves from write to read
Hybrid	fan out on write for ordinary users, on read for celebrities	bounds both costs
Bulkhead, circuit breaker, backpressure, load shedding	chapter 02	
CDN and edge caching	serve static and cacheable content near the user	origin share = 1 - hit ratio (chapter 07)
Multi-region active-passive	one region serves, another standby	failover time = detection + promotion; data loss = replication lag (RPO)
Multi-region active-active	both serve	needs conflict handling (chapter 11) and routes users by geography/latency (chapter 21)
Strangler fig	build new functionality beside the legacy system and move behaviour over piece by piece through seams, removing the transitional code at the end	risk spread over many small steps [S: martinfowler.com]

[repo] The simulator expresses fan-out on write directly: an edge's `fanoutFactor` turns one delivered request into that many deliveries, so the downstream genuinely receives N times the load and can saturate.

4. Back-of-envelope sizing (the standard interview skeleton)

1. **Requests per second** = daily active users x actions per user per day / 86,400; **peak** = average x a peak factor (2 to 3, or the diurnal ratio of chapter 05).
2. **Read/write ratio** splits it: reads = rate x R / (R + W).

3. **Storage** = records x bytes x retention x replication factor.
4. **Bandwidth** = rate x bytes per response.
5. **Servers** = peak / (per-server rate x target utilization) (chapter 23), plus N+1.

Worked, URL shortener [computed]: 100 million new links per month is 100e6 / 2.592e6 s = **39 writes/s**; at a 10:1 read ratio, **390 reads/s**; five years of links is 6 billion records x 500 bytes = **3 TB** (x3 replication = 9 TB). The reads are served mostly from cache (a Zipf popularity, chapter 05), so the database sees a fraction of the 390/s.

Other standard designs and what they test: **news feed** (fan-out, caching, hot keys), **chat** (long-lived connections, ordering, fan-out; chapters 19 and 13), **rate limiter** (token bucket, distributed counters; chapter 02), **payments** (idempotency, saga, reconciliation; chapter 20), **ticket or seat booking** (contention and reservations; chapter 11), **search** (inverted index, scatter-gather; chapter 12), **video streaming** (transcoding, CDN; chapter 19). [K]

5. Anti-patterns

- **Distributed monolith:** services that must be deployed and scaled together, with synchronous chains. [K]
- **Shared database between services:** couples schemas and removes isolation. [K]
- **Chatty calls:** N+1 calls in a loop (latency = N x hop). [K]
- **Unbounded queues and retries without budgets** (chapter 22). [K]
- **Single point of failure:** a component whose loss disconnects the system. [repo: the output lists `singlePointsOfFailure`]
- **Kitchen-sink design:** every component added regardless of the requirement; it costs money and failure modes. [K, repo: budget and anti-pattern checks]

6. Testable expectations (summary)

- End-to-end latency of a synchronous chain is the sum of its hops; availability is the product.
- A cell design confines a fault to its share of traffic (1 / number of cells).
- Fan-out on write multiplies downstream load by the follower count; the write path saturates before the read path.
- A queue in front of a slow consumer protects the producer's latency and builds backlog at (arrival - drain).
- A saga needs compensation: a failed late step leaves earlier steps applied unless compensated.
- The outbox yields at-least-once publication: duplicates are possible, losses are not.
- The sizing arithmetic above reproduces the stated worked numbers.

7. Open items

- Fetch and cite: the Microsoft Azure architecture centre pattern catalogue; Twitter's timeline architecture write-ups for fan-out; the Strangler Fig page's facade discussion; Google's SRE chapter on multi-region.
- Decisions `[?]`: whether the simulator should offer saga and outbox as first-class components or leave them as compositions of queue, database and idempotency nodes.

Sources

- Saga pattern, microservices.io [S]
- Transactional outbox, microservices.io [S]
- CQRS, Martin Fowler [S]
- Strangler Fig Application, Martin Fowler [S]
- What is a cell-based architecture?, AWS Well-Architected [P]

25. Architecture decisions

Marks: **[P]** primary, **[S]** secondary (fetched or searched), **[K]** established knowledge or derived here, not yet source-checked, **[?]** needs an owner decision. See README.

How a design choice is recorded and argued, and the trade-off tables behind the choices a system-design question asks for. Each table row points to the chapter holding the numbers.

1. Recording a decision (ADR)

Michael Nygard's architecture decision record has five parts: [S: Cognitect blog, Documenting Architecture Decisions]

Part	Content
Title	a short noun phrase ("ADR 9: LDAP for multitenant integration")
Status	proposed while stakeholders review, accepted once agreed, later deprecated or superseded with a pointer to the replacement
Context	the technological, political, social and project-local forces at play, stated as facts without judgment, showing where they conflict
Decision	the response, in full active sentences beginning "We will"
Consequences	every result, positive, negative and neutral, since all of them shape what comes next

Practice: one or two pages, numbered sequentially and never reused, never deleted when reversed (marked superseded instead), kept in the repository beside the code. [S]

A good ADR in a system-design answer names the **requirement** that forces the choice (a number: latency, rate, availability), the **options** considered, the **deciding trade-off** and the **cost accepted**. [K]

2. How to decide: from requirements to numbers

1. State the non-functional targets as numbers: p99 latency, availability, throughput, durability, cost per hour (chapter 29: NFR targets).
2. Estimate load (chapter 24 section 4) and the **dominant bottleneck** (compute, storage, network, resilience).
3. For each candidate, compute the figure the target is about, using the formulas of chapters 01 to 23.
4. Choose the cheapest option that meets every target with headroom (chapter 23), and write down what would change the decision (a different read/write ratio, a stricter SLO).
5. Check the choice by simulating it against the same targets (chapter 29).

3. Trade-off tables

SQL or NoSQL (chapters 09, 11, 12)

Question	Prefer relational	Prefer NoSQL
Need joins, constraints, multi-row transactions	yes	usually no
Access pattern	varied, ad hoc queries	known key-based access

Question	Prefer relational	Prefer NoSQL
Write scale	one primary (vertical, then shard by hand)	horizontal partitioning built in
Consistency	strong by default	tunable, often eventual
Schema	enforced, migrations	flexible, application-enforced

Synchronous or asynchronous (chapters 02, 13)

	Synchronous	Asynchronous (queue/stream)
Caller waits for the result	yes	no, gets an acknowledgement
Burst handling	overload propagates	buffered, backlog grows
Failure	caller sees it	retried, dead-lettered, needs idempotency
Latency	sum of path	producer fast, end-to-end longer
Use when	the answer is needed now	work can complete later, bursts, fan-out

Push or pull (chapters 13, 19): push gives low latency and needs held connections or fan-out writes; pull is simple and stateless but wastes requests polling (a 5 s poll by 1,000,000 clients is 200,000 requests/s whether or not anything changed). [K, computed]

Where to cache (chapter 10)

Layer	Hit saves	Limit
Client / browser	the whole round trip	stale until expiry; per-user
CDN / edge	origin load and distance	cacheable content only (chapter 07)
Application (in-process)	a network hop to the cache	per-instance copies, cold on restart
Shared cache (Redis)	database load	a network hop, an extra dependency, stampedes
Database buffer	disk reads	fixed by memory

Consistency level (chapter 11): strong or linearizable costs read latency (wait for the replica) and availability under partition; eventual is fast and available but returns stale reads with probability 1 - e^(-w x lag); read-your-writes sits between and is often enough for user-facing data. [K]

Monolith or microservices (chapter 24): start monolithic unless independent scaling, team autonomy or fault isolation is a stated requirement; each split adds a hop, a failure mode and an operational cost. [K]

Queue or stream (chapter 13): a queue deletes on acknowledgement and shares work among consumers; a stream keeps a replayable log and lets several groups read independently. [K]

Vertical or horizontal scaling (chapter 06): vertical is simple up to the largest instance and the USL peak; horizontal needs stateless services or partitioned state and a balancer. [K]

Build, buy or managed service (chapter 23): managed removes operations at a per-request price; self-run is cheaper at scale and costs engineers. [K]

4. Requirements that force a choice

Requirement	Forces
p99 below a few ms for reads	cache or in-memory store in front of the database

Requirement	Forces
Write throughput above one machine	partitioning (shard key choice, chapter 12) or an LSM store
Availability above 99.99%	redundancy across zones, no single point of failure, tested failover
Never lose an acknowledged write	synchronous or quorum replication, write-ahead log, idempotent retries
Global users	multi-region, CDN, latency-based routing, conflict handling
Spiky traffic	queue-based load leveling, autoscaling with headroom, load shedding
Tight budget	right-sizing, reserved capacity, spot for tolerant work (chapter 23)

5. Failure modes of decision-making

- Choosing by familiarity instead of by the numbers; **resume-driven** design.
- Optimising a component that is not the bottleneck (find it first: chapter 27's latency decomposition and failure locus).
- Ignoring the consequences column: the cost accepted is never written down, then forgotten.
- Over-design: every component added regardless of the requirement (a budget and a node-count constraint exist to prevent it).

6. Testable expectations (summary)

- A decision record contains a requirement, the options, the deciding figure and the accepted cost.
- Each table row's claim can be reproduced with the cited chapter's formula.
- For a stated requirement, the chosen design meets the numbers in a simulated run and a plainly worse alternative does not.

7. Open items

- Fetch and cite: adr.github.io templates (MADR), the AWS Well-Architected trade-off guidance, the CAP/PACELC papers' consequences for choices.
- Decisions `[?]`: whether grading should accept a written justification (the question package has a `justify` field; see chapter 29) as evidence of a decision.

Sources

- Documenting Architecture Decisions, Michael Nygard [S]

26. Configuration, JSON and types

Marks: **[P]** primary, **[S]** secondary, **[K]** established knowledge, **[?]** needs an owner decision, **[repo]** read from the type definitions and validator (`src/engine/core/types.ts`, `src/engine/validation/validator.ts`). See README.

This chapter is the field reference for the document the engine runs: `TopologyJSON`. Unlike chapters 01 to 25 it describes the simulator's own contract, so each row gives the type, unit, range and what the field stands for in a real system, with the chapter that explains the real-world side. Fields the engine does not read are listed in section 9.

1. The document

```
{
  "id": "…",
  "name": "…",
  "version": "1.0.0",
  "global": {
    "simulationDuration": 60000,
    "seed": "demo",
    "warmupDuration": 0,
    "timeResolution": "microsecond",
    "defaultTimeout": 5000,
    "traceSampleRate": 0.01
  },
  "nodes": [
    {
      "id": "api",
      "type": "microservice",
      "category": "compute",
      "label": "API",
      "position": { "x": 0, "y": 0 },
      "resources": { "instanceType": "m5.large", "instanceCount": 2, "workloadKind": "cpu-
bound" },
      "processing": { "distribution": { "type": "constant", "value": 5 }, "timeout": 1000
},
      "config": {}
    }
  ],
  "edges": [
    {
      "id": "client->api",
      "source": "client",
      "target": "api",
      "mode": "synchronous",
      "protocol": "grpc",
```

```
      "latency": { "distribution": { "type": "constant", "value": 1 }, "pathType": "same-
dc" },
      "bandwidth": 1000,
      "maxConcurrentRequests": 100000,
      "packetLossRate": 0,
      "errorRate": 0
    }
  ],
  "workload": {
    "sourceNodeId": "client",
    "pattern": "poisson",
    "baseRps": 100,
    "requestDistribution": []
  },
  "faults": [],
  "invariants": [],
  "locations": []
}
```

Top level: `id`, `name`, `version` (strings), `global`, `nodes`, `edges`; optional `workload`, `faults`, `invariants`, `scenarios`, `locations`, `networkModel`. [repo]

2. `global` (run settings)

Field	Type, unit	Range	Meaning	Real counterpart
`simulationDuration`	ms	> 0	simulated time to run	length of the test
`seed`	string	any	seeds every random draw; same seed, same run	reproducibility (chapter 04)
`warmupDuration`	ms	>= 0	metrics collected only after it	discarding the transient (chapter 04)
`timeResolution`	`microsecond` or `millisecond`		clock granularity	
`defaultTimeout`	ms	> 0	request timeout when a node does not set one	client deadline (chapter 02)
`traceSampleRate`	fraction	0 to 1, default 0.01	share of requests traced	trace sampling (chapter 16)

3. `nodes[]` (components)

Field	Type	Meaning
`id`, `label`	string	unique id, display name
`type`	one of the 131 component types (chapters 06 to 22)	decides the behaviour and default profile
`category`	one of the 14 categories	grouping; any of the 14 is accepted for any type, it is not checked against the type

Field	Type	Meaning
`role`	`source`, `processor`, `storage`, `router`, `sink`	structural role (optional)
`position`	{x, y}	canvas position, no effect on a run
`provider`	string	vendor label, no effect on behaviour
`placement`	{regionId, availabilityZoneId, subnetId}	where it lives, for fault domains (chapter 22)
`resources`	see 3.1	the instance model
`queue`	{workers, capacity, discipline, weights}	a raw queue run as written (3.2)
`processing`	{distribution, timeout}	service-time distribution and node timeout in ms
`resilience`	see 3.3	breaker, retry, rate limiter, bulkhead
`slo`	{latencyP99 ms, availabilityTarget, errorBudget}	targets checked against the run
`config`	free map	the per-type settings of chapters 06 to 22 (cache hit rate, shard key, lag, window, ...)

3.1 `resources`

Field	Meaning
`instanceType`	a catalogue SKU (`m5.large`, `c5.xlarge`, `r5.2xlarge`, ...): vCPU, RAM, price per hour and a perf factor come from the catalogue, never free-typed
`instanceCount`	horizontal scale (default 1); `maxInstances` caps it at build time
`workloadKind`	`cpu-bound` or `io-bound`: decides workers per vCPU (chapter 06)
`perRequestMemMb`	memory held by one in-flight request: divides RAM into the admission ceiling K
`pricingModel`	`on-demand` (1.0), `reserved` (0.6), `spot` (0.3): cost multiplier only (chapter 23)

Workers and queue depth are **derived** from instance and workload kind; they are not authored. [repo]

3.2 `queue`

`workers` (c), `capacity` (K, total in system), `discipline` (`fifo`, `lifo`, `priority`, `wfq` with per-type `weights`). An authored `queue` with no `resources` runs as written; with neither, the node gets the palette profile for its type (chapter 06). [repo]

3.3 `resilience`

Block	Fields
`circuitBreaker`	`failureThreshold` (0 to 1), `failureCount` (min requests), `recoveryTimeout` (ms), `halfOpenRequests`
`retry`	`maxAttempts`, `baseDelay` (ms), `maxDelay` (ms), `multiplier`, `jitter`
`rateLimiter`	`maxTokens`, `refillRate` (tokens/s)

Block	Fields
`bulkhead`	`partitions` (per-compartment caps), `defaultMaxConcurrent`, `keyField`; legacy `maxConcurrent` for serverless

Meaning of each: chapter 02. A node's retry policy governs calls **to** that node. [repo]

3.4 Distributions

`constant`, `deterministic` {value}, `log-normal` {mu, sigma>0}, `exponential` {lambda>0, per ms}, `normal` {mean, stdDev>0}, `uniform` {min, max}, `weibull` {shape, scale}, `poisson` {lambda}, `binomial` {n, p}, `gamma` {shape, scale}, `beta` {alpha, beta}, `pareto` {scale, shape}, `empirical` {samples, interpolation}, `mixture`. Units are milliseconds for latencies. Definitions, moments and when each is realistic: chapter 03.

4. `edges[]` (connections)

Field	Type, unit	Meaning
`id`, `source`, `target`	string	endpoints
`mode`	`synchronous`, `asynchronous`, `streaming`, `conditional`	how the call is made (chapters 08, 13)
`condition`	JS expression	for `conditional` edges, e.g. `request.type === "write"`
`protocol`	`https`, `grpc`, `tcp`, `udp`, `websocket`, `amqp`, `kafka`	per-request protocol cost (chapter 08)
`latency`	{distribution, pathType, derivedFromPathType}	propagation; path types `same-rack`, `same-dc`, `cross-zone`, `cross-region`, `internet`
`bandwidth`	Mbps	link rate: transmission time = size / bandwidth
`maxConcurrentRequests`	count	connection-pool style cap
`packetLossRate`, `errorRate`	0 to 1	per-request loss (retransmission cost) and error probability
`weight`	number	relative weight for weighted routing (chapters 07, 21)
`fanoutFactor`	count	one delivery becomes N (fan-out on write, chapter 24)
`connection`	{reuse, tls, ...}	TLS and connection set-up (chapter 08)
`batching`	{...}	Kafka-style producer batching (chapter 08)
`protocolOverheadMs`	ms	replaces the protocol's default per-request cost

Edge physics apply only in the network edge model; the default connector model ignores latency, errors and loss (the importer says so). [repo]

5. `workload`

Field	Meaning
`sourceNodeId`	the node that generates requests

Field	Meaning
`pattern`	`constant`, `poisson`, `bursty`, `diurnal`, `spike`, `sawtooth`, `replay` (chapter 05)
`baseRps`	requests per second, > 0
`spike` {spikeTime ms, spikeRps, spikeDuration ms}; `diurnal` {peakMultiplier, hourlyMultipliers[24]}	shape parameters
`requestDistribution[]`	request types: `type`, `weight` (sum to 1), `sizeBytes`, `metadata`, `keyspace` {field, size, skew}
`sessions` {count}	stamps a session id for read-your-writes and sticky routing
`origins[]`	weighted client populations by location
`stopCondition`	{mode `duration` or `requestBudget`, `maxRequests`, `haltOnSaturation` {utilization, errorRate}}

6. `faults[]`, `invariants[]`, `locations[]`

Block	Fields
fault	`targetId` (node or location id), `faultType` (`node-failure`, `chaos`, `cache-flush`), `timing`, `duration` (`fixed`, `until`, `permanent`), `params` {atMs, durationMs, mode, inFlightPolicy, recoveryPolicy, degradation} (chapter 22)
invariant	`id`, `description`, `condition` like `perNode.maxUtilization <= 0.8`
location	`id`, `kind` (`region`, `availability-zone`, `subnet`, `edge-pop`), `label`, `parentId`

7. Request metadata and keys

Requests carry `metadata`: `sessionId`, the keyspace field, `clientIp`, `origin`, `host`, plus per-type static metadata. Traits read named fields: `shardKey`, `partitionKey`, `idempotencyKey`, `authToken` / `authStatus` / `authScopes`. [repo]

8. Question package (for reference)

`QuestionPackage` wraps a scaffold topology, constraints, a suite of cases (overrides of `global`, `workload`, `faults`, `invariants`), structural rules, a rubric, budget and optional classroom settings; see chapter 29.

9. Fields declared but not read

Field	Status
`failureModes` (mtbf, mttr, trigger)	validated, but nothing in the engine reads it: random failures at a mean time between failures are not generated; use `faults`
`scaling` (horizontal/vertical, thresholds, cooldown)	not read; the autoscaler reads `config.autoscale*` (chapter 14)
`resources.workersPerInstance`, `queueSlots`	accepted for compatibility, **not read**; the validator warns
`resources.replicas`, `cpu`, `memory`, `maxReplicas`	deprecated aliases (`replicas` is read as `instanceCount`)
`dependencies` (critical, optional)	used by grading rules, not by the run

Field	Status
`presentation`, `position`, `provider`	authoring only

10. Testable expectations (summary)

- A document that violates a range in this chapter is rejected with the field named; an unknown component type or an unknown category value is rejected.
- The same document and seed produce the same output.
- A field marked "not read" has no effect on any output.
- Weights, probabilities and fractions outside their ranges are rejected.

11. Open items

- Decisions `[?]`: whether the unread fields should be removed, implemented (`failureModes` as random failure injection), or flagged at import.
- Cite: the JSON schema generated from the validator, once exported, as the machine-readable version of this chapter.

27. Metrics and output glossary

Marks: **[P]** primary, **[S]** secondary, **[K]** established knowledge, **[?]** needs an owner decision, **[repo]** read from the output types (`src/engine/metrics.ts`, `src/engine/analysis/output.ts`, `src/engine/analysis/verdict.ts`). See README.

Every number a run reports, its exact definition and unit, and the real-world quantity it stands for. The mathematics behind them is chapter 01 (queueing) and chapter 16 (observability). Section 8 lists the known differences between a metric's name and what it computes, found while building the flows in `e2e/`.

1. Conventions

- Times are **milliseconds** unless a field says otherwise; rates are per second; fractions are 0 to 1.
- **Post-warmup:** metrics keyed `postWarmup*` count only what happened after `global.warmupDuration` (chapter 04). Totals include warmup.
- **Success vs failure:** a request ends as success or as a failure of one cause: `rejected` (refused at a node or edge), `timed_out`, `connection_reset`. `failedRequests` includes the rejected, timed-out and reset ones. [repo]
- **In flight:** requests not yet terminal at the cutoff are in none of the outcome counts; `totalRequests` counts requests that reached an outcome. [repo]

2. Run summary (`summary`)

Field	Definition	Unit
`totalRequests`	requests that reached an outcome (success or failure), whole run	count
`postWarmupTotalRequests`	requests created after warmup	count
`successfulRequests`, `failedRequests`	outcomes; `failed = rejected + timed out + reset`	count
`rejectedRequests`, `timedOutRequests`, `connectionResetRequests`	failure causes	count
`throughput`	successful requests / post-warmup duration (**goodput**, not offered load)	req/s
`errorRate`	post-warmup failed / post-warmup total	fraction
`latency`	p50, p90, p95, p99, min, max, mean of **successful** requests, end to end	ms
`latencyWindows`	per-second windows (by termination time): counts, error rate, p50/p95/p99, dominant error cause	
`latencyDecomposition`	mean end-to-end latency split per component (edge transit, queue, service), bottleneck first, with `shareOfEndToEnd`	ms, fraction
`failuresByLocus`	failures grouped by the node or edge that ended them, with cause and `shareOfFailures` (the "who killed my request" Pareto)	
`timeToErrorByCause`	latency of failed requests by cause (a fast refusal and a timeout wall differ)	ms

Field	Definition	Unit
`offeredArrivalCV`	coefficient of variation of the source's inter-arrival gaps: ~0 constant, ~1 Poisson	
`duration`, `postWarmupDurationSec`	run length, and the post-warmup window used as the rate basis	ms, s

Real-world reading: latency is the golden-signal latency of successes (failed requests are tracked separately, chapter 16); throughput is the golden-signal traffic that succeeded; `errorRate` is the SLI complement. [K]

3. Per node (`perNode[id]`)

Field	Definition
`totalArrived` / `postWarmupArrived`	requests that reached the node
`totalProcessed` / `postWarmupProcessed`	requests that completed service (success)
`totalRejected`, `totalTimedOut`, `totalConnectionReset` (and post-warmup)	failures at this node
`totalFailedAfterService`	finished service then failed (for example a node error rate); never counted as processed
`avgQueueLength`, `peakQueueLength`	mean and maximum waiting (not in service)
`avgQueueWait`, `avgServiceTime`, `avgTimeInSystem`	W_q, S and their sum, mean per processed request (ms)
`avgInSystem`, `peakInSystem`, `finalInSystem`	requests present (waiting + in service): sampled mean, peak, and at the end
`utilization`	occupancy of the **binding resource**: CPU for instance-backed nodes (two-tier model), worker pool for legacy nodes
`workerUtilization`	worker-pool occupancy (integral of active workers / integral of capacity); equals `utilization` for cpu-bound nodes
`throughput`	post-warmup processed / post-warmup duration
`errorRate`, `availability`	node-level failure fraction and 1 minus it
`arrivalCV`	variation of arrival gaps at this node; above the offered CV means the network bunched requests
`latencyP50/P95/P99`, `latencyWindows`	node-local latency of successful requests
`cacheHits`, `cacheMisses`, `cacheHitRatio`	cache lookups (chapter 10)
`rejectionsByReason`	counts keyed by reason code (`queue_full`, `node_failed`, `storage_full`, `bulkhead_full`, `no_healthy_targets`, ...)
`traitCounters`	per-feature counters (cache, replication, stream, telemetry, fan-out, windowing, ...)

Real-world readings: `utilization` is rho (chapter 01), `avgInSystem` is L, `avgQueueWait` is W_q, `avgTimeInSystem` is W; the Little's-law check relates them. [K]

4. Per edge (`perEdge[id]`)

`totalSuccessfulTransits`, `totalFailedTerminals`, `bytesTransferred` (post-warmup, for egress cost), `transitLatency` percentiles, `latencyBreakdown` (propagation, transmission, protocol, handshake, ...

summing to the mean), `linkUtilization` (time the serialising link was busy / window; 1 means running at bandwidth), `connections` (opened, resumed, reused, waited, closed idle), `slots` (in-flight cap waits and refusals) and `batching`. Meaning: chapter 08. [repo]

5. Checks the run performs on itself

Field	Definition	Pass rule
`littlesLawCheck[]`	per node: observed L (time-averaged) vs lambda x W, with lambda from admitted arrivals	within 10% relative or 0.05 absolute
`conservationCheck[]`	per node: arrived = processed + rejected + timed out + reset + in flight	balanced when in flight is under 5% of arrivals
`warmupAdequacy`	warmup should be at least 10 x the largest node p99	adequate or a recommended warmup
`sloBreaches[]`	node `slo` targets (`latencyP99` and `availabilityTarget`) exceeded; severity critical at 1.25x the target; `errorBudget` is validated but not checked	
`invariantViolations[]`	authored `invariants` conditions that failed	
`singlePointsOfFailure[]`	nodes whose loss disconnects the source from the rest	
`acceptedWorkConservation`	accepted = delivered + lost for asynchronous hand-offs	
`reproducible`, `seed`, `eventsProcessed`	same seed gives the same run	

6. Feature outputs

`streamProjection` (partitions, offsets, group lag; chapter 13), `replicationProjection` and `consistency` (stale reads, read-your-writes and monotonic-read violations, catch-up waits, linearizability; chapter 11), `clusterProjection` (machines, pending pods, recovery times; chapter 14), `perOrigin` (requests, errors, latency, cache hit by client population), `requestOutcomes[]` (one record per request with status, reason code, latency, type, cache outcome and a state timeline), `traces` (sampled spans), `timeSeries` (per-second per-node queue length, active workers, workers, utilization), `statusTimeline`, `causalGraph`, `eventStream`. [repo]

7. The verdict (for grading)

`SimulationVerdict` is the grading-facing projection of the output: `meta`, `summary`, `perNode`, `sloBreaches`, `invariantViolations`, `conservation`, `littlesLaw`, and the feature sections. Rubric metrics address it by dotted path (`summary.errorRate`, `summary.latency.p99`, `perNode.<id>.utilization`) or by derived aggregate (`perNode.maxUtilization`, `sloBreaches.count`, `invariantViolations.count`). A metric that cannot be measured is **null and fails its check** rather than passing silently. See chapter 29. [repo]

8. Where a name and a computation differ (known)

These come from the flows in `e2e/README.md` (findings), kept here so a reader of the output is not misled.

Metric	Caveat
latency of a leaf asynchronous branch	when the only edge out of a service is asynchronous, the request is reported as ending at the target, with that target's latency (finding 27)
`avgInSystem`	a 1 Hz sampled mean whose first sample (t = 0) is always an empty system, so it reads low by about 1 / duration in seconds; the Little's-law check uses the exact time average (finding 16)
`totalRequests`	counts requests that reached an outcome, not requests generated; in-flight requests at the cutoff are separate (finding 7)
`summary.latency` blended with branches	an asynchronous branch beside a synchronous path is a separate record in `requestOutcomes` and in the blended percentiles (finding 11)
`failuresByLocus`, `perOrigin`, `requestOutcomes`	the locus table counts a failure when it ends and `postWarmupFailedRequests` when its request began (a few differ at the warmup edge); `perOrigin` counts requests generated after warmup including those still in flight; `requestOutcomes` has a record with status `in-flight` for every request not yet terminal, so it is longer than `totalRequests`; a timed-out request has status `timeout` and no reason code
random-dependent numbers (block rates, hit rates)	the page and the engine draw different random numbers for the same seed, so they agree within sampling error (finding 30)

9. Testable expectations (summary)

- Reported throughput = successes / post-warmup duration; error rate = failures / total; they add up with the outcome counts.
- A reported percentile is the percentile of the underlying successful latencies (within the histogram's 1% resolution, from 1 microsecond to about 18 minutes).
- Utilization stays within 0 to 1; Little's law and conservation checks hold for a healthy run.
- Each failure appears under exactly one cause and one locus; `failuresByLocus` shares sum to 1.
- The same document and seed give identical output.

10. Open items

- Decisions `[?]`: report asynchronous branches separately from user-facing requests; make `avgInSystem` the exact time average.
- Fetch and cite: the Prometheus and OpenTelemetry metric naming conventions for the equivalent real-world signals.

28. Glossary

Marks: **[K]** established knowledge (definitions are standard unless a chapter cites a source). See README. One line per term, with the chapter that explains it. Terms that are simulator settings are in `code font`.

Term	Meaning	Ch
Abstraction (architect, guided, classroom)	how much of the simulator a learner works with	29
ACID	atomicity, consistency, isolation, durability of database transactions	09
Active-active / active-passive	all regions serve / one region serves and one waits	24
ADR	architecture decision record: title, status, context, decision, consequences	25
Amdahl's law	speed-up limit 1 / ((1 - p) + p / n) from the serial fraction	06
Anomaly score (WAF)	running total of rule matches; a request is blocked above a threshold	15
Asynchronous edge	the caller continues without waiting for the response	08, 13
At-least-once / at-most-once / exactly-once	delivery guarantees: may duplicate / may lose / effect once	13
Availability	fraction of time or requests served; series multiply	02, 20
Backpressure	a bounded buffer pushing back on producers	02, 13
Bandwidth-delay product	bytes in flight to fill a link: bandwidth x round-trip time	08
Blackhole	failure where packets vanish and the client waits its timeout	22
Blast radius	share of users or capacity one fault can affect	22, 24
Bloom filter	compact set test with false positives, used to skip files in an LSM store	09
Bulkhead	per-compartment concurrency cap so one noisy workload cannot take every worker	02
Burn rate	error-budget consumption speed relative to the SLO	16
Cache stampede	many concurrent misses for one key hitting the backend	10
Canary	releasing to a small share of traffic first	17
CAP / PACELC	consistency, availability, partition tolerance trade-off, and its latency extension	11
Cardinality (metrics)	number of distinct label combinations	16
CDC	change data capture: a stream of row changes	13
Cell-based architecture	isolated full copies of a workload behind a router	24
Che approximation	closed-form LRU hit ratio under independent requests	10
Circuit breaker	stops calling a failing dependency, then probes	02
Coefficient of variation (CV)	standard deviation / mean; 0 deterministic, 1 exponential	01, 03
Cold start	extra latency on first use (serverless, cold cache, restarted node)	06, 22
Compensating transaction	an action that undoes a completed saga step	24
Connection pool	a bounded set of reusable connections	08, 09
Consistent hashing	keys and nodes on a ring so adding a node moves about 1/(N+1) of keys	07, 12
Consumer group	consumers sharing a stream's partitions; each group reads everything	13

Term	Meaning	Ch
CQRS	separate models for commands and queries	24
Dead-letter queue (DLQ)	where messages go after too many failed deliveries	13
Degraded	serving slowly or partly rather than down	22
Discrete-event simulation (DES)	advancing a clock from event to event	04
Diurnal	daily cycle of load	05
DNS TTL	how long an answer may be cached	21
Envelope (evaluation)	frozen, checksummed record of one graded submission	29
Erlang B / C	blocking probability / probability of waiting in M/M/c	01
Error budget	1 - SLO: the failures allowed over the window	16
Event sourcing	the event log is the system of record	13
Eventual consistency	replicas converge if writes stop	11
Failover	switching to a standby	11, 21, 22
Fan-out (on write / on read)	copying a post to followers' feeds when written / assembling at read	24
Fault domain	a region, zone or subnet that fails together	22
Fencing token	increasing number that lets a store reject a stale lock holder	11
Golden signals	latency, traffic, errors, saturation	16
Goodput	successful work per second	27
Gray failure	failure the health checker does not see but applications do	22
Hash ring	the structure behind consistent hashing	12
Head / tail sampling	decide at the start of a trace / after it completes	16
Headroom	unused capacity, 1 - utilization	23
Health check	probe that removes a failing target; detection = interval x failures	02, 07
Hedged request	duplicate request sent after a delay to cut tail latency	12
Hot key / hot shard	one key or shard carrying disproportionate load	12
HPA	Kubernetes horizontal pod autoscaler: ceil(current x metric / target)	14
Idempotency key	client token that makes a retried request a no-op	20
Idempotent	repeating has the same effect as doing it once	13, 20
In flight	requests accepted but not finished	27
Invariant	a condition that must hold over a run	26, 29
IOPS	I/O operations per second a device can do	09
Kingman's formula	queue wait approximation for G/G/1	01
Lag (replication / consumer)	how far behind a follower or consumer is	11, 13
Latency percentile (p50, p95, p99)	value below which that fraction of requests fall	01, 16
Leader / follower	the replica that accepts writes / those that copy it	11
Little's law	L = lambda x W	01
Load balancer (L4 / L7)	spreads requests across targets by connection / by request content	07
Load shedding	rejecting work early to protect the rest	02
LRU / LFU / TTL	evict least recently used / least frequently used / by age	10

Term	Meaning	Ch
LSM-tree	write-optimised store: memtable, sorted files, compaction	09
M/M/1, M/M/c, M/M/c/K, M/G/1, M/D/1	queue notation: arrivals/service/servers/capacity	01
Mathis bound	TCP throughput limit from loss and round-trip time	08
Metastable failure	overload sustained by a feedback loop after the trigger ends	22
Microservices	many small independently deployed services	24
Monotonic reads	a session never sees older data than it already saw	11
MTBF / MTTR	mean time between failures / to repair (declared in `failureModes`, not read by the engine)	26
N+1 / N+2	capacity to survive one / two failures	23
NFR	non-functional requirement: a latency, availability or throughput target	29
Observability	asking new questions of a system from its outputs	16
Open / closed workload	arrivals independent of / dependent on completions	05
Outbox (transactional)	message written in the same transaction as the data, relayed later	24
P-K formula	Pollaczek-Khinchine mean wait for M/G/1	01
Paranoia level	how many WAF rules are active	15
Partition (network)	a split in which nodes cannot talk	11
Partition (stream)	an ordered log segment; consumers get whole partitions	13
Poisson process	memoryless arrivals at a constant rate	03, 05
Power of two choices	pick two at random, take the less loaded	07
Priority / WFQ	queue disciplines: by priority / weighted fair queueing (`queue.discipline`)	26
Quorum	majority agreement; R + W > N overlaps	11
Raft	consensus protocol with leader election and a replicated log	11
Rate limiter (token bucket / leaky bucket)	caps request rate	02
RBAC / IAM	role-based access control / identity and access management	15
Read-your-writes	a session sees its own writes	11
Reject	instant refusal (a fast failure)	22
Replay (trace / log)	re-running recorded arrivals / re-reading a log	05, 13
Retry budget	cap on retries as a fraction of traffic	02, 22
Rho (utilization)	arrival rate x service time / servers	01
RPO / RTO	data loss tolerated / time to recover	24
Saga	a sequence of local transactions with compensation	24
Saturation	how full the most constrained resource is	16, 27
Scatter-gather	query all shards and merge; latency is the slowest, mean mu x H_n	12
Seed	random-number start value making a run reproducible	04, 26
Service mesh / sidecar	proxies beside services doing TLS, retries, telemetry	07, 24
SFU / MCU / mesh	video topologies: forward / mix / peer to peer	19
Shard / shard key	a partition of data / the field that picks it	12
SLI / SLO / SLA	measured indicator / internal target / contractual promise	16

Term	Meaning	Ch
Snapshot (event store)	saved state that bounds replay	13
SPOF	single point of failure	24, 27
Spot instance	discounted capacity the provider can reclaim	23
Stale read	a read that misses a newer write	11
Strangler fig	gradual replacement of a legacy system	24
Structural rule	a graph-shape requirement checked before simulation	29
Thundering herd	many clients acting at once (reconnect, cache expiry)	10, 19, 22
Timeout	the longest a caller waits	02
Tumbling / sliding / session window	fixed non-overlapping / overlapping / gap-delimited windows	18
USL	universal scalability law: contention and coherency limit scaling	06
Utilization	share of capacity in use	01
Visibility timeout	how long a received message stays hidden before redelivery	13
Warmup	initial period excluded from metrics	04
Watermark	stream progress marker for event time	18
Write amplification	bytes written to disk per byte of application write	09
Zipf distribution	popularity where item r has weight r^(-s)	05, 10, 12

29. Question and grading concepts

Marks: **[P]** primary, **[S]** secondary, **[K]** established knowledge, **[?]** needs an owner decision, **[repo]** read from the question and grading types (`src/engine/analysis/question.ts`, `rubric.ts`, `structural.ts`, `gradingCriteria.ts`, `evaluationEnvelope.ts`, `src/engine/classroom/`). See README.

How a design question is described, run and graded. Like chapters 26 and 27 this documents the simulator's own contract; what makes a good question comes from assessment practice (section 8). The numbers a rubric checks are chapter 27; the decisions a question asks for are chapter 25.

1. A question package

A `QuestionPackage` (version 1.0) carries: [repo]

Part	Content
identity	`id`, `title`, `description`, `difficulty` (beginner, intermediate, advanced, expert), `tags`, `estimatedTimeMinutes`, `author`
`type`	the grading archetype: `fix`, `build-budget`, `optimize`, `open-build`, `scaling`, `ha-chaos`, `tradeoff`
`entryFormat`	how the learner starts: `blank-canvas`, `requirements-first`, `partial-scaffold`, `broken-scaffold`, `baseline-optimize`, `locked-lab` (orthogonal to `type`)
`prompt`	the statement (markdown), functional requirements, non-functional targets (NFRs), scale parameters
`scaffold`	the starting topology (`empty`, `partial`, `complete`), locked node and edge ids, and for optimize questions a baseline verdict to beat
`constraints`	allowed or forbidden node types, node-count and worker caps, budget, whether the scaffold may be changed or removed
`structuralRules`	graph-shape requirements (section 3)
`semanticCriteria`, `justify`	meaning of the graph and graph-consistent written reasons (sections 4 and 5)
`budget`	a cap on `cost`, `nodes` or `edges` (the anti-kitchen-sink axis)
`workloadCategory`, `domains`, `concepts`	what the question is about: `read-heavy`, `write-heavy`, `connection-heavy`, `correctness-heavy`, `batch-heavy`; bottleneck domains (compute, storage, network, resilience, correctness, cost); lesson-level concept slugs
`suite`	the grading cases (section 2)
`rubric`	scored checks (section 3)
`classroom`	settings for concept-first modes (section 7)

NFR targets state what the design must achieve in numbers: `latency_p99`, `latency_p50`, `availability`, `error_rate`, `throughput`, each with an operator, a value and a unit (`ms`, `percent`, `req_per_sec`, `nines`). Scale parameters: daily active users, peak requests per second, read/write ratio, storage, retention, growth. [repo]

2. The suite: grading cases

A **case** is a set of **overrides** (`global`, `workload`, `faults`, `invariants`, optionally a chaos `experiment`) applied to the learner's topology; it carries no topology of its own because the design under test is the learner's. A suite is a named list of cases; `visibleToStudent` false makes it a hidden contest suite, and `dryRunCase` is a representative case the learner can run. [repo]

The stated scale is the load. Where a case sets no rate of its own, the run uses the question's `prompt.scale.peakRps` as the base rate (and `readWriteRatio` as the read/write mix) in place of whatever the learner configured, so a design cannot be graded at a lower load than the problem states; a case's own `workload` override wins. [repo]

Typical cases: the **steady** load, a **peak** or spike, a **fault** (a node or zone down), a **scaled** load. Running several cases catches designs that pass only the easy one. [K]

Grading invariants (for example `perNode.maxUtilization <= 0.8`) are declared in a case, since learners never author them, and checked through an invariant rubric check. [repo]

3. Rubric, metrics and structural rules

Rubric. A list of `RubricChecks {id, description, kind, metric, op, value, points}` and a `passThreshold` (fraction of points required, default 1). Kinds: `topology` (about the graph), `simulation` (about the run) and `invariant` (about invariants and self-checks). Operators `<`, `<=`, `>`, `>=`, `==`, `!=`. [repo]

Metric selectors:

- simulation: dotted paths into the verdict (`summary.errorRate`, `summary.latency.p99`, `perNode.<id>.utilization`) and aggregates `perNode.maxUtilization`, `perNode.maxErrorRate`, `perNode.maxLatencyP99`, `sloBreaches.count`;
- invariant: `invariantViolations.count`, `conservation.unbalanced`, `littlesLaw.violations`;
- topology: `topology.nodeCount`, `edgeCount`, `sourceCount`, `totalWorkers`, `totalReplicas`, and others.

Honesty rules: a metric that cannot be measured (an unknown path, or a percentile with no successful samples) is **null and the check fails** with a note; a case whose execution did not complete fails its checks as skipped; a hidden execution failure is itself a check (`__execution__`). [repo]

Structural rules are checked on the graph before any simulation and can short-circuit grading: `requires_component`, `requires_category`, `requires_edge` (from type to type, optionally by mode), `requires_path`, `requires_redundancy` (minimum replicas), `max_component_count`, `forbids_component`, `requires_connected_graph`, `requires_single_source`, `min_node_count`, `max_node_count`. [repo]

4. Semantic criteria

Checks on what the graph **means**, not just its shape, so a design cannot pass by adding names: `componentPresence`, `componentProperty` (a setting meets a condition), `placement`, `guardedPath` (a protection such as a limiter or breaker sits on the path), `fanout`, `storageFit` (the store suits the access pattern), `forbidUnjustified`, `stateTransition` and `stateSequence` (runtime states that requests actually went through, such as a redelivery or a dead-letter). [repo]

5. Justification and anti-gaming

A `justify` prompt asks the learner to defend a decision bound to a real graph element ("why this database type?"). It is graded **structurally**: the answer must reference the node or type actually chosen (graph-consistency, the core defence against stuffing), optionally cite a scale number the question defines, and state a trade-off. [repo] Other anti-gaming axes: the **budget** cap, **forbidden** and **allowed** node types, **node-count** limits, **hidden** suites, and **anti-pattern** detection. [repo]

6. The attempt, the grade and the envelope

- **Attempt states:** `DRAFT`, `AUTOSAVED`, `SUBMITTED`, `GRADING`, `GRADED`, `LOCKED`. [repo]
- **Grade (`AttemptGrade`):** `structural`, `constraints`, `semantic`, `justification`, `budget`, `baselineComparison` (optimize questions: the metrics against the baseline), `experiments` (chaos), `classroom`, the full `graded` batch (rich data stays inside the simulator) and a collapsed `contract` (booleans) sent to the host across the iframe seam. [repo]
- **Evaluation envelope:** the frozen record of one submission: a snapshot of the exact topology, the per-case verdicts, a bounded **replay digest** (counts and an event-stream checksum), the contract, and a content **checksum** so the grade can be audited and reproduced from the envelope alone. The checksum is an integrity primitive, not a signature. [repo]
- **Determinism:** the same topology, case and seed give the same verdict, so a grade is reproducible (chapter 04).

7. Classroom abstractions

The same package can be worked at three levels: **architect** (the full simulator, today's grading), **guided** and **classroom** (concept-first modes). A `classroom` block states the numbers a problem gives about each component in human units (requests per second per copy, dollars per hour, a fixed delay), lecture labels, targets, controlled choices, guided knobs and plain-language checks. In those modes the result includes **feedback-only** checks (shown to the learner, never part of the score or host contract), **issues** for components the mode cannot model (any issue fails the structural gate) and **misconceptions**: teaching feedback matched to a typical mistake, presented in priority order. [repo]

8. What makes a good question (assessment practice)

- **Discrimination:** the reference solution passes every case, and each plausible wrong design fails at least one named check. A question that every design passes, or none, teaches nothing. [K]
- **One lesson per check:** each check tied to a concept, so a failure points at a misunderstanding. [K]
- **Numbers from the statement:** targets derived from the stated scale, with the arithmetic in the question's own explanation (chapters 23 and 24). [K]
- **Hidden variation:** a suite case that differs from the dry run (a different rate or fault) so a design tuned to the visible case fails. [K]
- **Misconception coverage:** common errors (no cache on a read-heavy path, a synchronous call to a slow dependency, retries without a budget, a single point of failure) each have a check or a feedback rule. [K]
- **Graph consistency:** written reasons must refer to what is actually built. [repo]

The authored question bank in this repository is validated both ways: every reference solution passes and every deliberately wrong design fails.

9. Testable expectations (summary)

- An unmeasurable metric fails its check; it never passes by default.
- A structural-rule failure stops grading before simulation and says which rule.
- The same submission and seed give the same grade and the same envelope checksum; changing the topology changes the checksum.
- A reference solution passes all checks; a design that removes the required protection fails the matching check.
- A justification that names a component not in the graph fails the graph-consistency test.
- A budget or node-count cap is enforced independently of performance.
- Feedback-only classroom checks never change the score.

10. Open items

- Cite: published guidance on constructed-response and performance-based assessment; the concept registry of lesson slugs.
- Decisions `[?]`: whether `errorBudget` (validated but unchecked) should be a rubric selector; whether discrimination should be measured automatically for every authored question.